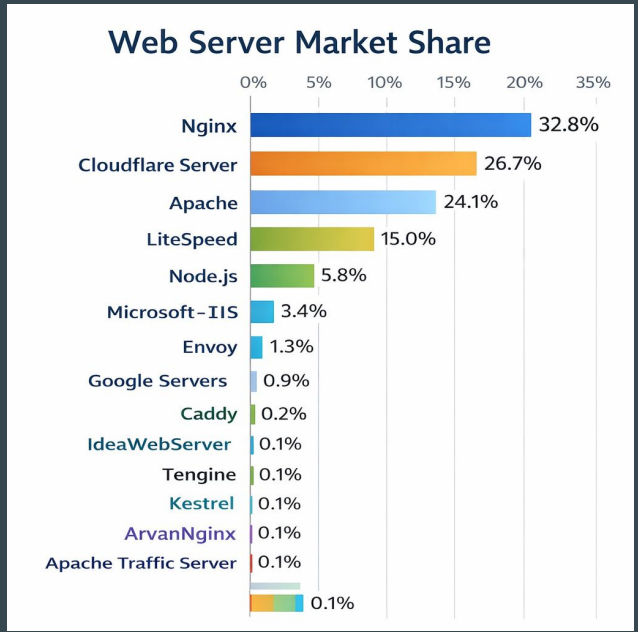
Nginx - Beginner to Advanced

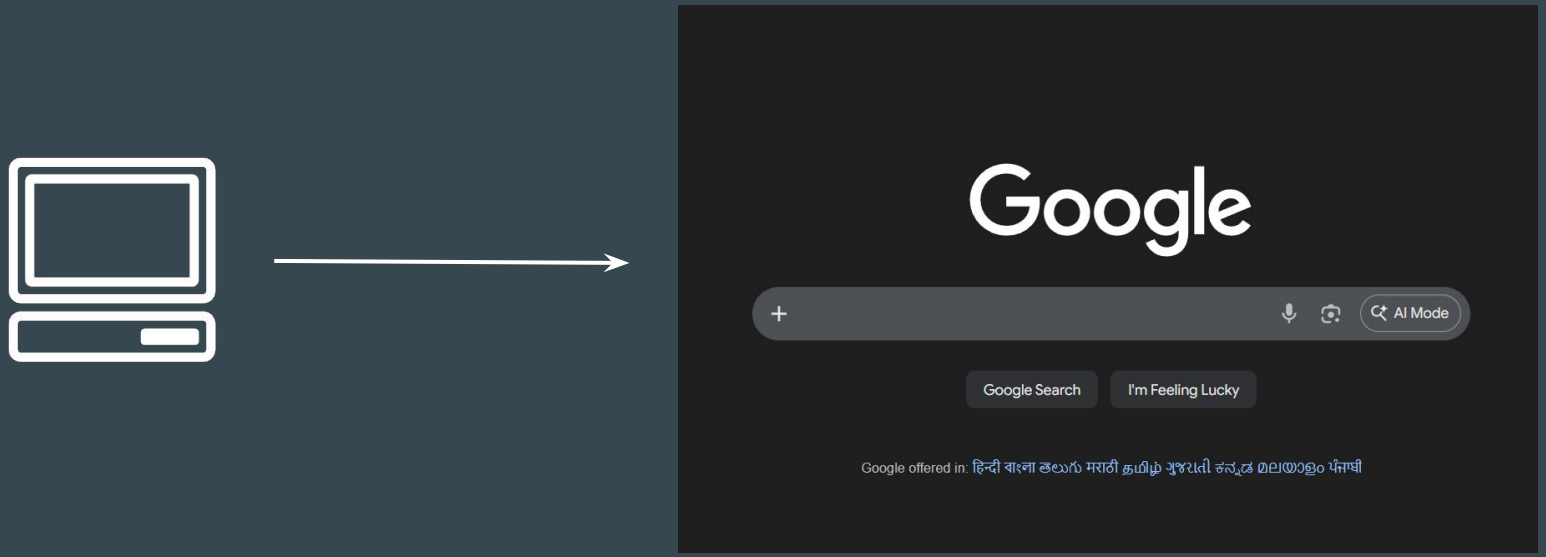
Setting the Base

NGINX is the most widely used web server on the internet.

According to W3Techs surveys on web servers they know, Nginx is used by 32.8% of all the websites.



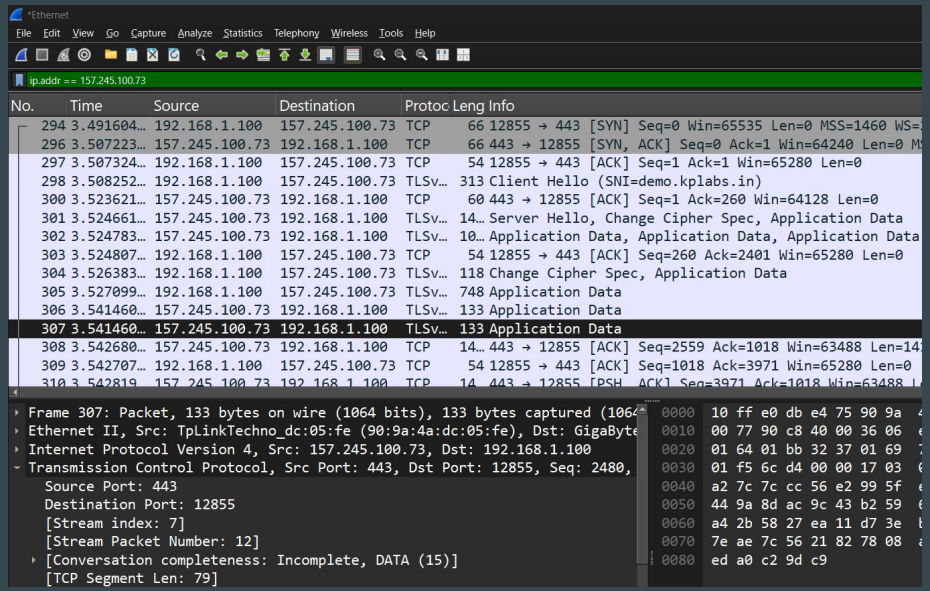
What Happens Behind the Scenes?



The Actual Behind the Scenes

Behind the scenes, there is entire new world :

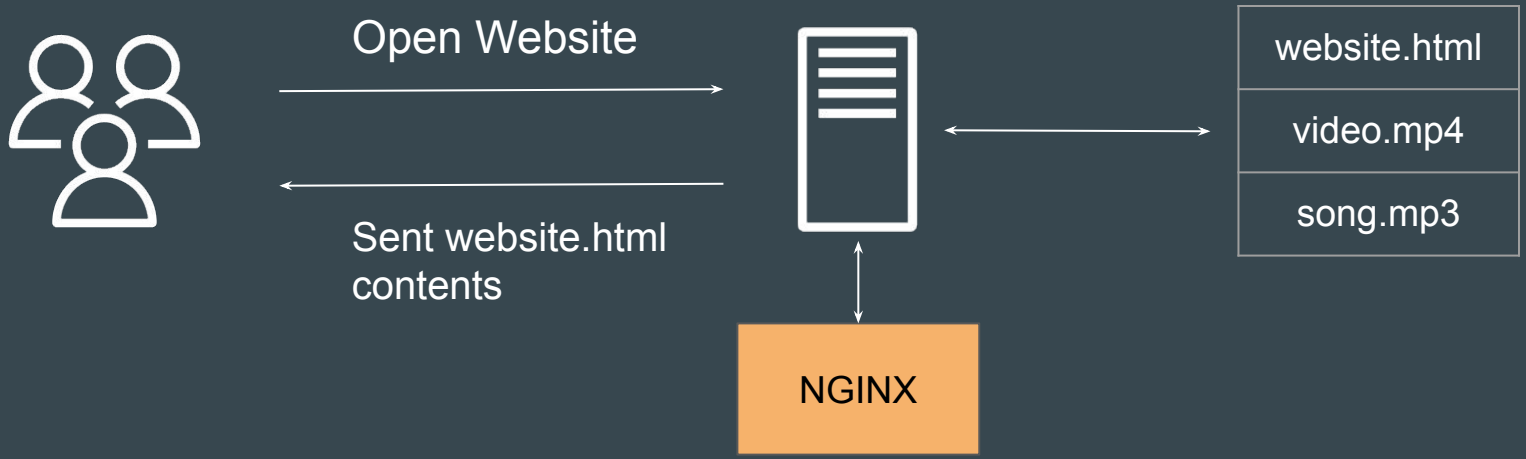
- DNS Lookup
- TCP 3 way handshake
- TLS handshake
- HTTP GET Request
- HTTP Response



Overview of Nginx

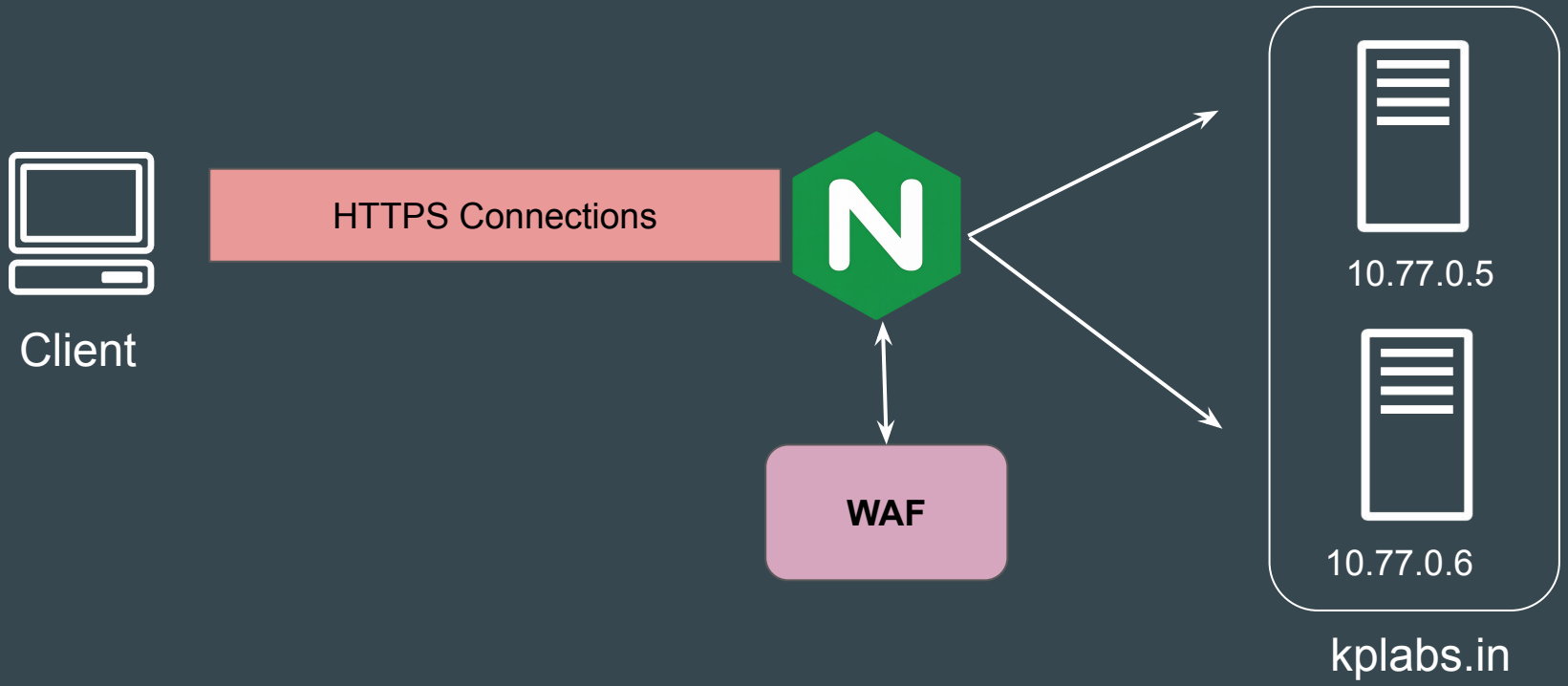
Nginx is one of the most popular web-server available in the industry.

Known for efficiently handling thousands of simultaneous connections with low resource usage, making websites faster and more reliable.



Learnings on This Course

In this course, you will learn about Nginx from absolute basics, and as the course progresses, we will discuss advanced features in detail.



What is this course all about?

This course will help you build a very strong foundation in Nginx

At the end of the course, you will learn how to:

1. Deploy Nginx for Production Environments.
2. Perform Load Balancing for High-Availability.
3. Protect your websites against attacks by Hackers.
4. Optimize Nginx for Specific Use-Cases

About Me

- DevSecOps Engineer - Defensive Security.
- Teaching is one of my passions.
- I have total of 18+ courses, and around 450,000+ students now.

Something about me :-



- RedHat Certified Architect, Certified Ethical Hacker.
- AWS Certified [Advanced Networking, Security Specialty, DevOps Pro, SA Pro, …]
- HashiCorp Certified [Terraform Professional [Vault and Consul Associate]
- Part time Security Consultant + 13 more Certifications

Join us in our Adventure

Be Awesome



kplabs.in/chat



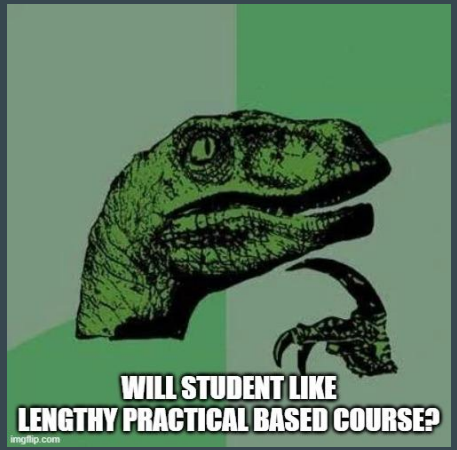
kplabs.in/linkedin

About Course and Important Resources

2 Choice for Instructor

1. Cover Theory and Basic Demos -- Course Length Maintained

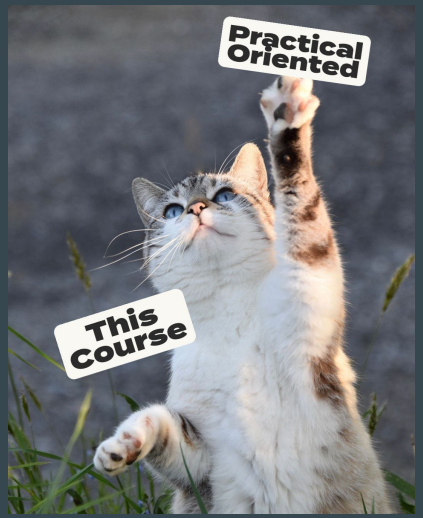
2. Cover Practically -- Course Length will be Longer



Our Choice

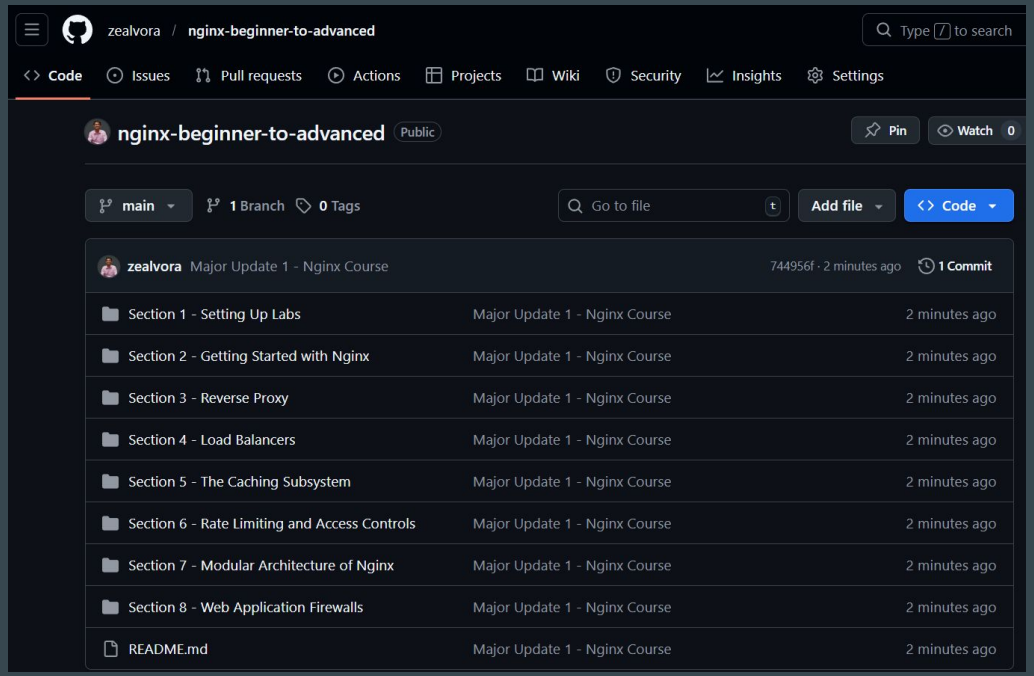
This course follows practical based approach.

The course will be lengthy but worth it.



Course Resource - GitHub

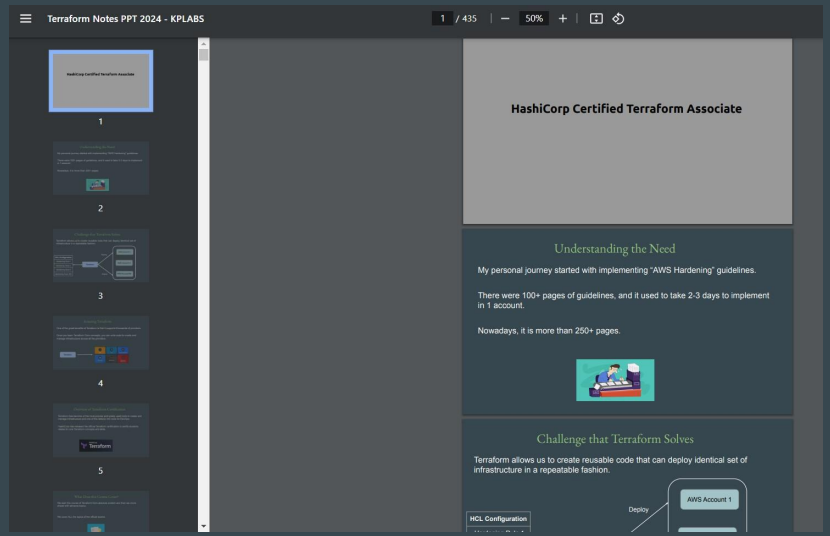
All the code that we use during practicals is available on our GitHub page.



Course Resource - PPT Slides

All the slides we use in this course are available to download as PDFs.

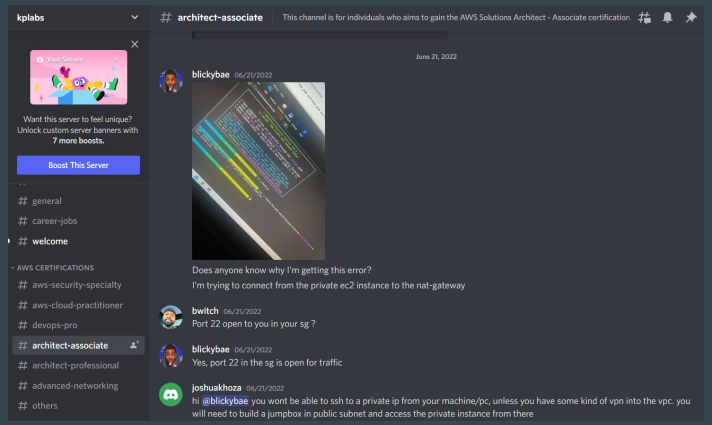
The PDF is attached as part of the lecture titled “Central PPT Notes”.



Our Community (Optional)

We also have a Discord community where individuals preparing for the same certification can connect, discuss, and seek technical support.

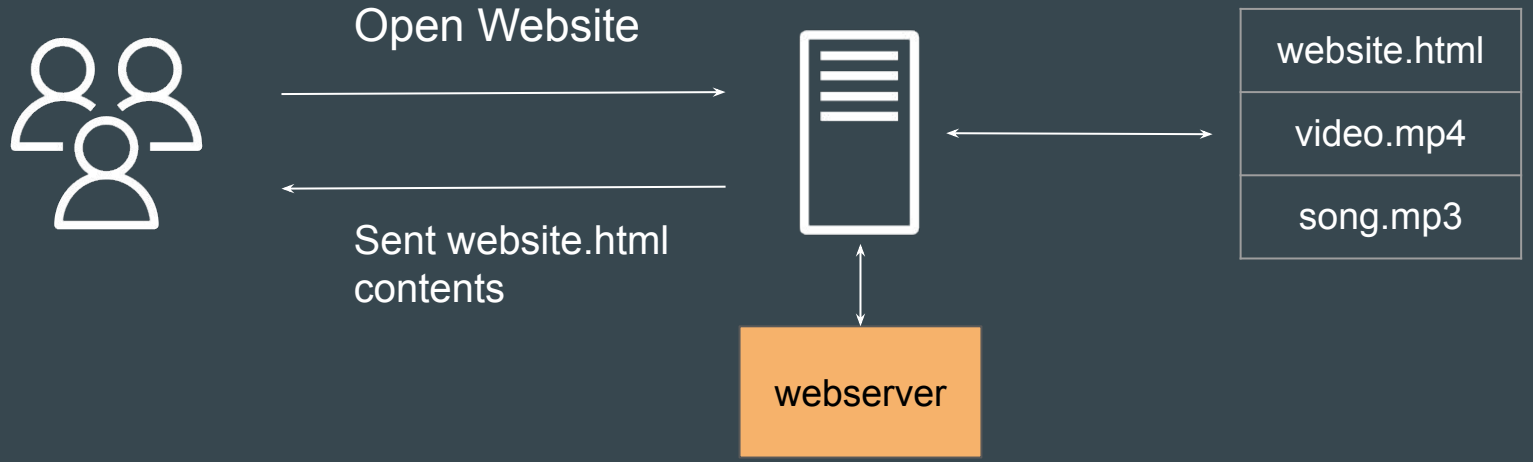
https://kplabs.in/chat



Introduction to Nginx

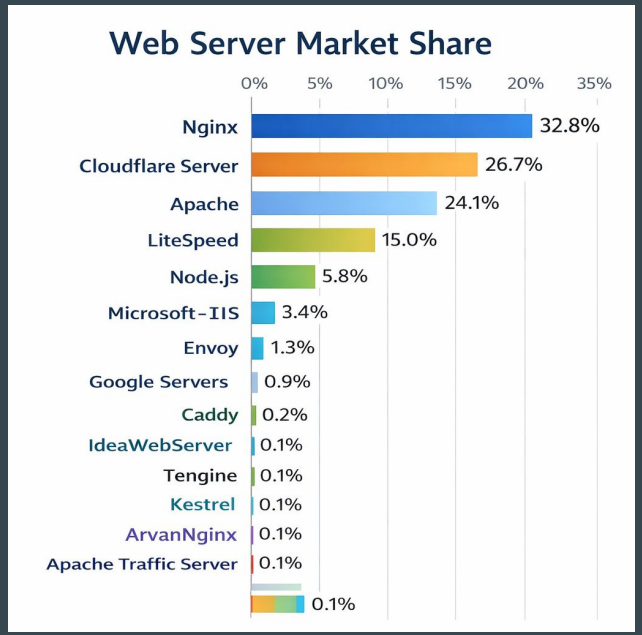
Basics of Web Servers

A web server is a software that delivers web content—for example, HTML pages, files, images, video—primarily in response to HTTP requests from a web browser.



Popular Web Servers

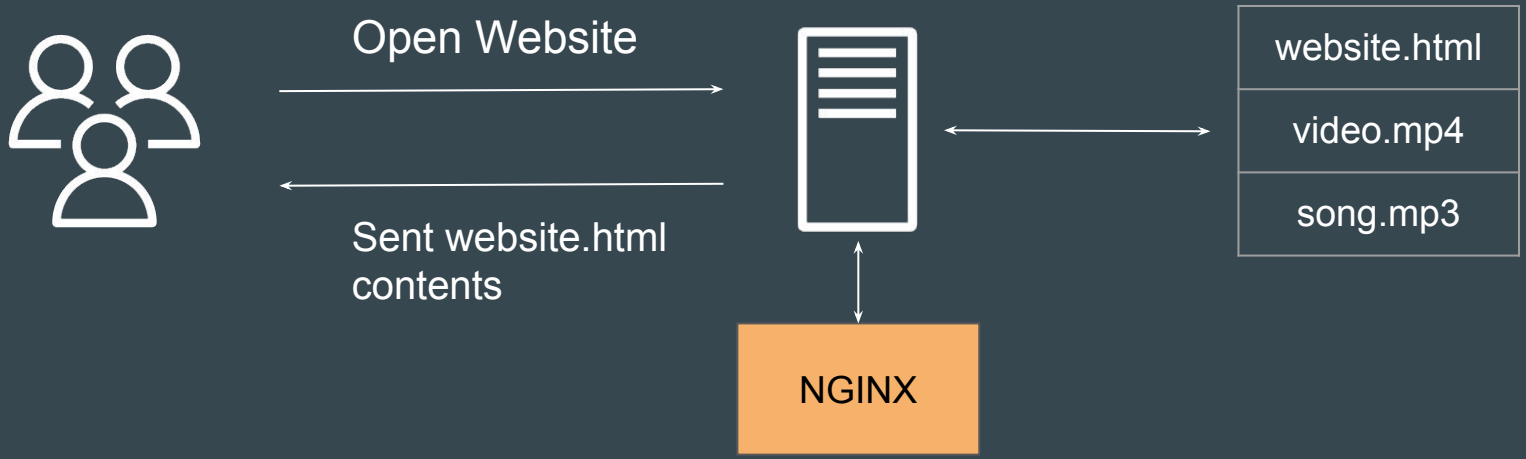
There are many popular web-servers like Apache, Nginx, IIS available in the industry.



Overview of Nginx

Nginx is one of the most popular web-server available in the industry.

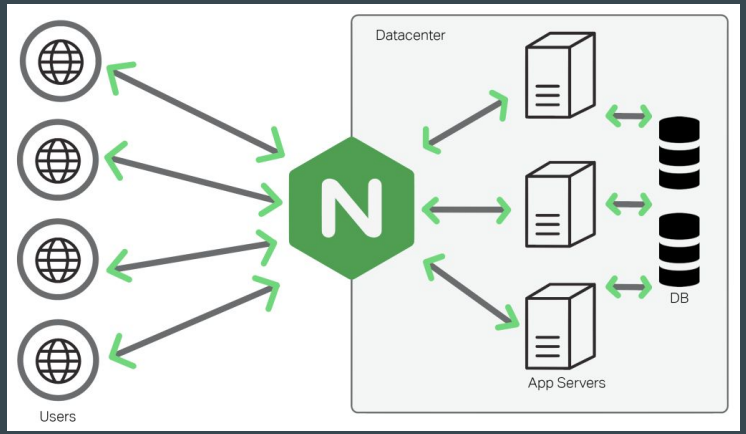
Known for efficiently handling thousands of simultaneous connections with low resource usage, making websites faster and more reliable.



Nginx is More then Web-Server

Nginx has much more capability then a typical web-server.

It can act as a reverse proxy, do load-balancing, mail proxy, HTTP caching, serve static assets etc.

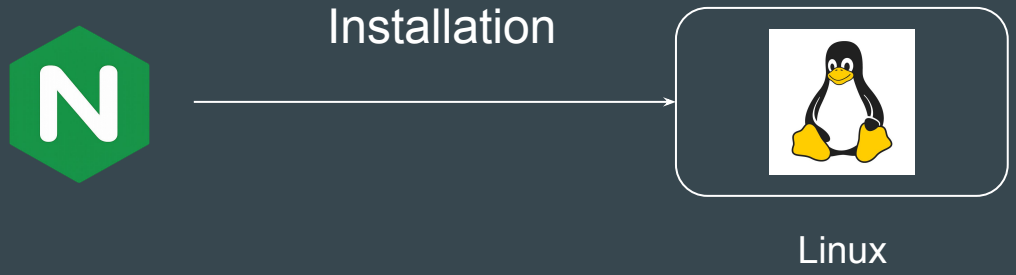


Infrastructure Setup Options for Nginx

Supported Operating System

Linux is the primary supported OS for Nginx.

It is supported on almost all major distributions.



How to Create a Linux VM?

There are multiple approaches to create a Linux based VM.

1. Make use of VMWare Workstation / VirtualBox.

2. Create a Linux VM in Cloud (AWS, Azure, GCP, Digital Ocean etc).

Approach 1 - Use Virtualization Products

- Install VMware Workstation / VirtualBox.
- Create New VM based on Linux.
- Install Nginx.



Approach 2 - Use Cloud Provider

- Choose a Cloud Service Provider of your choice.
- Launch a new VM based on Linux
- Install Nginx.



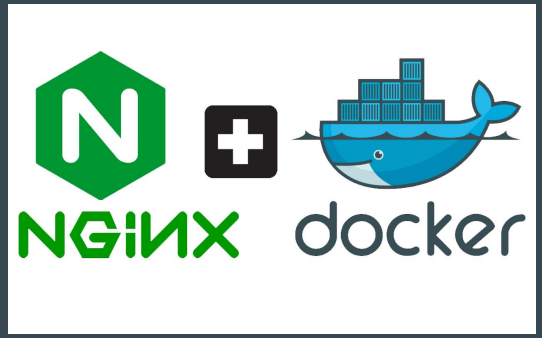




Approach 3 - Use Docker (Only for Experienced)

- Install Docker in your laptop.

- Launch Nginx Docker container.



Lab Setup - Nginx

Supported Operating System

Linux is the primary platform for Nginx.

It is supported on almost all major distributions.



Revising the Choices

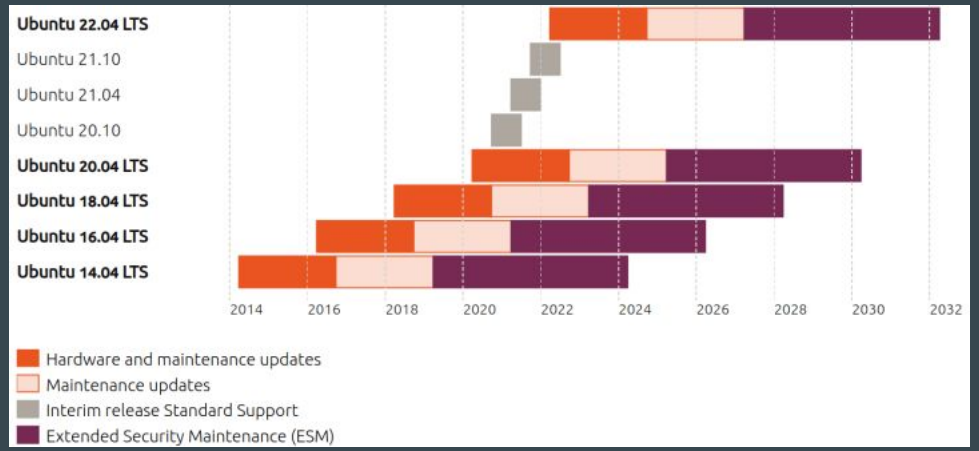
Following diagram illustrates the architecture that we will be following in this course:

Criteria	Choices
Operating System	Ubuntu LTS (Latest)
Provider	Digital Ocean (Optional)

Point to Note - OS

It is recommended to use the same OS version that we use in practicals.

We have intentionally used Ubuntu LTS (Long Term Support)



Why Digital Ocean?

1. Reasonably priced servers and pay per hour.

2. Good Amount of Credits for New Users (Referral) - $100 - $200

3. Keep it simple approach.

Nginx Versions - Mainline vs Stable

Setting the Base

NGINX Open Source is available in two versions: Mainline and Stable.

Mainline	Mainline version is the latest development version, updated approximately every 1 or 2 months, includes the latest features, bug fixes and security fixes. This version is recommended for production unless your organization has strict requirements for stability
Stable	Stable version is updated typically once a year or as needed for critical bug fixes or security fixes that are always backported from the Mainline version. This version is recommended for environments with strict requirements for stability.

Mainline vs Stable

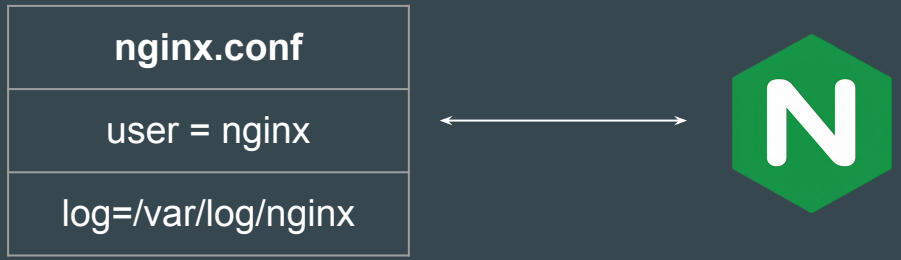
Feature	Mainline	Stable
Primary Goal	Active development & new features.	Production-focused version with only critical bug fixes
Release Frequency	Every 1-2 months	Typically once per year
Testing Level	Well tested but newer changes may still evolve	Extremely tested and conservative
New Features	Included immediately (e.g., HTTP/3, QUIC).	None added during the release cycle.
Recommended For	General production, edge servers, and latest tech.	Mission-critical servers with 3rd-party modules.

Nginx Configuration File Architecture

Setting the Base

The way Nginx works is determined by its configuration file.

By default, the configuration file is named nginx.conf and located in /etc/nginx



Structure of nginx.conf



nginx.conf

Main Context

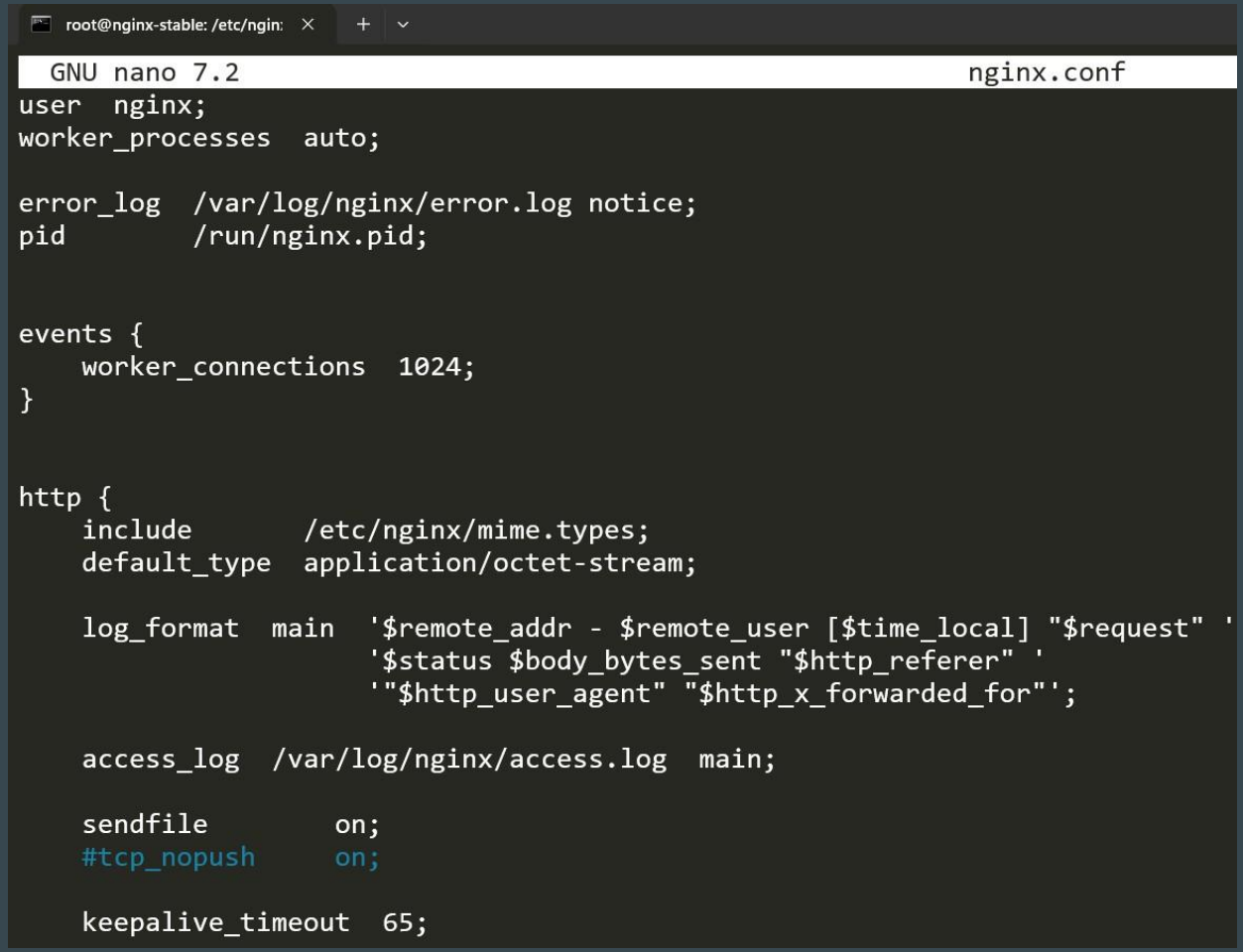
```
user  nginx;
worker_processes  auto;
```

Events Block

```
events {
    worker_connections 1024;
}
```

HTTP Block

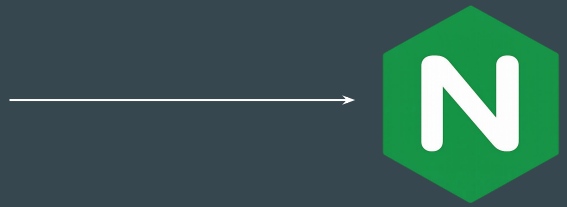
```
http {
    include mime.types;
    server {
        listen 80;
        server_name example.com;
        location / {
            proxy_pass http://localhost:3000;
        }
    }
}
```


root@nginx-stable: /etc/ngin:
GNU nano 7.2 nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;
events {
worker_connections 1024;
}
http {
include /etc/nginx/mime.types;
default_type application/octet-stream;
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
'$status $body_bytes_sent "$http_referer" '
'"$http_user_agent" "$http_x_forwarded_for"';
access_log /var/log/nginx/access.log main;
sendfile on;
#tcp_nopush on;
keepalive_timeout 65;

Basic Context

The main nginx configuration file is divided across range of contexts (sections).

Contexts
main
events
http
mail



Points to Note

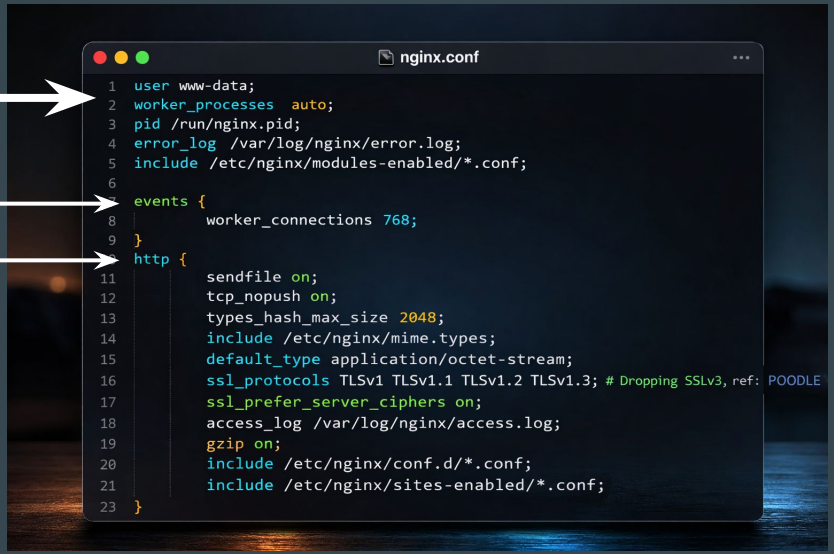
If a block directive can have other directives inside braces {}, it is called a context (examples: events, http, server, and location).

Directives placed in the configuration file outside of any contexts are considered to be in the main context.

Main context directives

Events Context

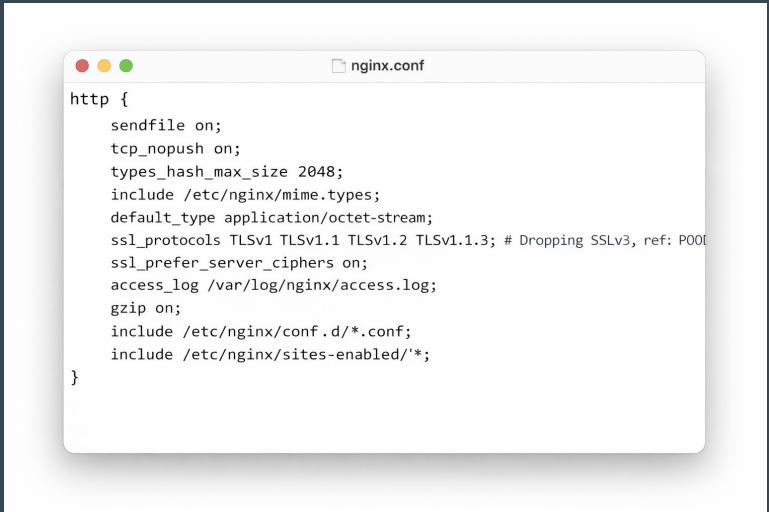
Http Context



HTTP Context

This is the "parent" context for all web-related traffic and defines how Nginx handles the HTTP protocol.

You will extensively be working with the HTTP context in production environments.

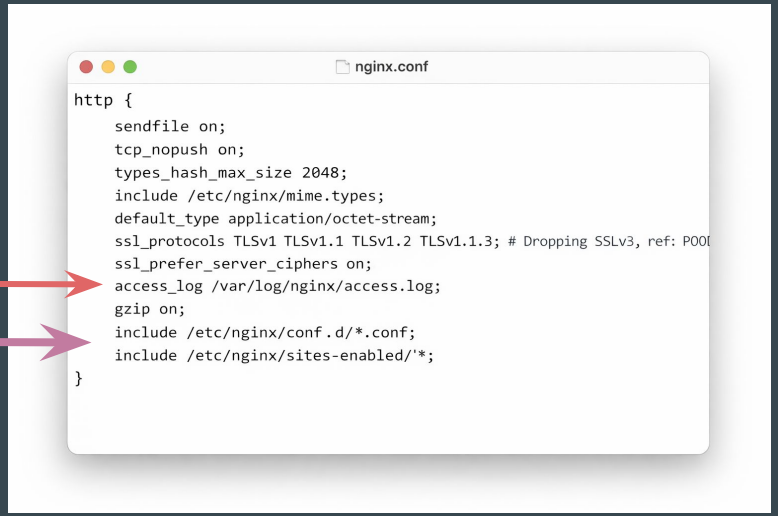


HTTP Context

HTTP Context

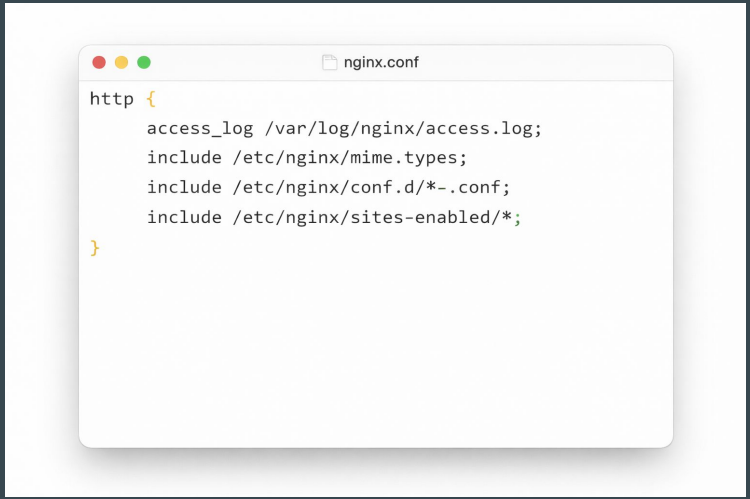
The http {} wraps around almost everything that defines how your websites behave, from caching and logging to defining individual/multiple websites.

You will extensively be working with the HTTP context in production environments.



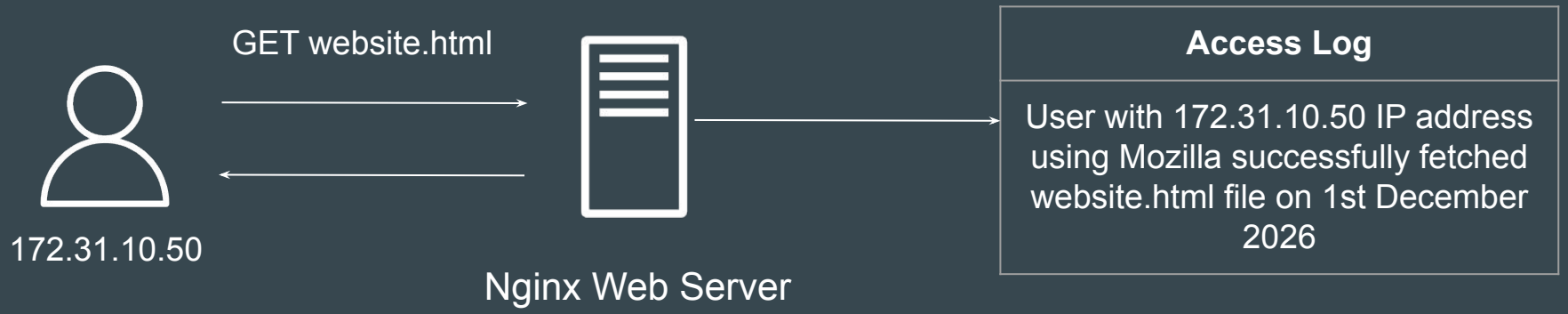
Important Directives

Directives	Description
access_log	Defines the path where access log files are stored
include	Used to embed the contents of other files into the main configuration, making the configuration easier to maintain and organize.

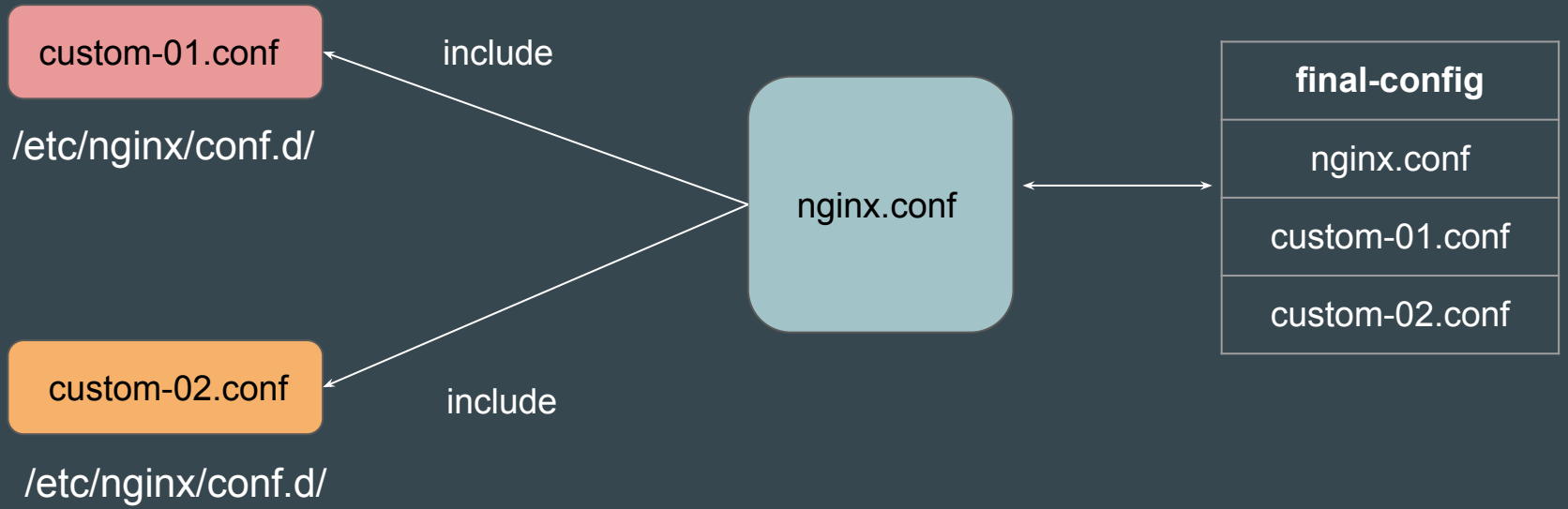


Access Logs

Nginx Access Logs record all HTTP requests with client details, response codes, and timing,etc.



Include Directive Workflow



Point to Note

A single Nginx instance can handle single as well as multiple websites.

When you place directives inside the http { ... } block, they apply to every website hosted on that Nginx instance.

Server Blocks

Server Context

A server block is the configuration section that defines how Nginx handles requests for a specific website, domain, or virtual host.



```
http {
  server {
    listen         80;
    server_name    example.com;

    location / {
        root       /usr/share/nginx/html;
        index      index.html;
    }
  }
}
```

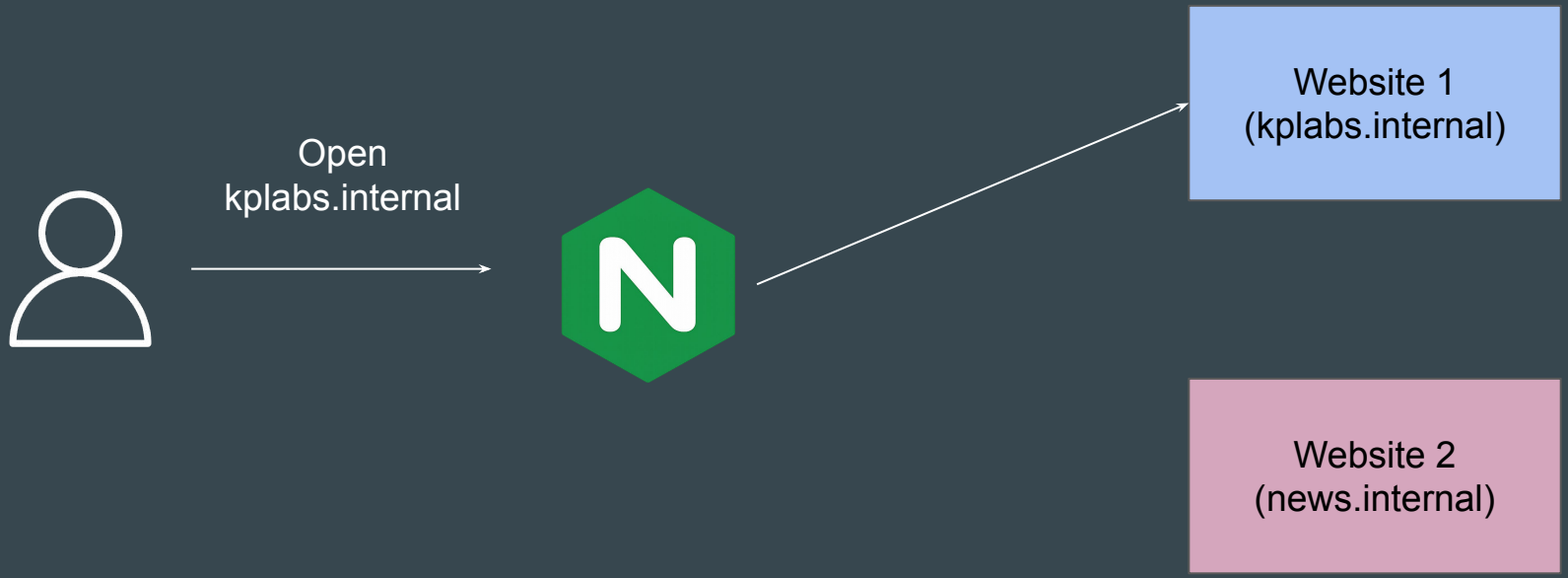
Important Directives

Directives	Description
server_name	Defines the domain names or hostnames handled by this server block
listen	Defines the IP address and port (e.g., 80 for HTTP or 443 for HTTPS) the server responds to.
root	Defines the file system directory where the website's files are stored.
index	Specifies default files served when a directory is requested. Example: index.html

Overview of Virtual Hosts

Setting the Base

Nginx allows users to host multiple websites easily.



Configuration

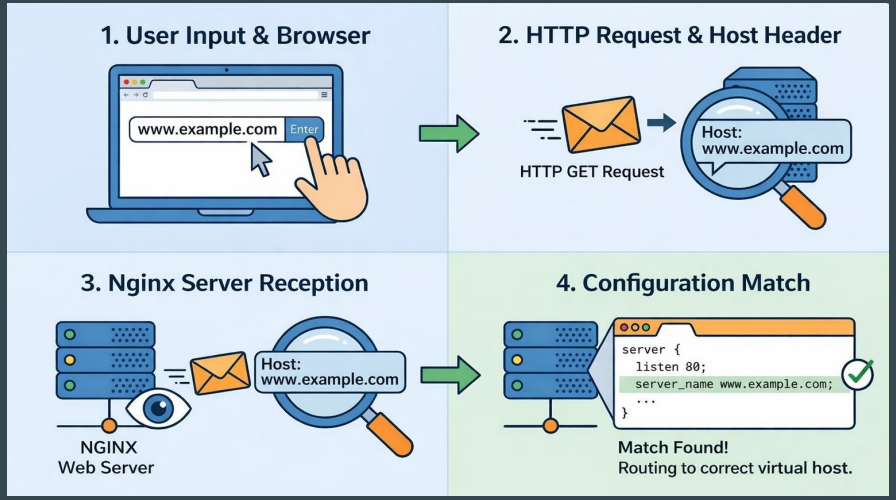
Internally, you have to configure multiple server blocks associated with each website domain.



How it Works (Internally)

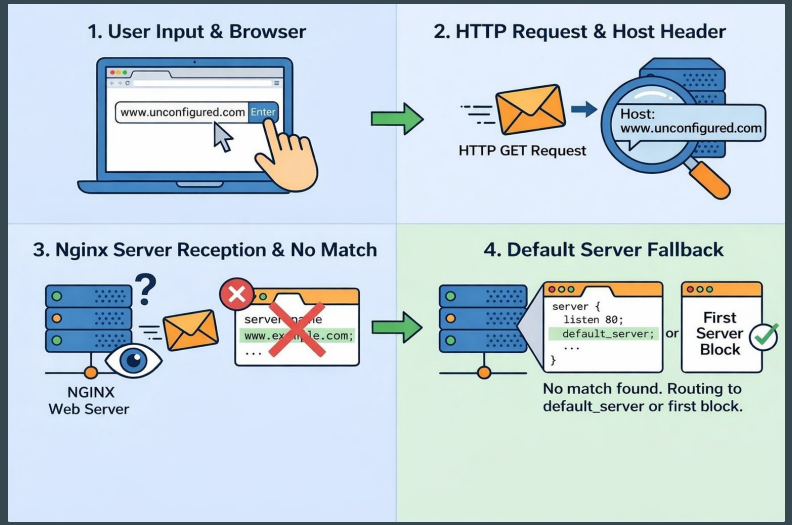
When user type a URL into your browser, the browser sends an HTTP request to the server. That request includes a Host Header.

Nginx looks at this header and matches it against the server_name defined in its configuration files.



Point to Note - The Default Server

If a request comes in for a domain that isn't configured, Nginx will send it to the first server block it finds or the one marked as default_server



nginx.conf

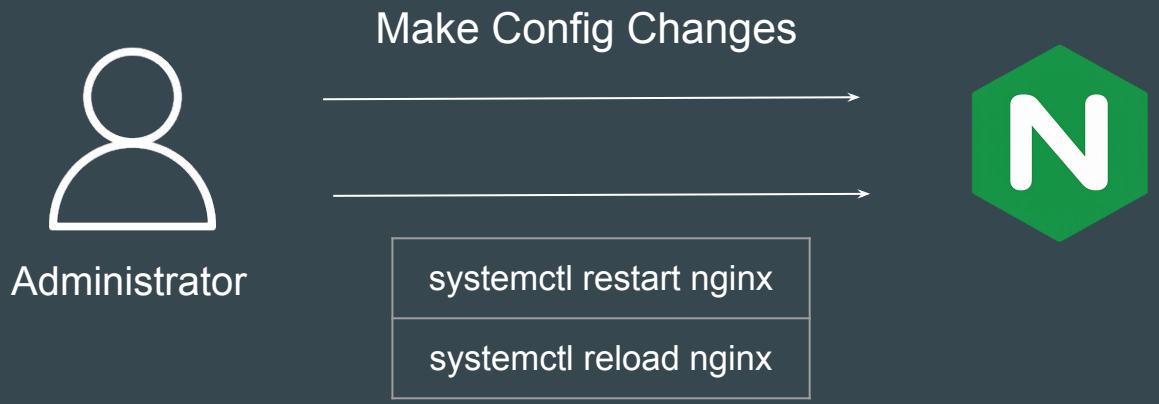
```
server {
    listen 80;
    server_name example.org www.example.org;
    ...
}
server {
    listen 80 default_server;
    server_name example.net www.example.net;
    ...
}
server {
    listen 80;
    server_name example.com www.example.com;
    ...
}
```

Nginx - Reload vs Restart

Setting the Base

The configuration changes made to Nginx are not applied automatically.

You will have perform either restart or reload operation.

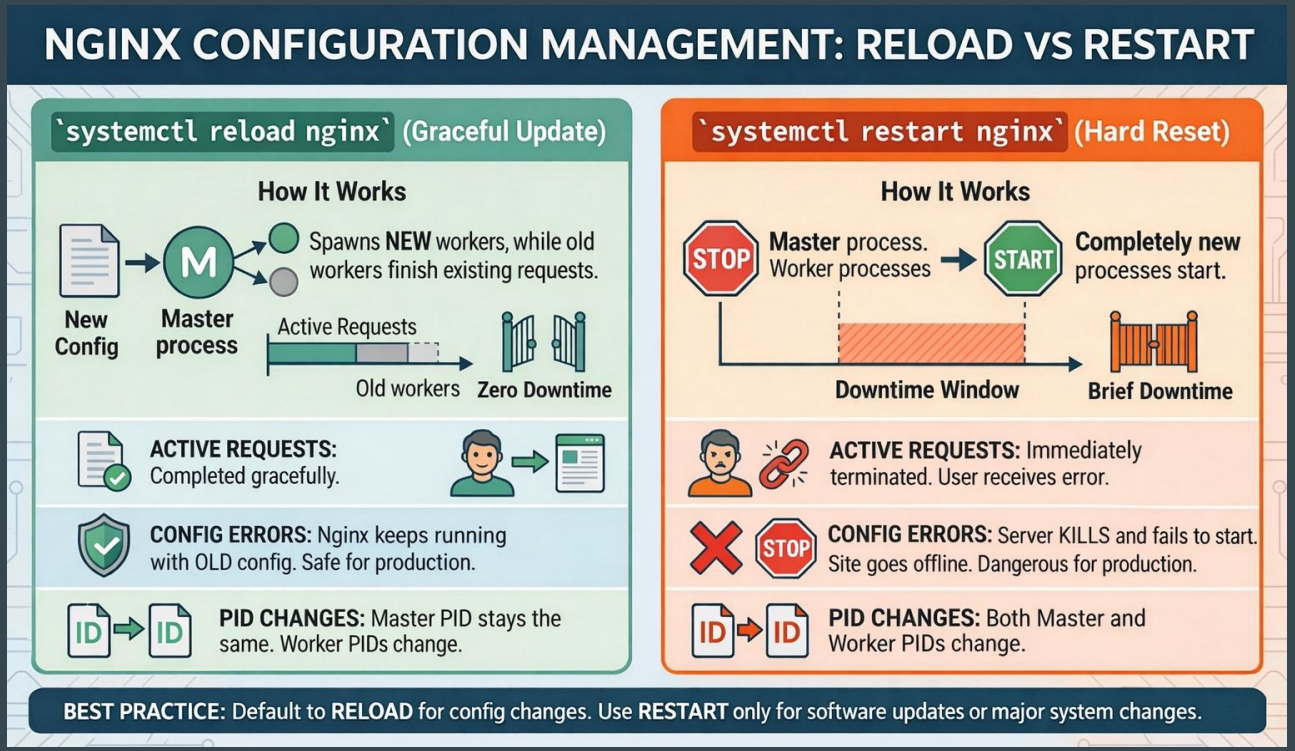


systemctl restart nginx	This is a "hard" reset. It kills the Nginx process entirely and then starts a brand-new one.
systemctl reload nginx	This is a "graceful" update. It tells Nginx to check the new configuration and apply it without stopping the server.

Comparison Table

Feature	systemctl restart nginx	systemctl reload nginx
Downtime	Yes (Brief)	No (Zero)
Active Requests	Terminated immediately	Completed gracefully
Config Errors	Server fails to start (stays down)	Server ignores new config (stays up with old config)
Process ID (PID)	Changes	Master PID stays the same

Reference Image for Revision



Points to Note

Reloading Nginx is the standard and safest method for applying most configuration updates, such as adding a new server block or changing proxy settings

Nginx Management - Systemd vs Native Signals

nginx -s signals

Once nginx is started, it can be controlled by invoking the executable with the **-s** parameter.

Use the following syntax: nginx -s signal

Signal	Description
stop	Fast Shutdown
quit	graceful shutdown
reopen	reloading the configuration file
reload	reopening the log files

root@application-server: ~

```
root@application-server:~# nginx -s reload
root@application-server:~#
```



Points to Note

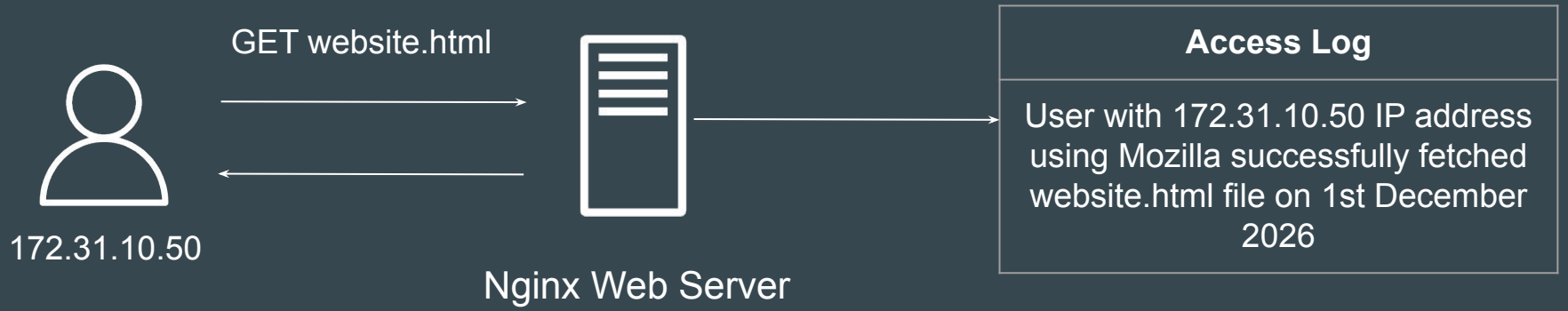
Use systemctl reload nginx for 99% of your tasks. It is safer, more "standard" for Linux administration, and integrates with the rest of your system's health monitoring.

Use nginx -s reload only if you are working in a environment where systemd isn't used (like some Docker containers) or if you are manually compiling/testing Nginx in a custom directory.

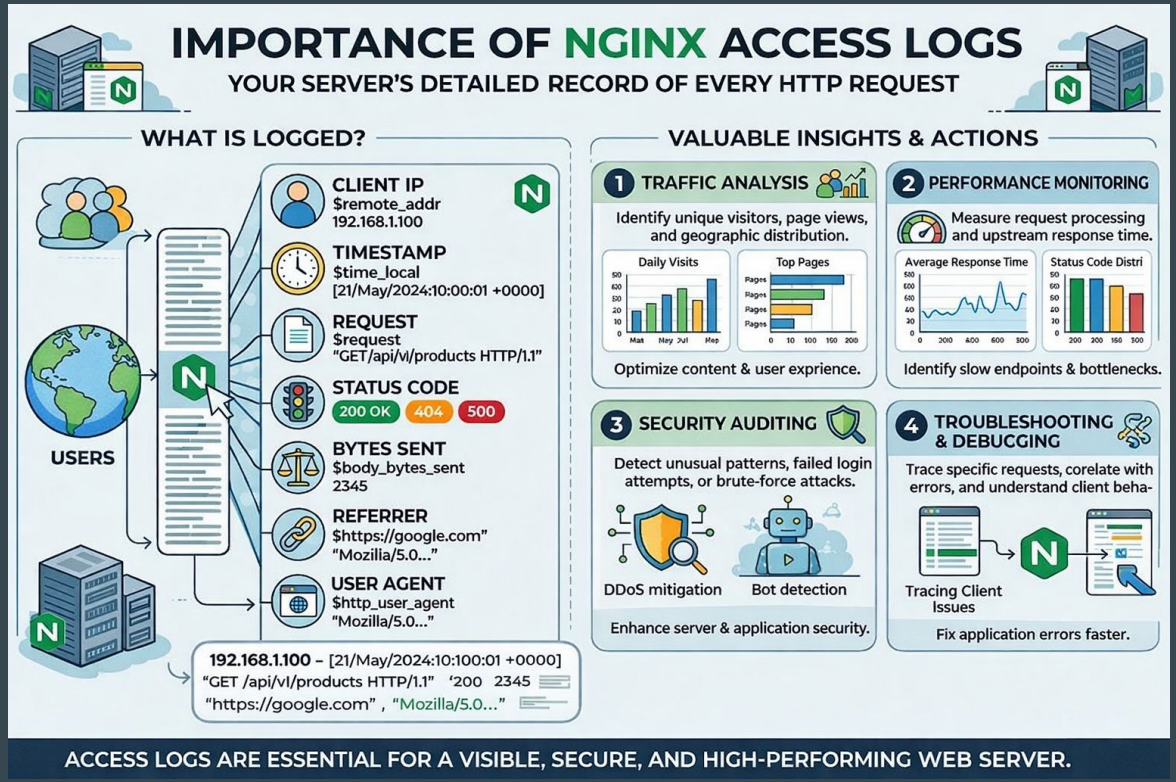
Access Logs

Access Logs

Nginx access logs provide a record of all requests processed by the server. This record captures metadata including the client's IP address, request method, URI, HTTP protocol version, status codes, etc.

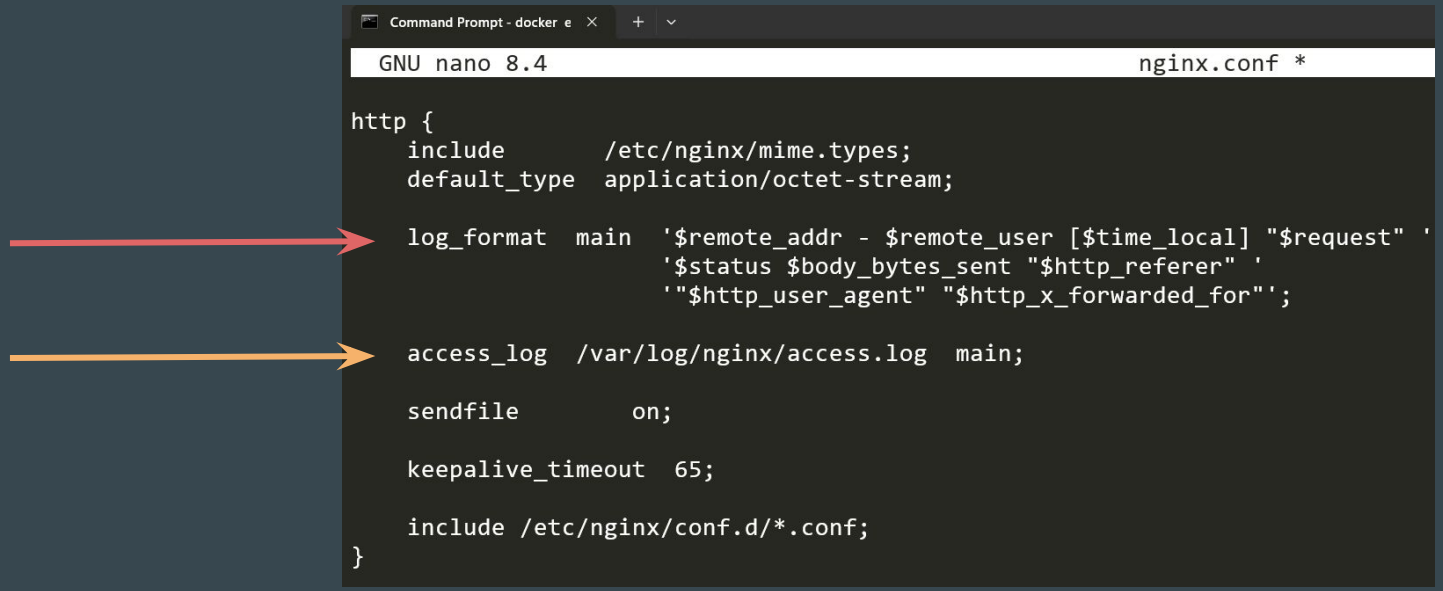


Reference Image for Revision



Important Directives

Component	Configuration
access_log	Defines the path where access log files are stored
log_format	Defines the format of the access log events.



Mapping

'$remote_addr - $remote_user [$time_local] "$request" ' '$status $body_bytes_sent"$http_referer" ' '"$http_user_agent" "$http_x_forwarded_for"';



Command Prompt - docker e

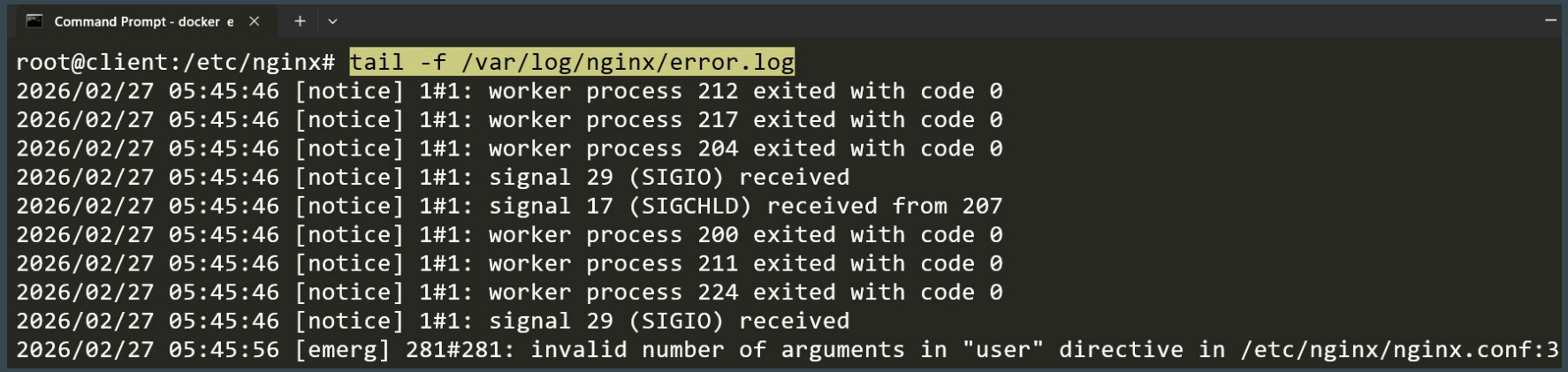
```
root@app-01:/var/log/nginx# cat access.log
172.17.0.2 - - [26/Feb/2026:18:07:13 +0000] "GET / HTTP/1.0" 200 28 "-" "curl/8.14.1" "-"
172.17.0.4 - - [26/Feb/2026:18:07:46 +0000] "GET / HTTP/1.1" 200 28 "-" "curl/8.14.1" "-"
172.17.0.2 - - [26/Feb/2026:18:08:12 +0000] "GET / HTTP/1.0" 200 28 "-" "curl/8.14.1" "-"
172.17.0.2 - - [26/Feb/2026:18:14:30 +0000] "GET / HTTP/1.0" 200 28 "-" "curl/8.14.1" "-"
172.17.0.4 - - [26/Feb/2026:18:15:21 +0000] "GET / HTTP/1.1" 200 28 "-" "curl/8.14.1" "-"
```

Variable	Log Value	Description
$remote_addr	172.17.0.2	The IP address of the client.
$remote_user	-	The username used for HTTP basic authentication (empty here).
[$time_local]	[26/Feb/2026:18:07:13 +0000]	The local time of the server when the request was processed.
"$request"	"GET / HTTP/1.0"	The full request line (Method, URI, and Protocol).
$status	200	The HTTP response status code (Success).
$body_bytes_sent	28	The number of bytes sent to the client, excluding headers.
"$http_referer"	"-"	The address of the page that linked to this URL (empty/none).
"$http_user_agent"	"curl/8.14.1"	The software/browser used by the client.
"$http_x_forwarded_for"	"-"	The original client IP if the request passed through a proxy.

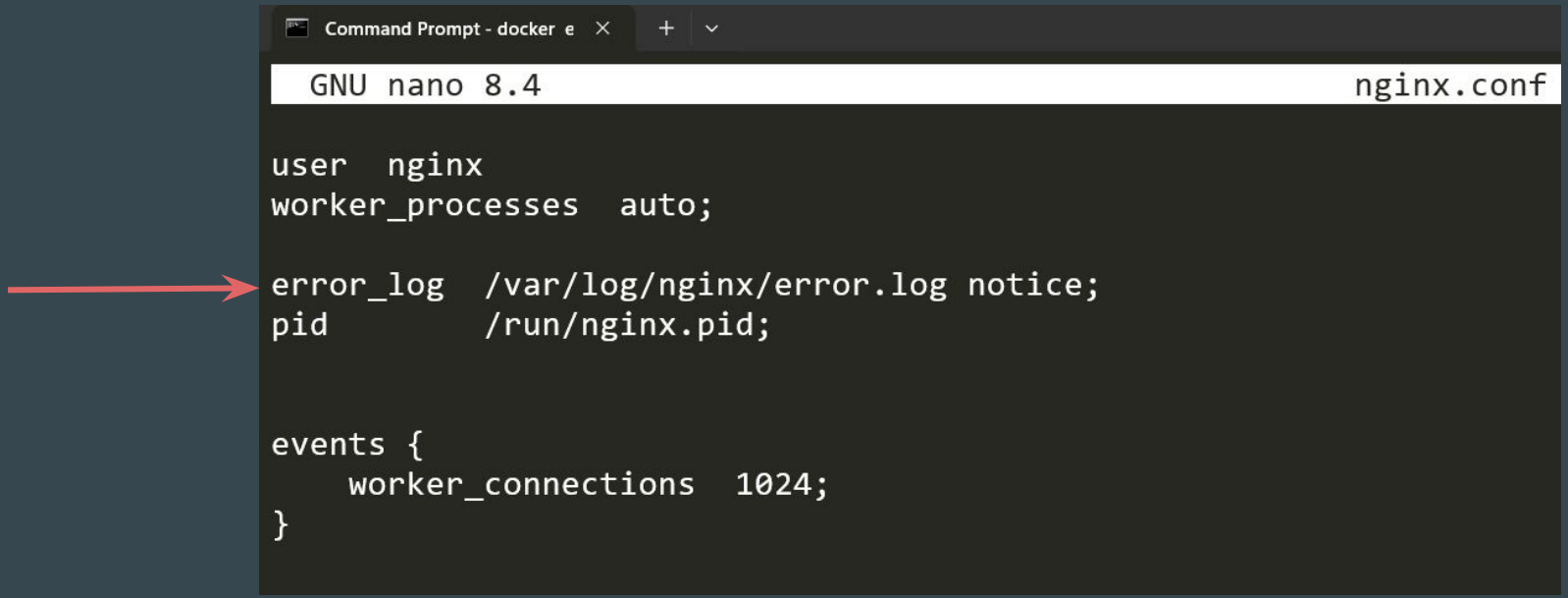
Error Logs

Setting the Base

In Nginx, Access Logs tell you who visited, Error Logs tell you why things are breaking.

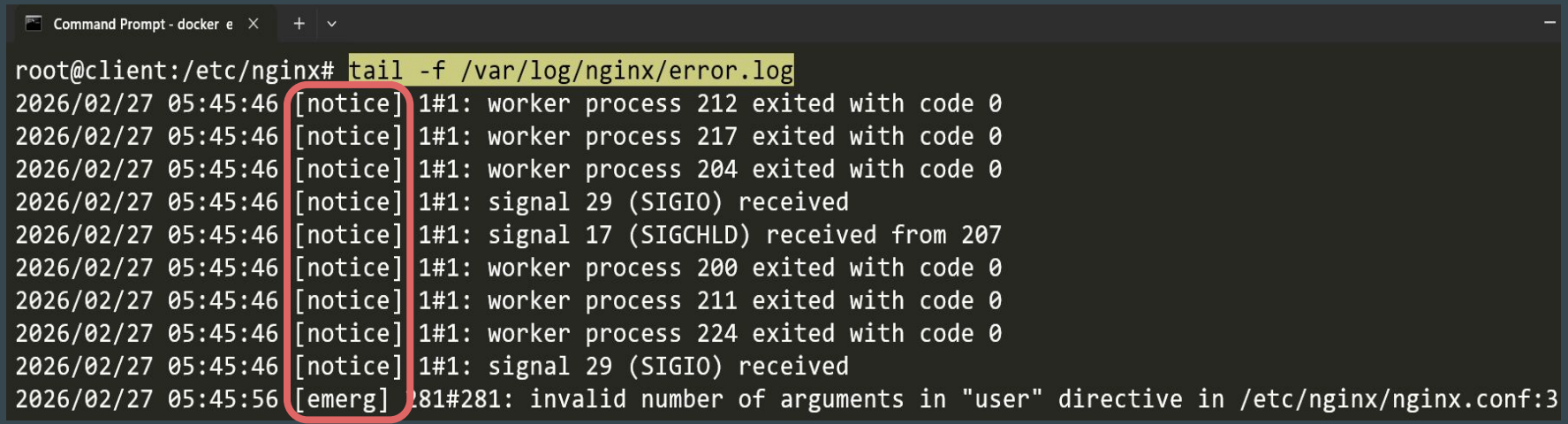


Error Log Directive



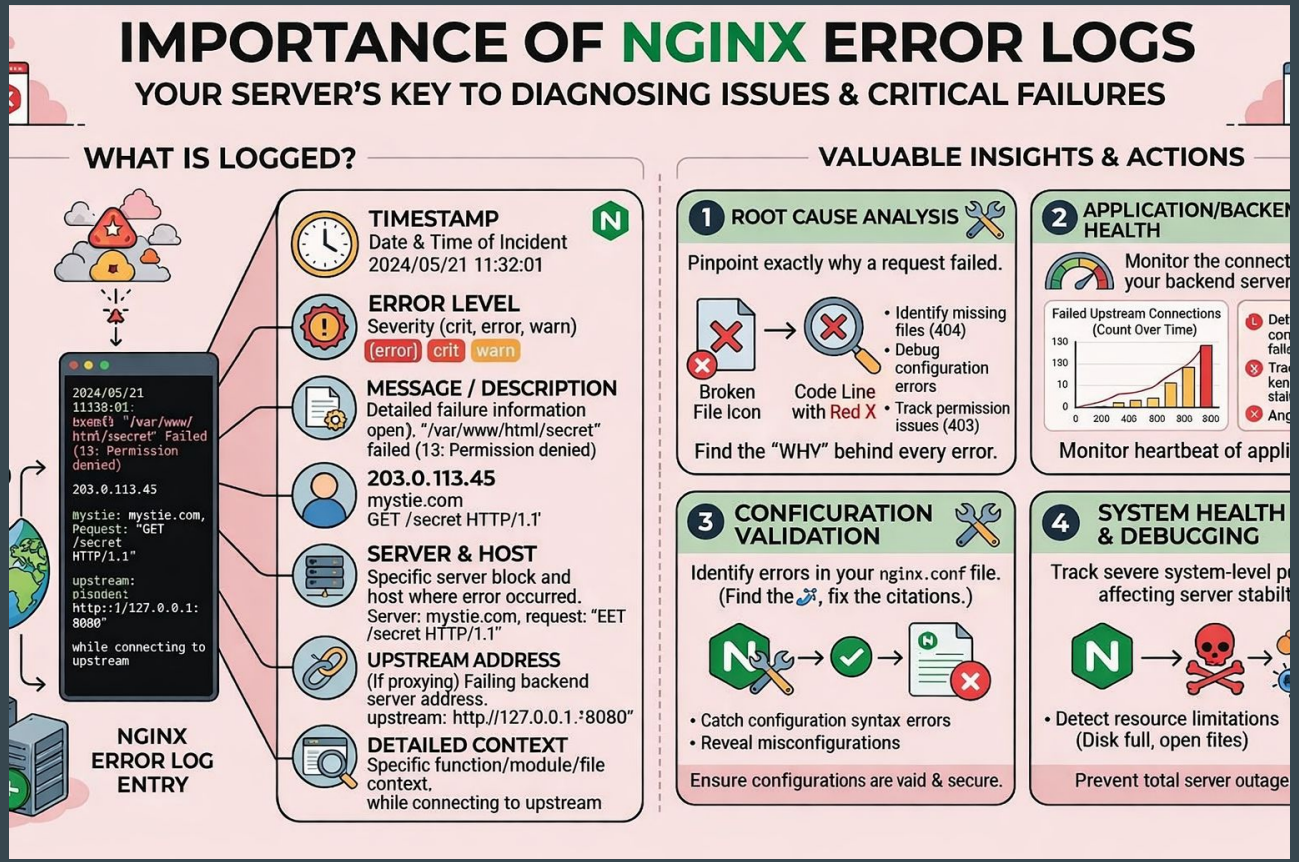
Log Levels

Level	Description
emerg	Emergency; system is unstable.
alert	Action must be taken immediately.
crit	Critical conditions.
error	Standard runtime errors (e.g., file not found).
warn	Warning messages.
notice	Normal but significant conditions


Command Prompt - docker e
root@client:/etc/nginx# tail -f /var/log/nginx/error.log
2026/02/27 05:45:46 [notice] 1#1: worker process 212 exited with code 0
2026/02/27 05:45:46 [notice] 1#1: worker process 217 exited with code 0
2026/02/27 05:45:46 [notice] 1#1: worker process 204 exited with code 0
2026/02/27 05:45:46 [notice] 1#1: signal 29 (SIGIO) received
2026/02/27 05:45:46 [notice] 1#1: signal 17 (SIGCHLD) received from 207
2026/02/27 05:45:46 [notice] 1#1: worker process 200 exited with code 0
2026/02/27 05:45:46 [notice] 1#1: worker process 211 exited with code 0
2026/02/27 05:45:46 [notice] 1#1: worker process 224 exited with code 0
2026/02/27 05:45:46 [notice] 1#1: signal 29 (SIGIO) received
2026/02/27 05:45:56 [emerg] 281#281: invalid number of arguments in "user" directive in /etc/nginx/nginx.conf:3

Log Levels

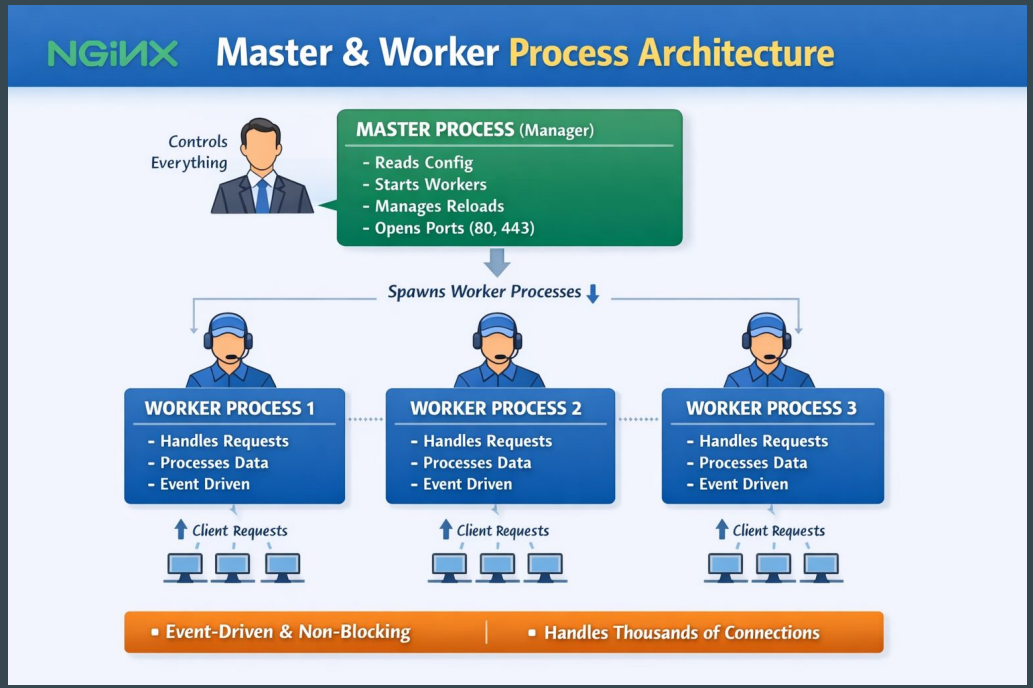
Reference Image for Revision



Master and Worker Processes

Setting the Base

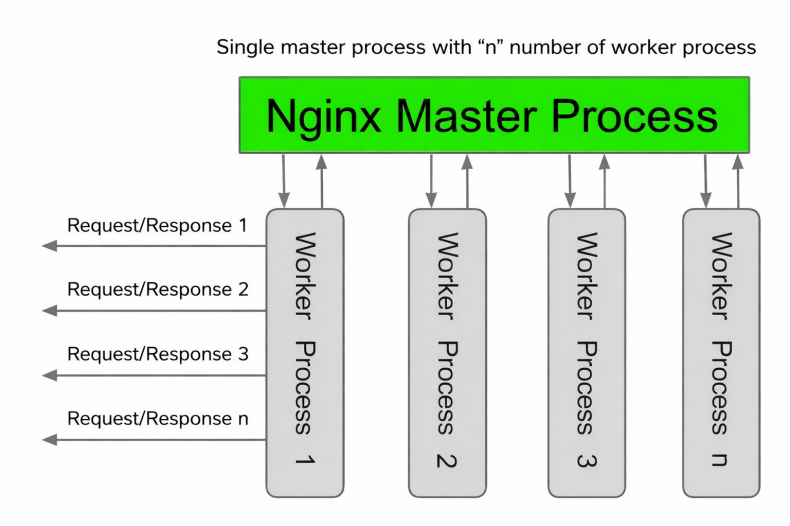
NGINX has one master process and one or more worker processes.



Understanding the Importance

The master process performs the privileged operations such as reading configuration and binding to ports, as well as maintain the worker processes.

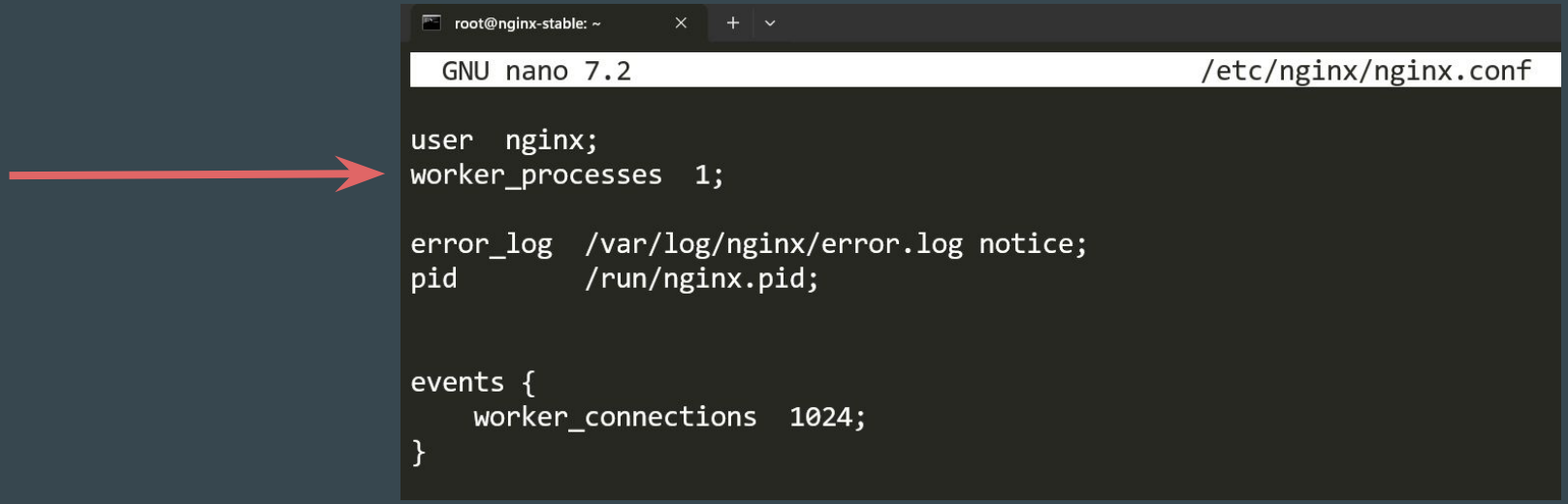
The worker processes do the actual processing of requests.



Recommended Configuration

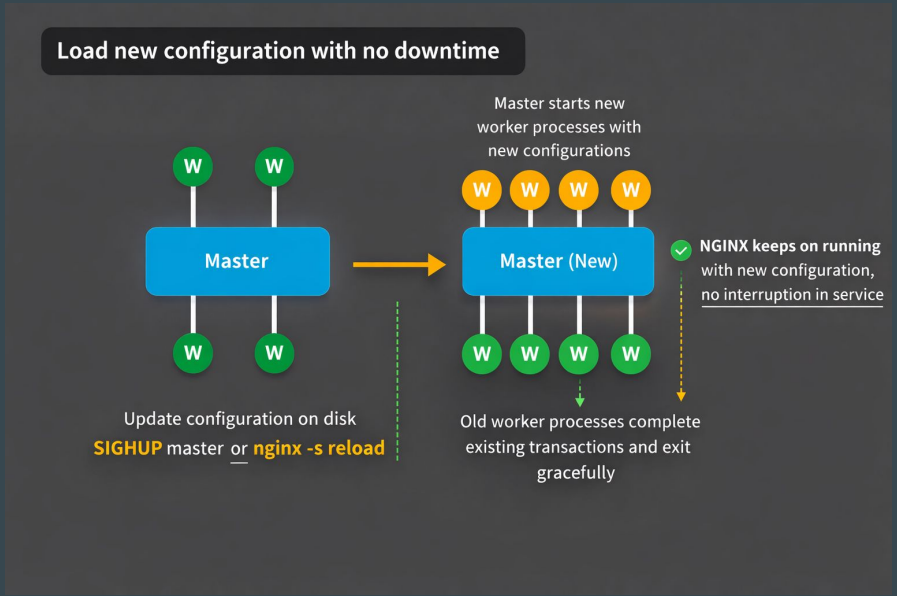
The NGINX configuration recommended in most cases – running one worker process per CPU core – makes the most efficient use of hardware resources.

You configure it by setting the auto parameter on the worker_processes directive



Updating Nginx Configuration

Updating Nginx configuration means running the nginx -s reload command, which checks the configuration on disk and sends the master process a SIGHUP signal.



Command Line Options

Let's CLI

Basics of CLI Options

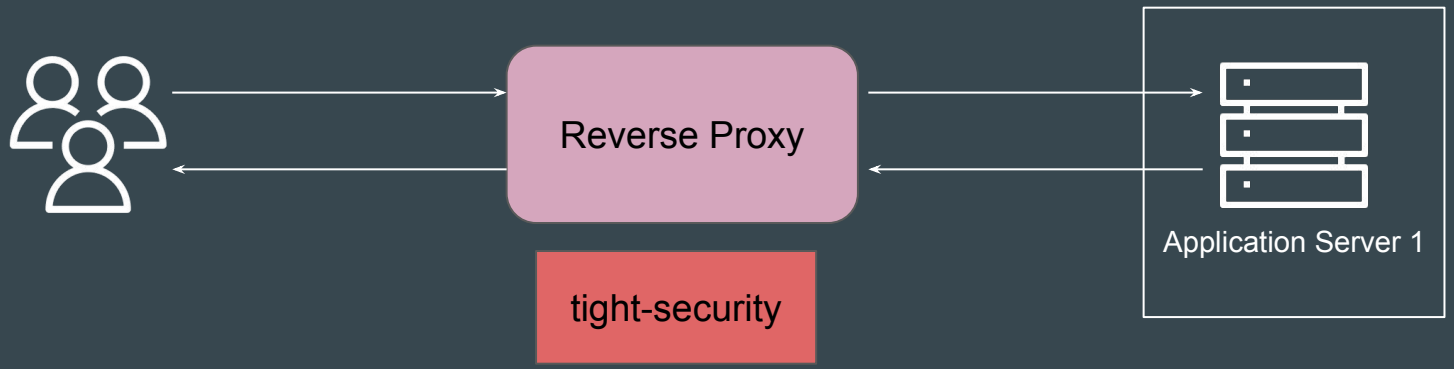
Nginx provides various command line options for performing specific operations.

Options	Description
-v	Print version.
-V	Print NGINX version, compiler version and configure parameters.
-t	Don’t run, just test the configuration file
-c	Specify which configuration file NGINX should use instead of the default.

Basics of Reverse Proxy

Basics of Reverse Proxy

A reverse proxy acts as a middle layer between the user (someone visiting a website) and the actual application server where the content lives.

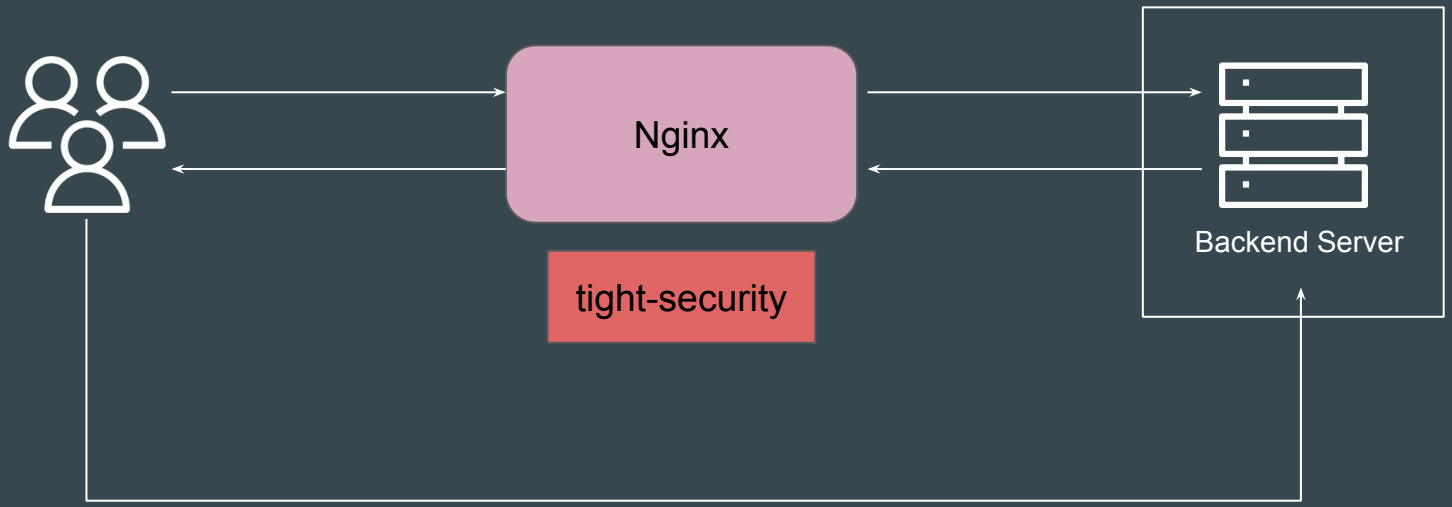


Benefits of Reverse Proxy

Benefit Category	Description
Security	Your backend servers stay in a private network, hidden from the public internet. It can protect against attacks.
Load Balancing	Reverse Proxy can distribute traffic across multiple servers so one doesn't get overwhelmed.
Caching	Commonly used data can be Cached at Reverse Proxy Level
SSL Termination	Reverse Proxy can handle the heavy lifting of HTTPS/Encryption, freeing up your app to just focus on logic.

Nginx As Reverse Proxy

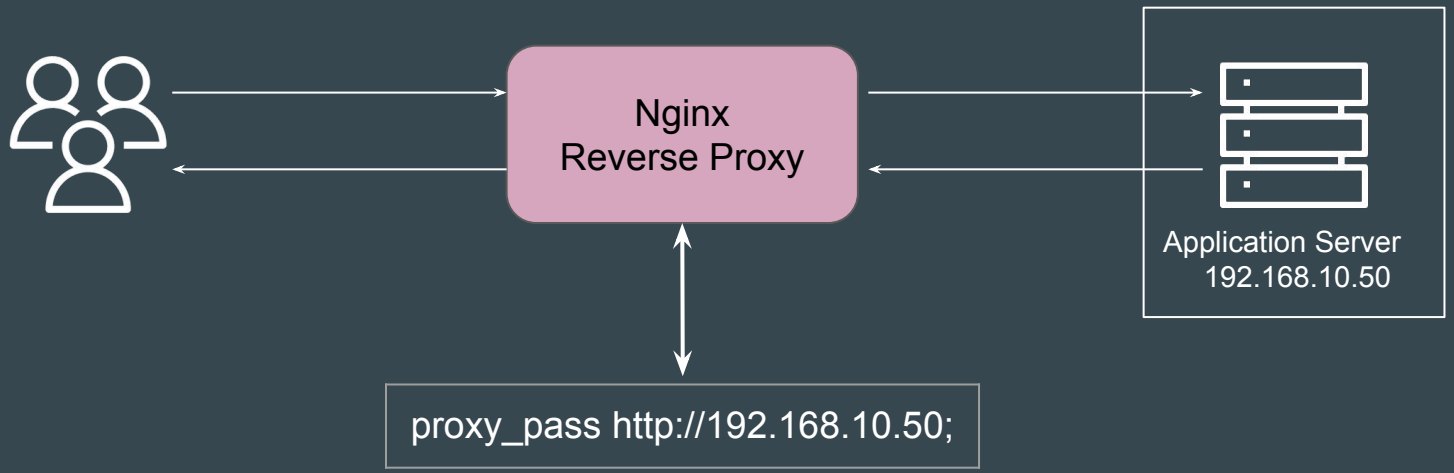
Nginx can act as a Reverse Proxy where it can receive all the request and forward it to the backend servers.



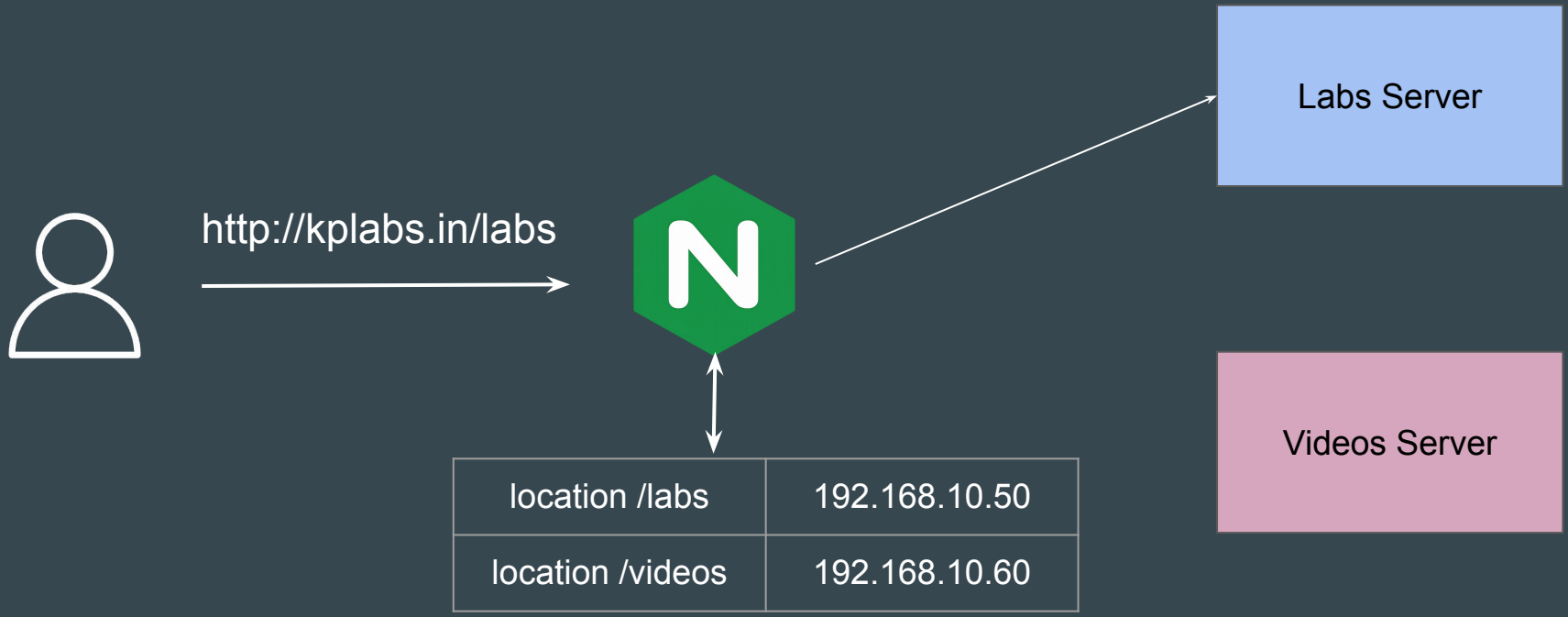
proxy_pass directive

Basics of proxy_pass

proxy_pass is an NGINX directive used within location blocks to forward incoming HTTP requests to a backend server



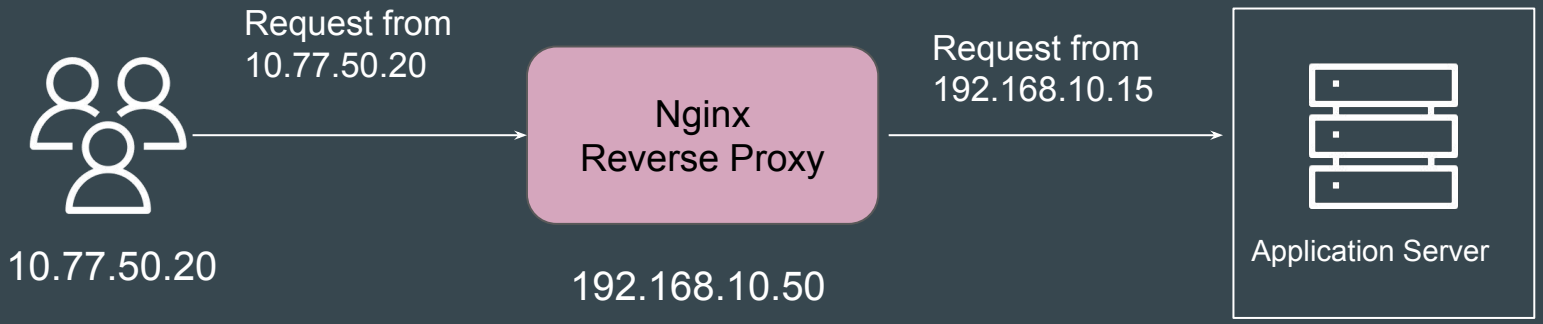
Examples of proxy_pass



X-Real-IP

Understanding the Challenge

When you place a proxy in front of your application server, the application server no longer sees the Client's IP. Instead, it sees the Proxy's IP for every single request.



Challenges

Challenge	Description
Geo-Location Issues	The backend application can’t serve localized content (like currency or language) because the server thinks everyone is located where the proxy is.
Inaccurate Analytics	Any analytic configured at the backend level will show that 100% of your traffic is coming from your environment (reverse proxy)

Introducing the Solution

The X-Real-IP header is a convention used to pass the original client's IP address through the proxy chain to the final destination.



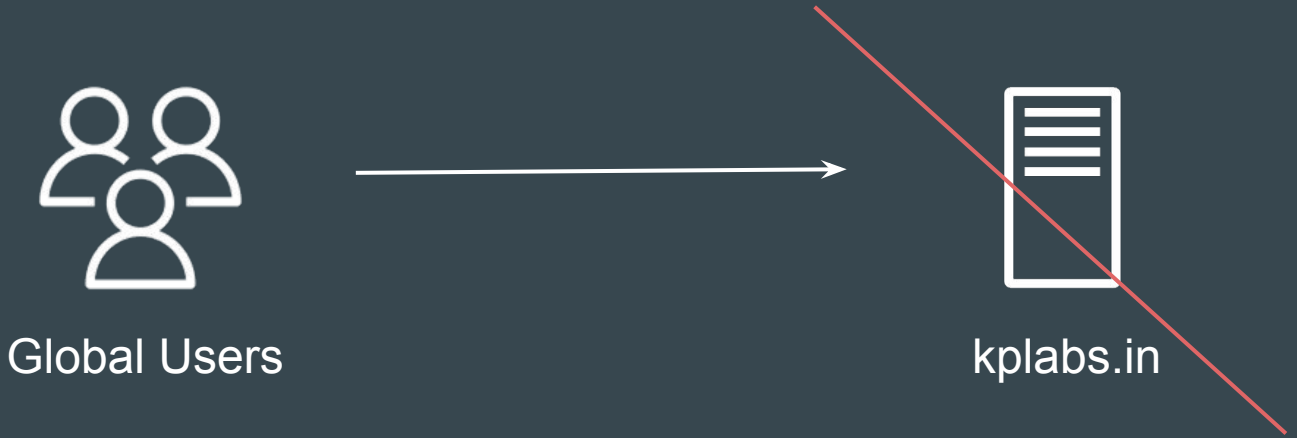
2 Important Points

Component	Configuration
Reverse Proxy	proxy_set_header X-Real-IP $remote_addr;
Backend	$http_x_real_ip;

Load Balancers

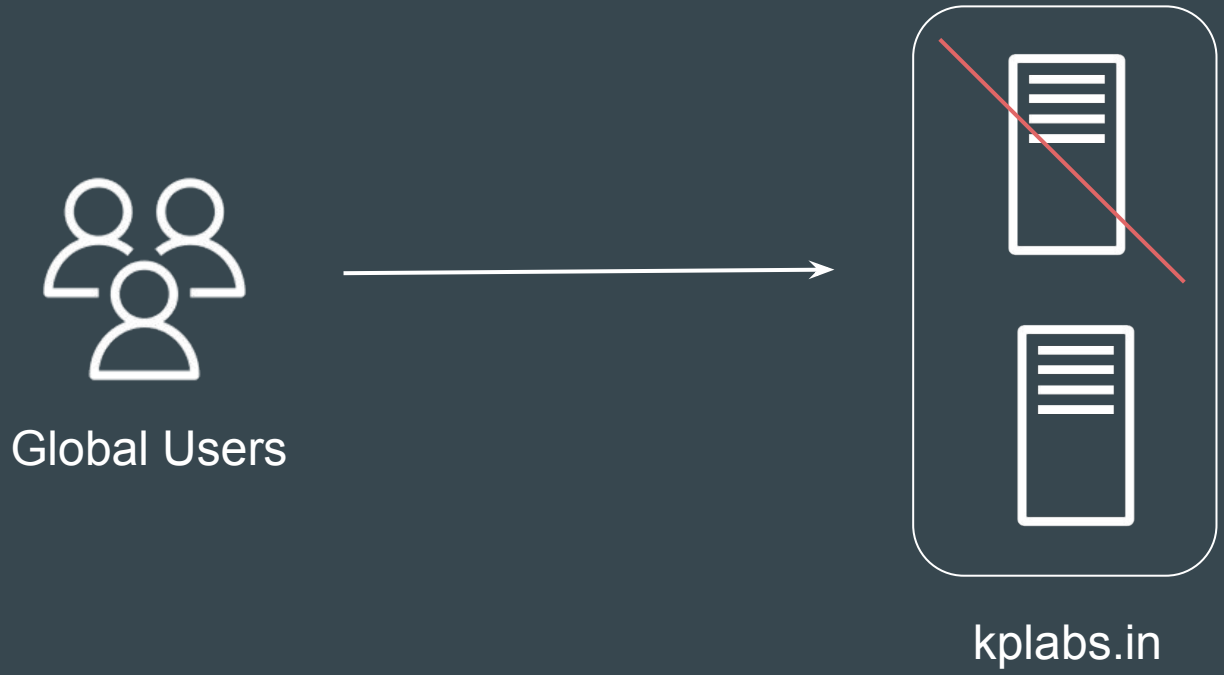
Understanding the Challenge

A single server creates a single point of failure, risking complete downtime if it crashes.



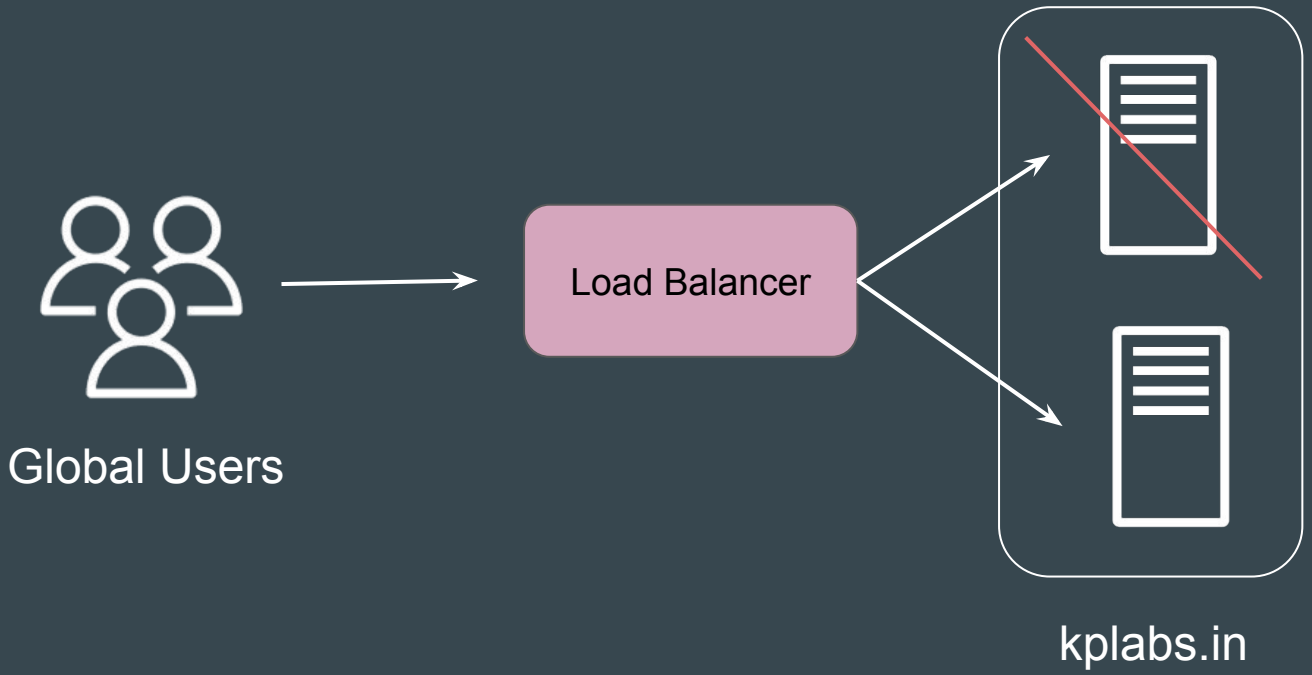
Multiple Server Architecture is Good

Organization uses multiple set of servers to prevent a single point of failure



Importance of Load Balancers

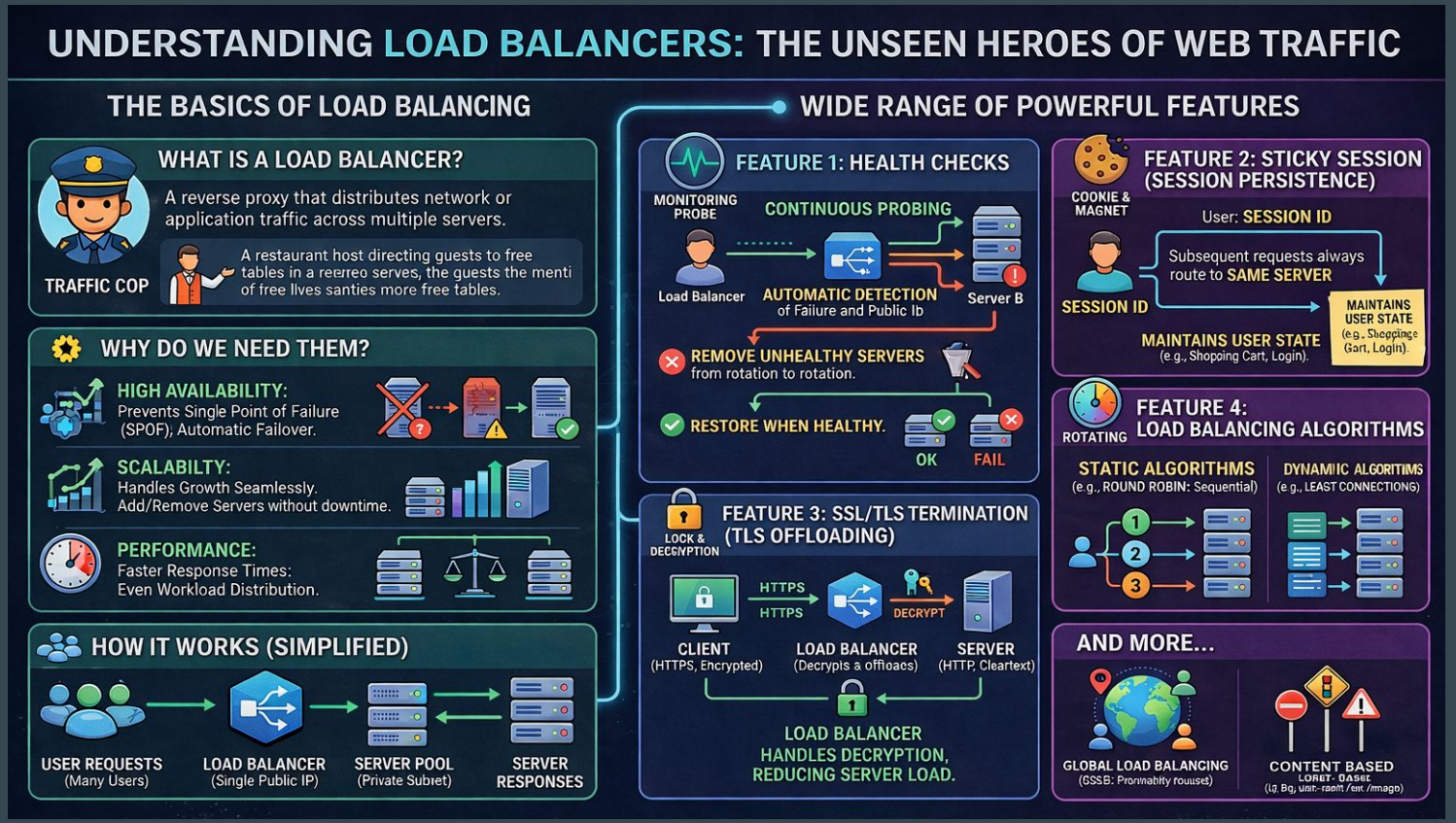
Load balancers distribute incoming traffic evenly across multiple servers, preventing overload. If one server goes down, the load balancer automatically redirects users to healthy servers.



Load Balancer Features

Features	Description
Health Checks	Load Balancer monitors the health status of backend server. If a backend server goes down, it stops sending request to it.
SSL Termination	Decrypting encrypted traffic (HTTPS) is CPU-intensive. A load balancer can handle the decryption process itself, passing the "plain text" request to the backend servers. This frees up the servers to focus on application logic rather than encryption
Sticky Sessions	The Load Balancer can use various methods to ensure users gets connected to the same backend for requirements related to session persistence.

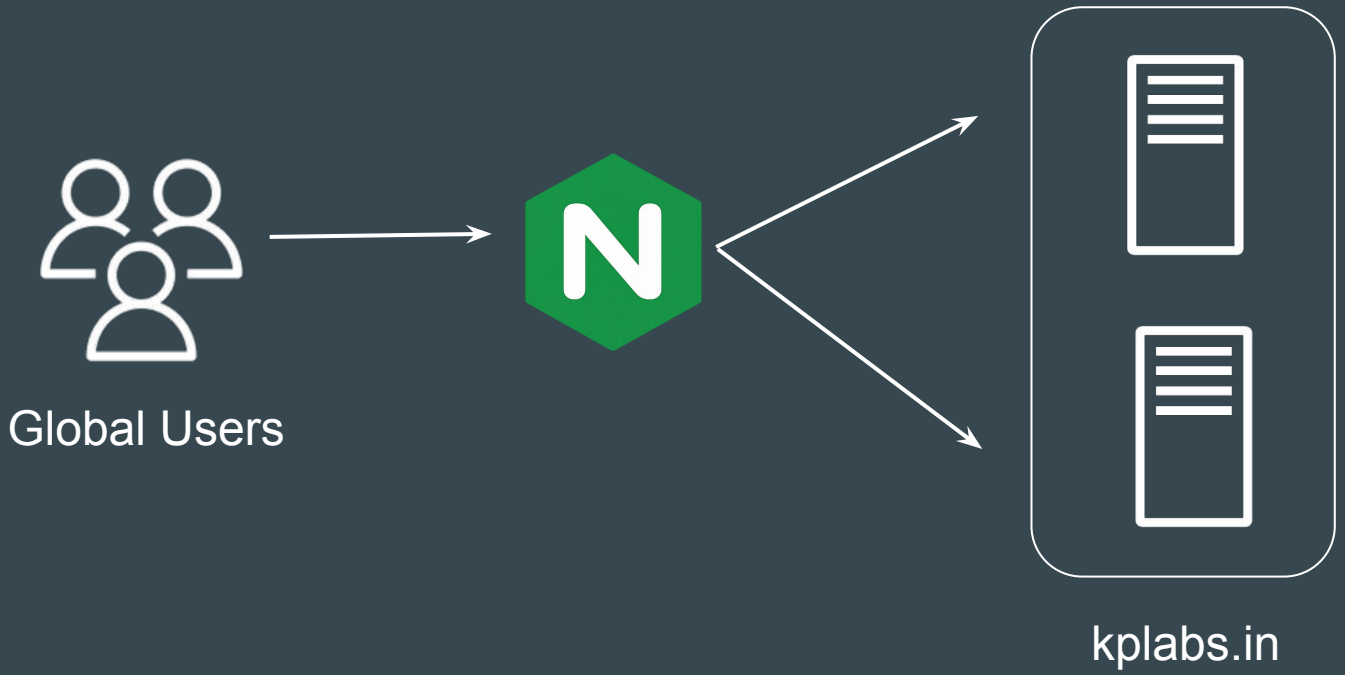
Reference Image for Revision



Nginx as Load Balancer

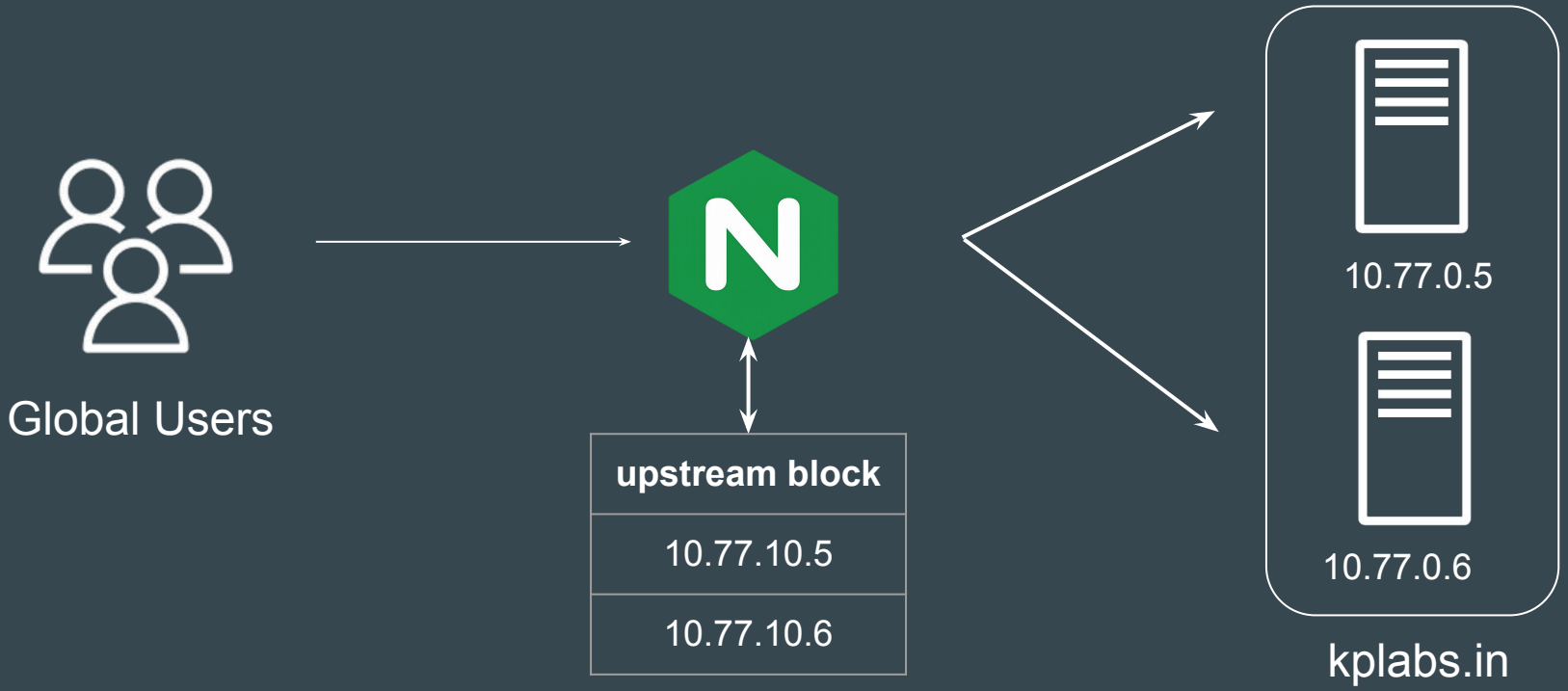
Setting the Base

NGINX can be used to load balance traffic across a group of servers.



Importance of upstream block

upstream block can be used to specify the group of servers for which you want the load balanced traffic to be distributed.



Upstream Configuration

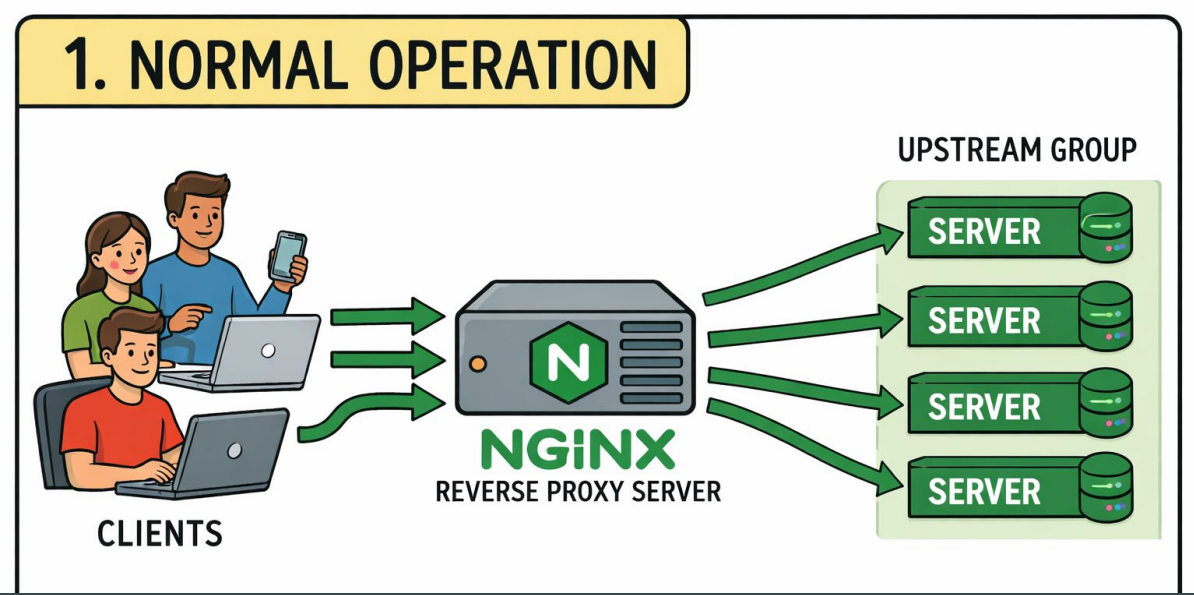
load-balancer.conf

```
upstream backend {
   server 10.139.0.3;
   server 10.139.0.4;
}

server {
    listen          80;
    server_name     localhost;

    location / {
       proxy_pass   http://backend;
    }
}
```

Reference Image



Practical Steps

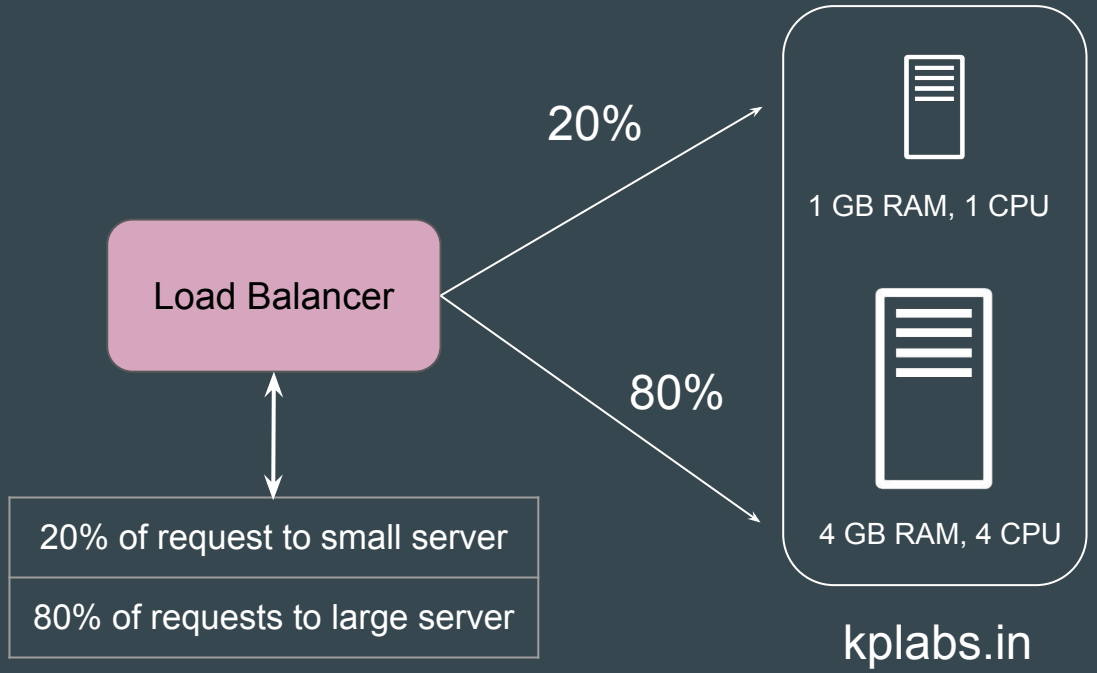
1. First, define the group of servers with the upstream directive. Place the directive in the http context.

2. To pass requests to a server group, the name of the group is specified in the proxy_pass directive with the server group

Server Weights

Unequal Distribution of Traffic

server weight allows us to customize the request flow between Nginx acting as a load balancer to the upstream backends.



Reference Configuration

load-balancer.conf — nano — 80×24

```
upstream backend {
    server 10.139.0.3;
    server 10.139.0.4 weight=2;
}

server {
    listen 80;
    server_name localhost;

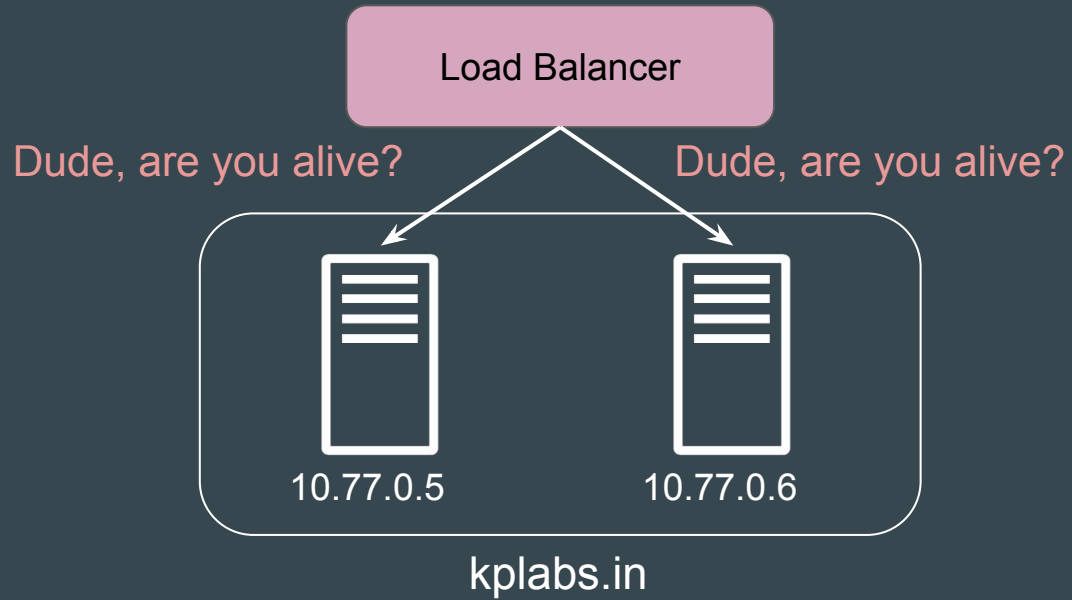
    location / {
        proxy_pass http://backend;
}
```

Basics of Health Checks

Setting the Base

Health checks are used to monitor the health of the servers in upstream group.

If a specific server is not responding, Nginx will stop sending requests to it.



Active vs Passive Health Checks - Nginx

Setting the Base

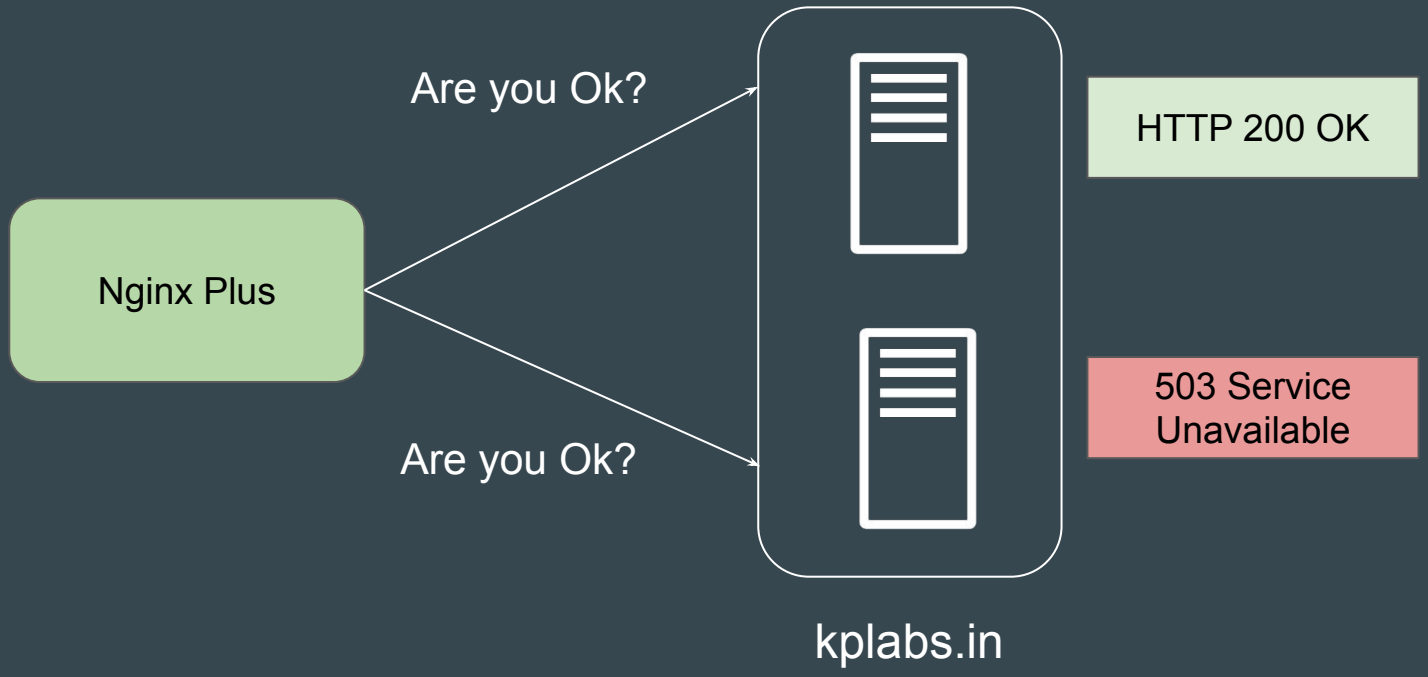
There are two type of Health Checks supported by Nginx:

1. Active Health Checks (Nginx Plus)

2. Passive Health Checks

Active Health Checks

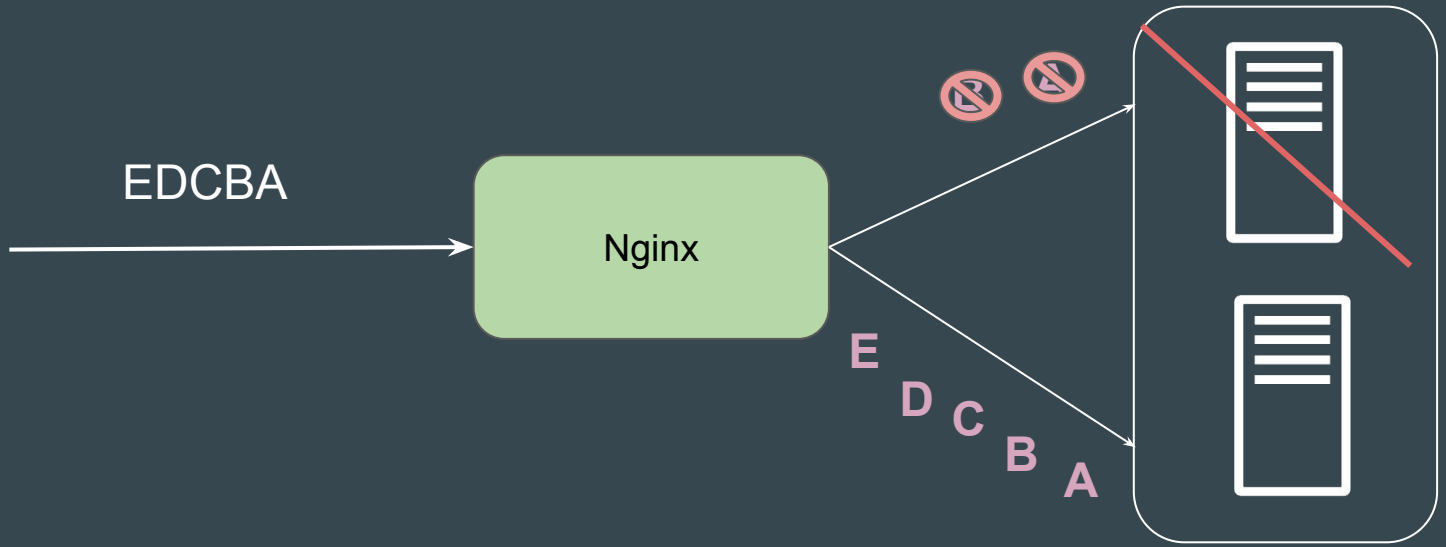
NGINX Plus can periodically check the health of upstream servers by sending special health-check requests to each server and verifying the correct response.



Passive Health Checks

In Passive Health Checks, the nginx monitors the communication between client and the upstream server.

If the upstream server is not responding or rejecting connections, the passive health check will consider the server to be unhealthy.

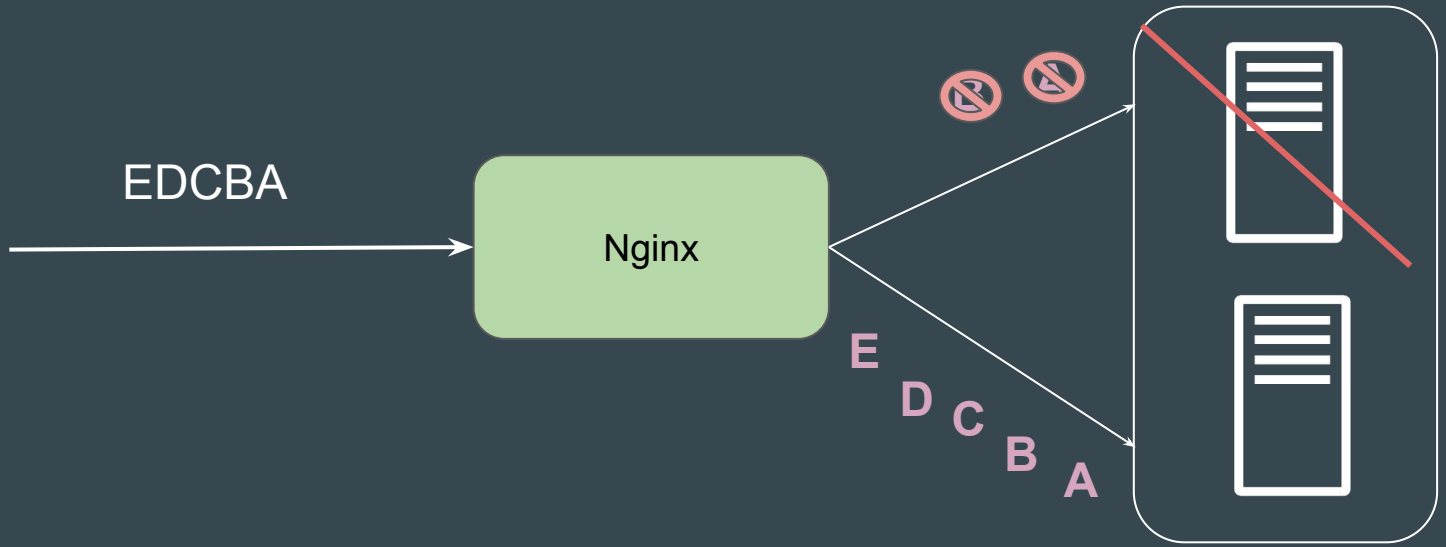


Parameters - Passive Health Check

Revising Passive Health Checks

In Passive Health Checks, the nginx monitors the communication between client and the upstream server.

If the upstream server is not responding or rejecting connections, the passive health check will consider the server to be unhealthy.



Important Parameters

The conditions under which an upstream server is marked unavailable are defined for each upstream server with parameters to the server directive in the upstream block.

Parameters	Description
fail_timeout	The duration of the time window used to count failures and the length of time the server stays "down." Once this limit is reached, Nginx stops sending traffic to that server temporarily.
max_fails	The number of failed attempts allowed within a specific time window before the server is marked "down."

Example Configuration

In the following example, if NGINX fails to send a request to a server or does not receive a response from it 3 times in 30 seconds, it marks the server as unavailable for 30 seconds.

load-balancer.conf — nano — 80×24

```
upstream backend {
    server backend1.kplabs.in max_fails=3 fail_timeout=30s;
    server backend2.kplabs.in max_fails=3 fail_timeout=30s;
}

server {
    listen 80;
    server_name  kplabs.in;

    location / {
         proxy_pass http://backend;
    }
}
```

Basics of Caching

Simple Analogy - Studying for Exams

The library (slow source) has every book, but it’s far away and takes 10 minutes to walk there.

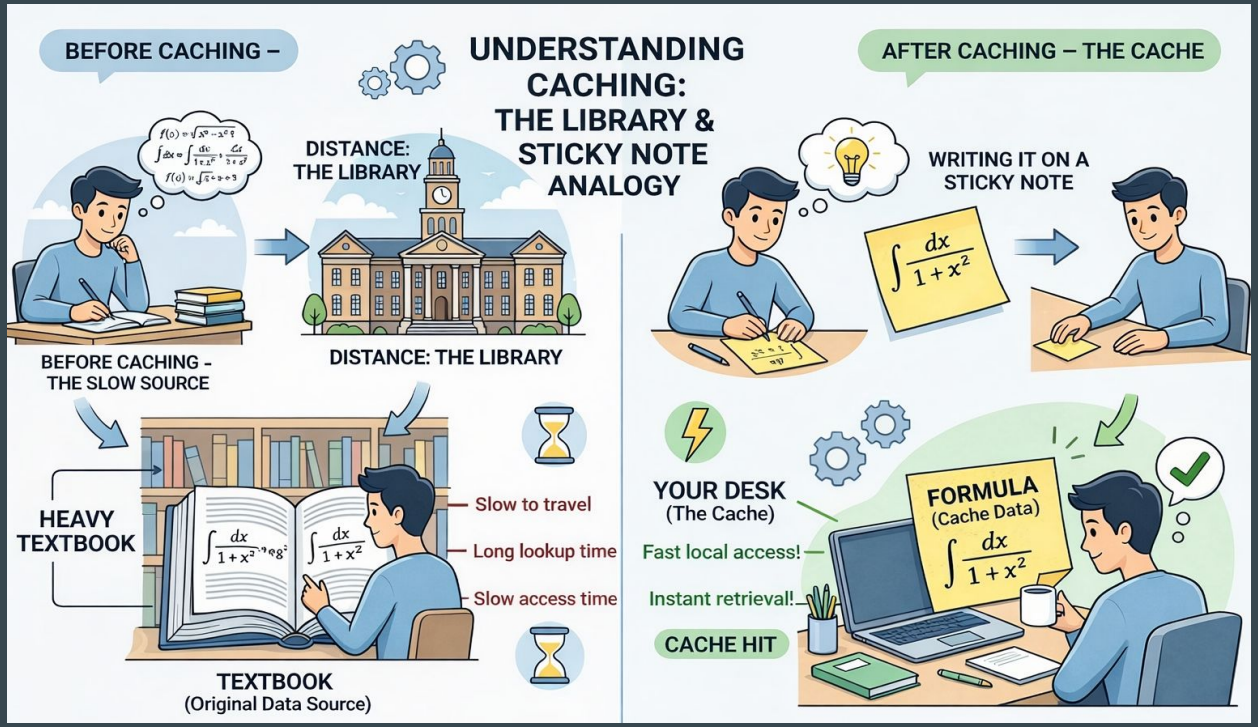


Simple Analogy - Start Caching

You keep the 5 books you use most on your study desk (the cache).



One more Analogy - Sticky Note



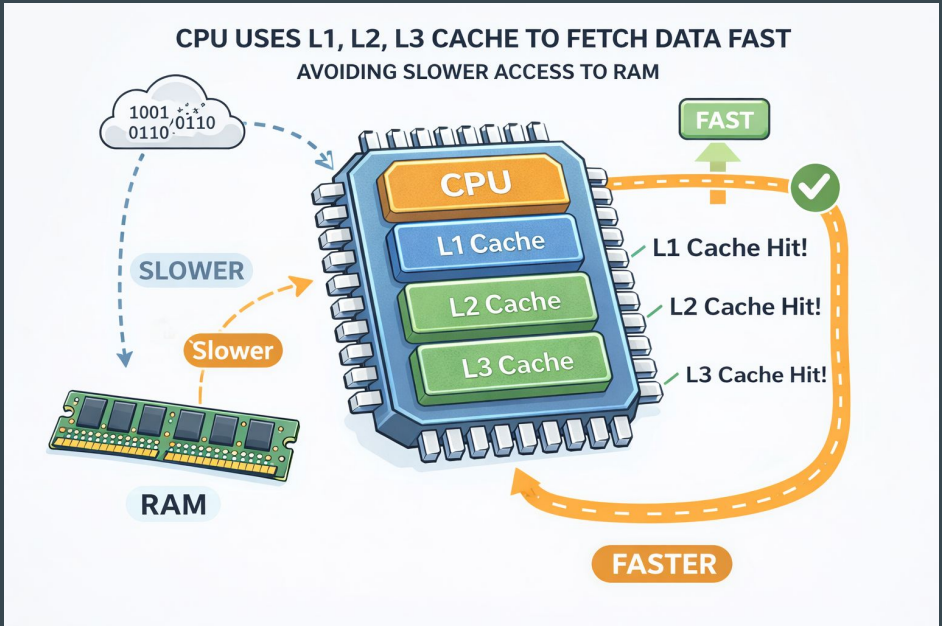
What is Caching?

A cache is a small, fast storage layer that keeps a copy of data you've recently used — so the next time you need it, you get it instantly instead of waiting for the slow original source.

Pointers	Description
Speed	Cache can be 10x–1000x faster than the original source (disk, network, database).
Limited Size	Cache is small and expensive. Only a fraction of data can be kept there.
Temporal Locality	Data accessed once is often accessed again soon. Caching uses this pattern.

Point to Note

Caching is used everywhere: browsers save web pages, databases store query results, DNS, Hardware like CPUs, etc.



Cache Hit and Cache Miss

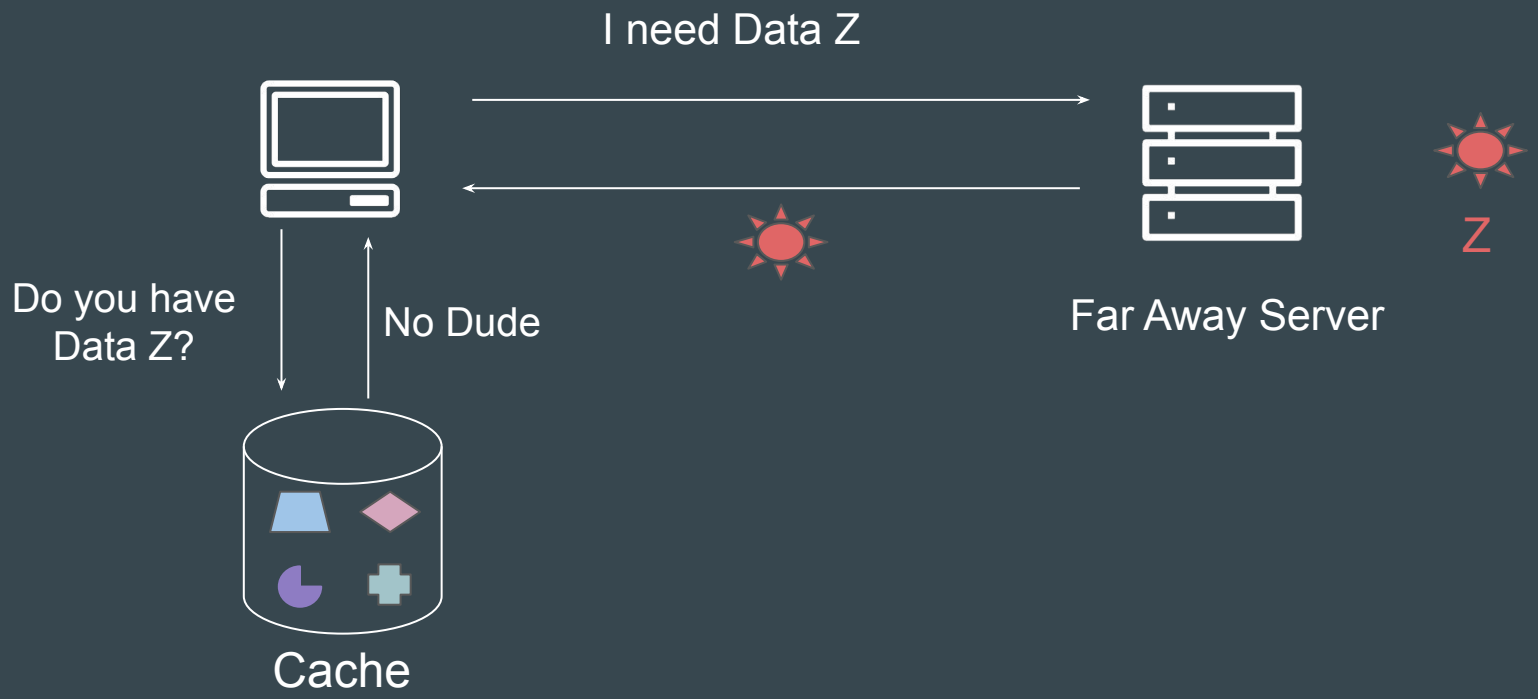
Every time your application needs data, it first checks the cache.

Cache Hit	When the data is in the cache — the data is returned immediately. No trip to the slow data source needed.
Cache Miss	When the data is not in cache — need to fetch it from the slow origin, then store it in cache for next time.

Reference Image - Cache Hit



Reference Image - Cache Miss

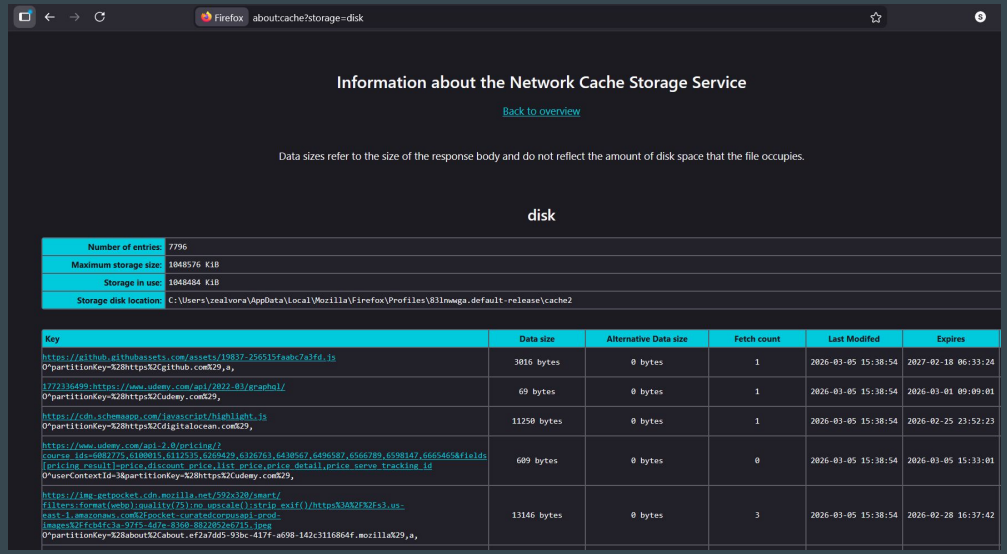


HTTP Caching

Setting the Base

All HTTP requests are first routed to the browser cache to check for a valid match.

If found, the response is served locally, eliminating network latency and data transfer costs.



Understanding the Challenge

Every time a browser, CDN, or proxy receives an HTTP response, it has to decide: should I store this and reuse it later, or fetch it fresh every time?

This is where various HTTP specific Cache Control headers proves to be useful.



Don’t Worry.

Let me help you decide

Cache-Control Headers

Controlling the Behaviour

The HTTP Cache's behavior is controlled by using appropriate HTTP headers.

```
< HTTP/1.1 200 OK
< Server: nginx/1.20.1
< Date: Thu, 05 Mar 2026 11:04:18 GMT
< Content-Type: image/jpeg
< Content-Length: 113445
< Last-Modified: Thu, 05 Mar 2026 10:58:46 GMT
< Connection: keep-alive
< ETag: "69a961e6-1bb25"
< Expires: Sun, 08 Mar 2026 11:04:18 GMT
< Cache-Control: max-age=259200
< Cache-Control: public
< Accept-Ranges: bytes
```

Sample Headers for Reference

Headers	Description
Expires	An absolute date/time after which the response is considered stale. (Deprecated by max-age).
ETag	A unique string (version ID) for a resource. If the content changes, the ETag changes.
If-Modified-Since	Sent by the browser; asks "Give me the file only if it has changed since this date."

Cache-Control Headers

Understanding the Challenge

Every time a browser, CDN, or proxy receives an HTTP response, it has to decide: should I store this and reuse it later, or fetch it fresh every time?

The Cache-Control header is your direct line of communication with that decision-making process.



Don’t Worry.

Let me help you decide

Cache-Control Headers

Setting the Base

Cache-Control headers tells browsers and intermediary caches (like CDNs) exactly how long they should hold onto a file before asking the server for a new one.

```
< HTTP/1.1 200 OK
< Server: nginx/1.20.1
< Date: Thu, 05 Mar 2026 11:04:18 GMT
< Content-Type: image/jpeg
< Content-Length: 113445
< Last-Modified: Thu, 05 Mar 2026 10:58:46 GMT
< Connection: keep-alive
< ETag: "69a961e6-1bb25"
< Expires: Sun, 08 Mar 2026 11:04:18 GMT
< Cache-Control: max-age=259200
< Cache-Control: public
< Accept-Ranges: bytes
```

Response Directives — Storability

Cache-Control: public	Response may be stored by any cache — browsers, CDNs, proxies.
Cache-Control: private	Only the end-user browser can cache this. Shared caches (CDNs, proxies) must not store it.
Cache-Control: no-store	Never store this response anywhere. Used for sensitive data (banking, health). No caching of any kind — strongest no-cache directive.
Cache-Control: no-cache	Store the response but revalidate with origin before serving it. Misleadingly named — it does cache, but always checks freshness first.

Scenario 1

You have an image (logo.jpg) that rarely changes.

You want the user's browser to cache it locally for a very long time so subsequent page loads are lightning-fast.

```
location /images/logo.jpg {
    add_header Cache-Control "public, max-age=31536000";
}
```

Understanding Scenario 1 Options

```
location /images/logo.jpg {
    add_header Cache-Control "public, max-age=31536000";
}
```

public	Response may be stored by any cache — browsers, CDNs, proxies.
max-age	Defines how long (in seconds) the file is considered "fresh."

Scenario 2

You update logo.jpg frequently for different holidays, but the URL stays the same.

You want the browser to cache the image to save bandwidth, but it must check with your server every single time to ensure it has the latest version.

```
location /images/logo.jpg {
    add_header Cache-Control "no-cache";
}
```

Scenario 3

Imagine your image is a temporary, highly sensitive QR code for banking authentication.

You absolutely do not want it stored on the user's hard drive or on any public Wi-Fi proxy.

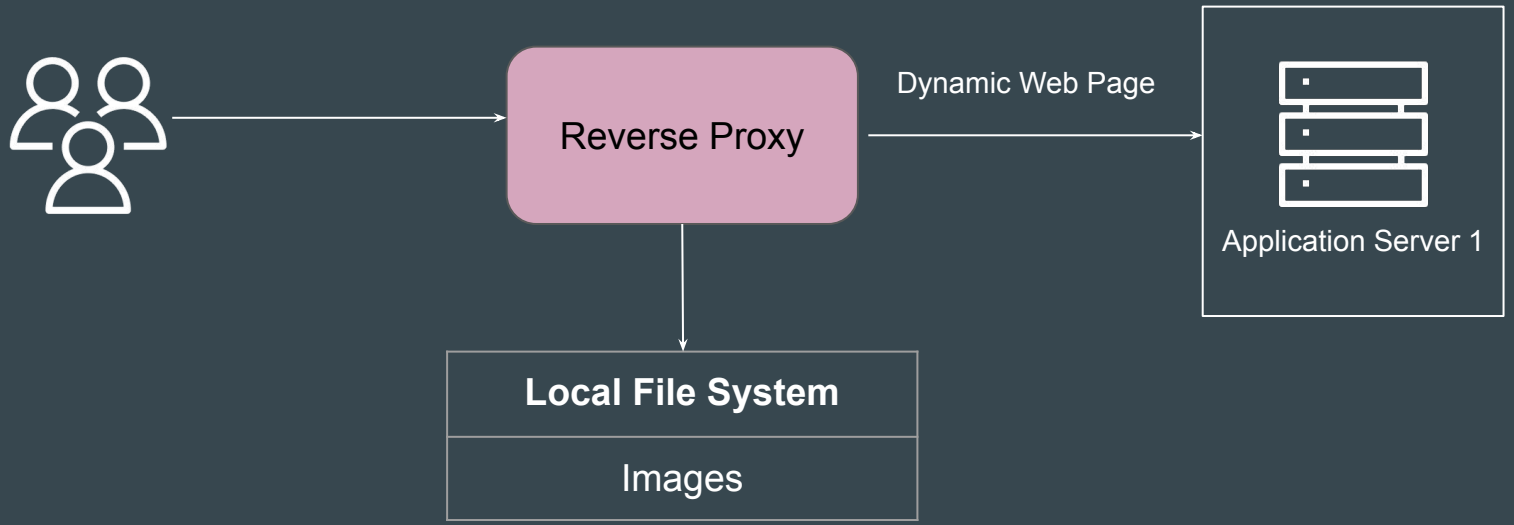
```
location /images/logo.jpg {
    add_header Cache-Control "no-store";
}
```

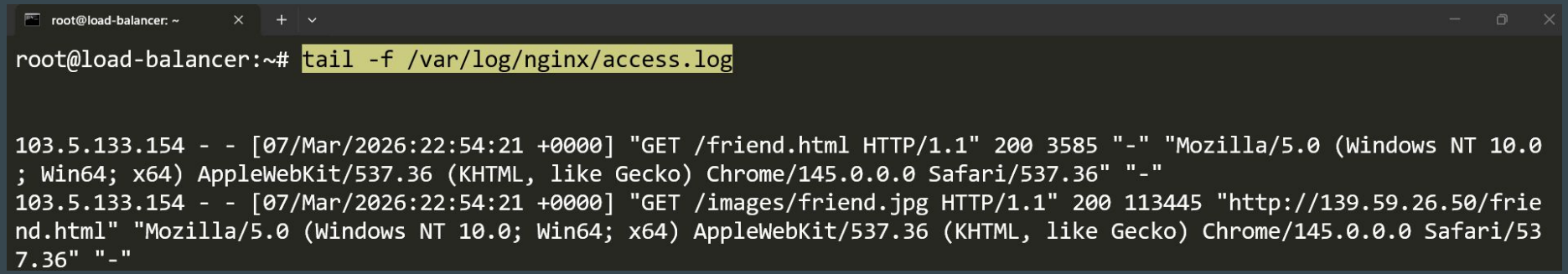
Serving Static Content via Nginx

Setting the Base

Your backend code can be heavy. If it had to read static files like images, videos from a disk and stream it, it would use up a lot of memory and CPU.

These static files can be stored at Nginx level to save load and bandwidth.



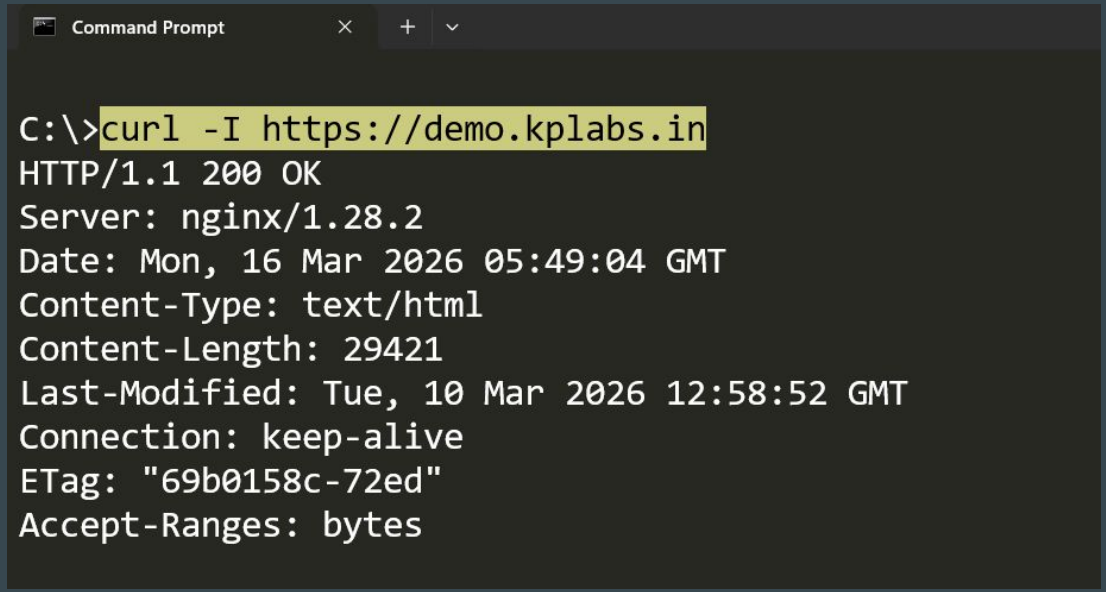

root@load-balancer: ~
root@load-balancer:~# tail -f /var/log/nginx/access.log
103.5.133.154 - - [07/Mar/2026:22:54:21 +0000] "GET /friend.html HTTP/1.1" 200 3585 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" "-"
103.5.133.154 - - [07/Mar/2026:22:54:21 +0000] "GET /images/friend.jpg HTTP/1.1" 200 113445 "http://139.59.26.50/friend.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" "-"

HTTP Response Status Codes

Setting the Base

When a client types a URL into a browser, they are sending a HTTP Request.

The server doesn't just send back a webpage; it also sends a three-digit numeric code that tells the browser how the request was handled.



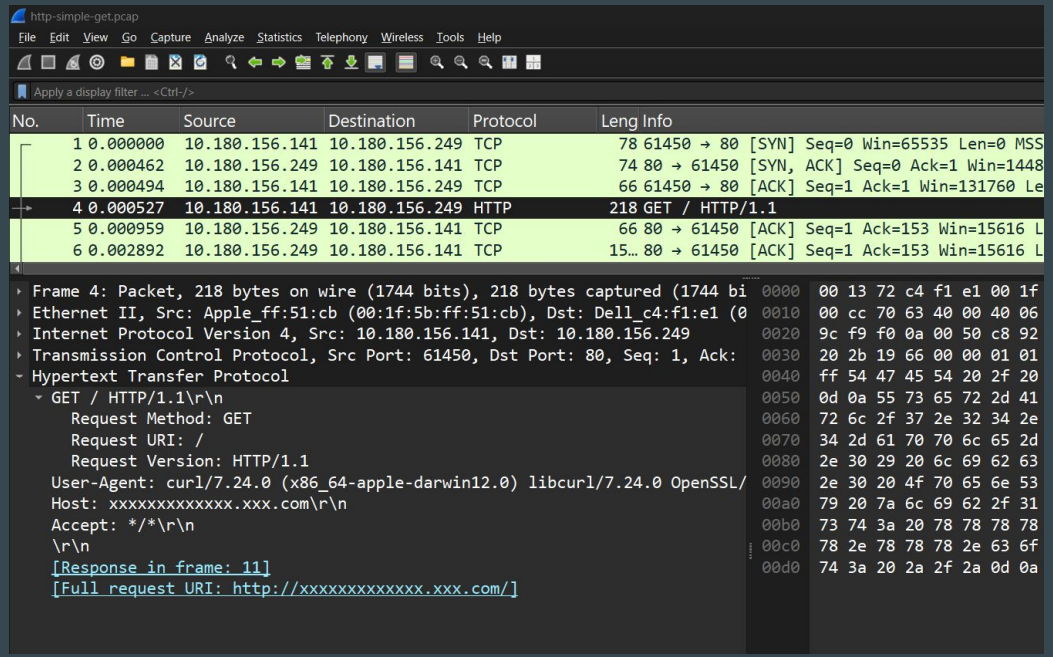
Anatomy of Response Code

Response Code	Primary Category	Description
1XX	Informational	"Hold on, I'm still processing." (Rarely seen by users).
2XX	Success	"Got it! Here is what you asked for."
3XX	Redirection	I know what you want, but it’s over there now.
4XX	Client Error	You made a mistake (bad URL, no password)
5XX	Server Error	I (the server) messed up or I'm broken

Built-in Nginx variables

Setting the Base

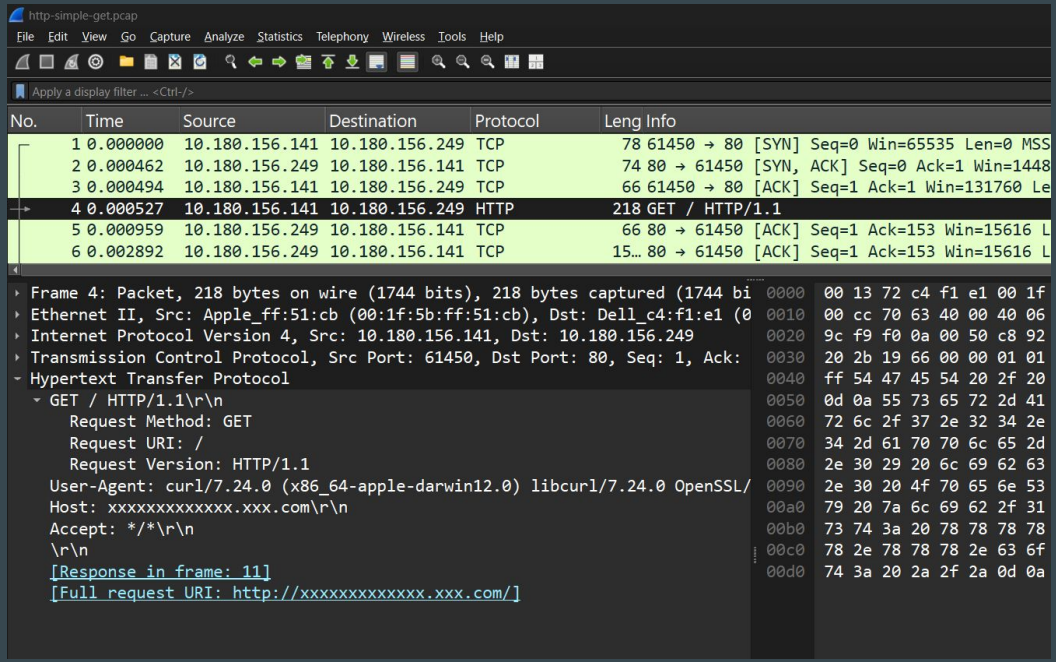
Every HTTP request carries lots of metadata — method, host, IP, headers, URI — all sent by the client automatically



Importance of Variables

Nginx exposes this metadata values as built-in variables so you can use them directly inside your config — no code required

Variables	Values
$remote_addr	10.180.156.141
$server_addr	10.180.156.249
$request_method	GET
$request_uri	/
$http_user_agent	curl



If Directive in Nginx

Setting the Base

The if directive evaluates a condition. If the condition is true, the block executes.



Simple Example

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    if ($remote_addr = "192.168.1.100") {
        return 403;
    }
}
```

Custom Variables - The map Directive

Setting the Base

The map directive in NGINX is used to create a new variable whose value depends on another variable.

In simple terms, map works like a conditional lookup table.

```
map $http_user_agent $index_file {
    default           "desktop.html";
    ~*iPhone          "mobile.html";
    ~*Android         "mobile.html";
}
```

Complete Configuration Example

```
map $http_user_agent $index_file {
    default           "desktop.html";
    ~*iPhone          "mobile.html";
    ~*Android         "mobile.html";
}

server {
    listen       80;
    server_name  localhost;
    root   /usr/share/nginx/html;

    location / {
        index $index_file;
    }
}
```

Regular Expressions

Let's Regex

Basics of Regular Expressions

Regular Expressions (regex) is a sequence of characters that defines a search pattern.

"There is a Rainbow which arises on the south shore of Mumbai"

Rainbow - Literal Character

Meta Character is a character or sequence of character that has special meaning that provides information about the other characters

Meta Characters

\d - Any digit from 0-9
\w - Any word (A-Z, a-z, 0-9]
\s - whitespace

. - Any character.
[] - Matches characters in brackets.
[^] - Matches characters not in brackets.

Example 1: Matching Phone Numbers

Sample Data:

125-450-955
550.220.900

Example 2: Matching List of Names

Sample Regular Expression:

Mr. Zeal
Mr Harsh
Mrs Surekha
Mr L
Ms Alice

The rewrite Directive

Setting the Base

A rewrite changes the URI of an incoming request to a different URI internally before Nginx looks for the file on the disk.

User URL	Internal URI Rewrite.
https://example.com/products/shoes	https://example.com/product.php?id=10

Basic Syntax

rewrite	regex	replacement	[flag]



Examples	Goals
rewrite ^/old-page$ /new-page;	Tells Nginx to map any request coming in for /old-page to /new-page
rewrite ^/user/([a-z]+)$ /profile.php?user=$1;	You want users to type example.com/user/john but have Nginx actually serve example.com/profile.php?user=john.
rewrite ^/(.*)\.html$ /$1;	You've decided to drop .html from your URLs for a sleeker look.

Sample Configuration Example

```
server {
    listen       80;
    server_name  localhost;
    root   /usr/share/nginx/html;

    location /myimages {
        rewrite ^/myimages(.*)$ /images$1;
    }
}
```

```
server {
    listen       80;
    server_name  localhost;
    root   /usr/share/nginx/html;

    location /myimages {
        rewrite ^/myimages(.*)$ /images$1;
    }
}
```

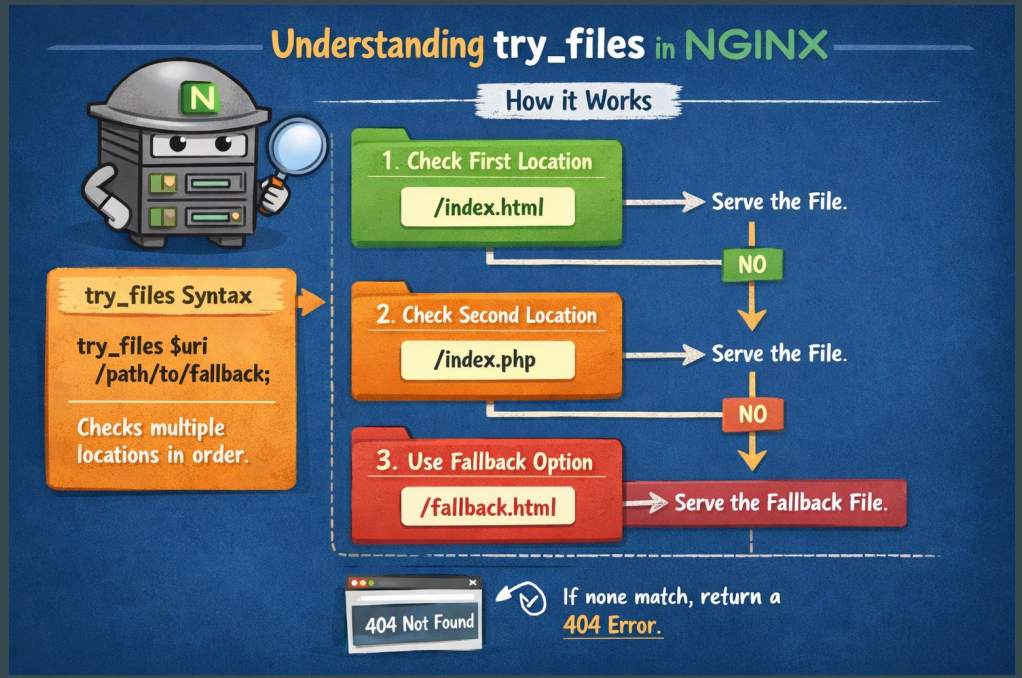


^/myimages	Matches any URI that begins with /myimages.
(.*)	It grabs everything that comes after /myimages (like /logo.png) and stores it in a variable called $1
/images$1	This tells Nginx to replace the match with /images followed by whatever was captured in the first group.

Trying Several Options (try_files)

Basics Architecture

The try_files directive can be used to check whether the specified file or directory exists; NGINX makes an internal redirect if it does, or returns a specified status code if it doesn't.



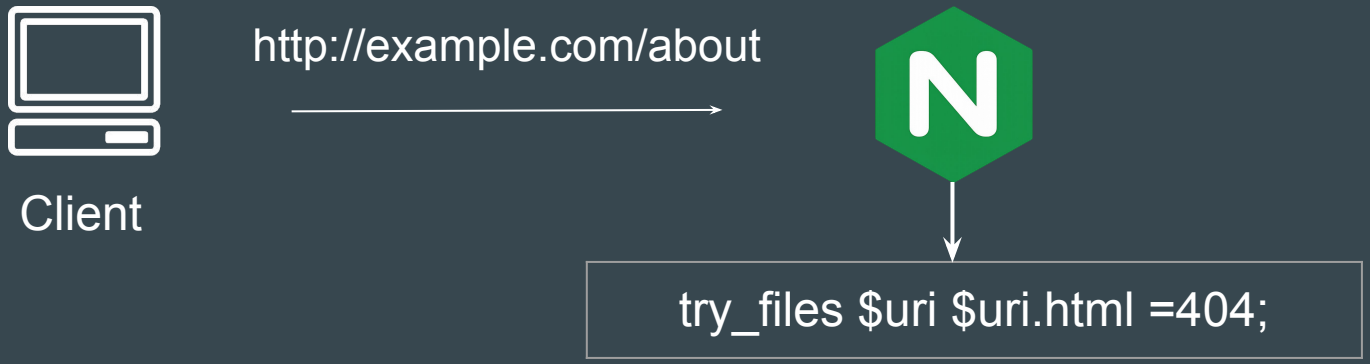
Basic Syntax

```
try_files file1 file2 ... fallback;
```



1. Nginx checks each fileN in order
2. If a match is found, it serves that file immediately.
3. If none match, it processes the final argument

Example 1



1. Check if there is a file at $uri (e.g., /about).
2. If not, try $uri.html (e.g., /about.html).
3. If neither exists → return HTTP status code of 404

Sample Configuration

```
server {
    listen       80;
    server_name  localhost;
    root         /usr/share/nginx/html;
    index        index.html index.htm;

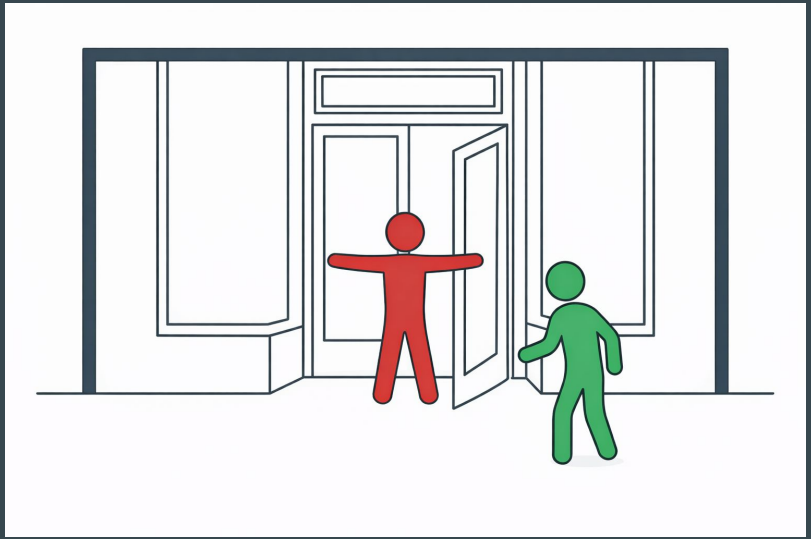
    location / {
        try_files $uri $uri.html =404;
    }
}
```

Denial of Service Attacks

Simple Analogy - Denial of Service

One person stands at your front door and refuses to move.

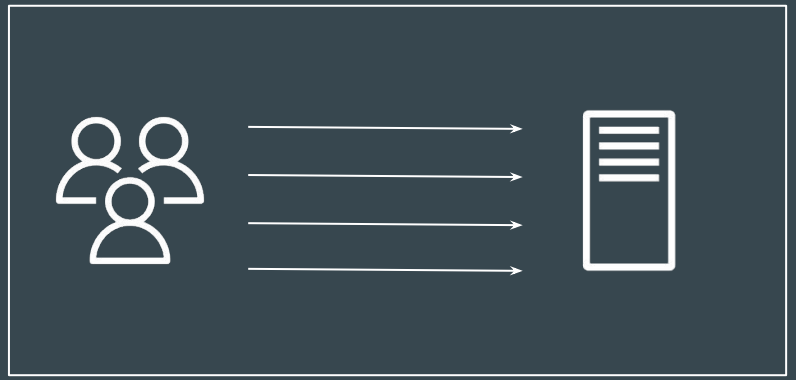
This person blocks actual customers from entering. It is annoying, but your security guard can easily identify this one person and remove them.



Basics of Denial of Service

Every system in the world has a maximum capacity.

Denial of Service (DoS) attack overwhelms a system with excessive traffic making it unavailable to legitimate users by exhausting its resources.



Legitimate Traffic

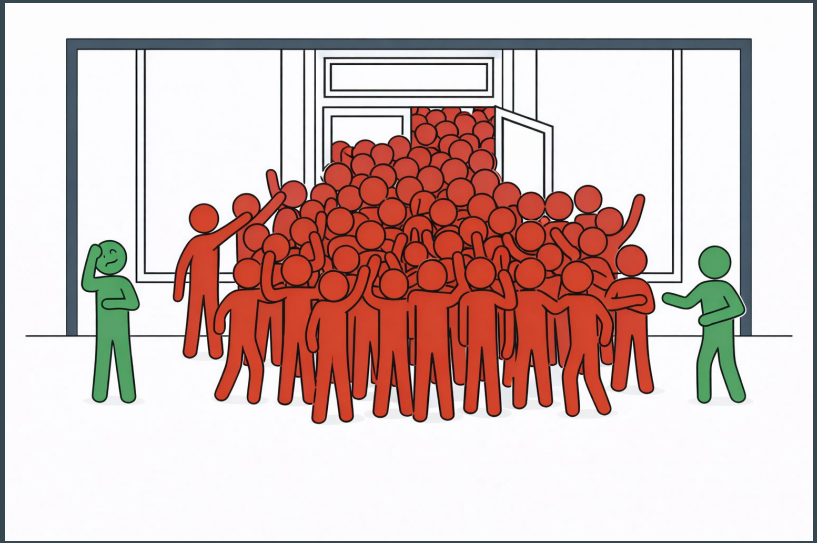


Denial of Service Attack

Simple Analogy - Distributed Denial of Service

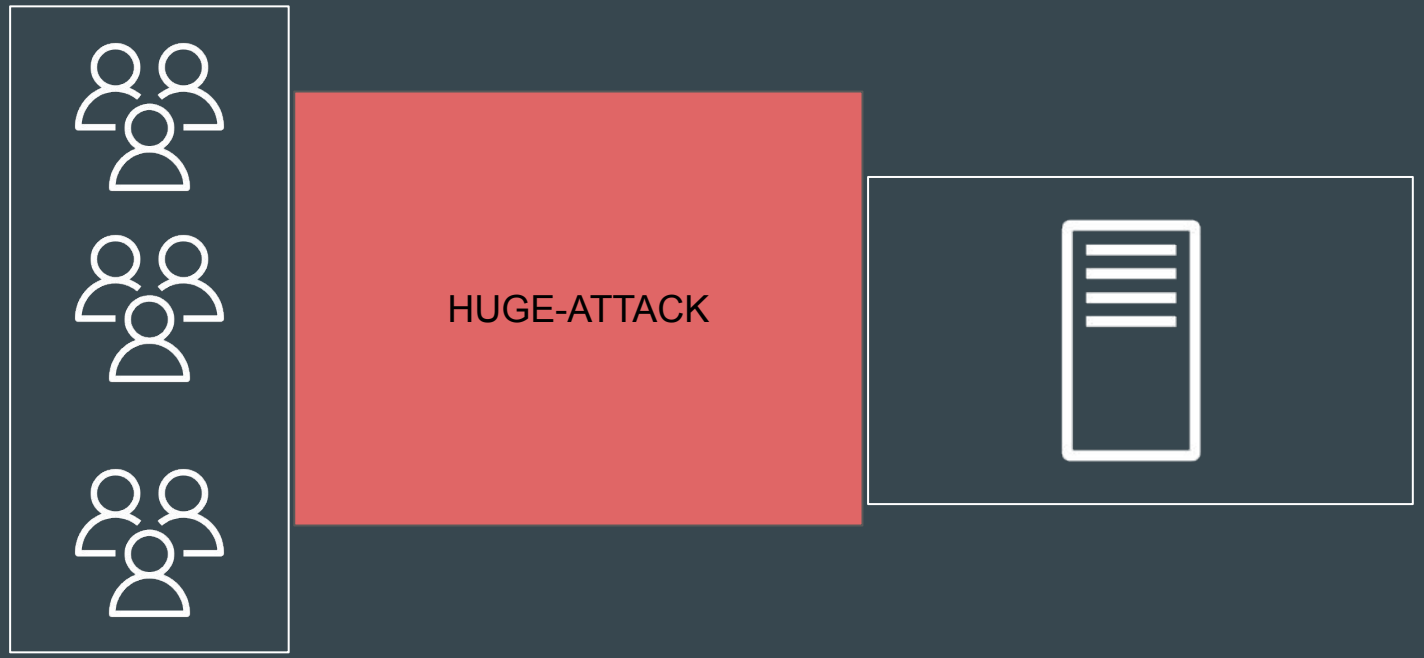
500 people are standing at your front door and refuses to move.

They block actual customers from entering. It is annoying, and your security guard is completely overwhelmed.

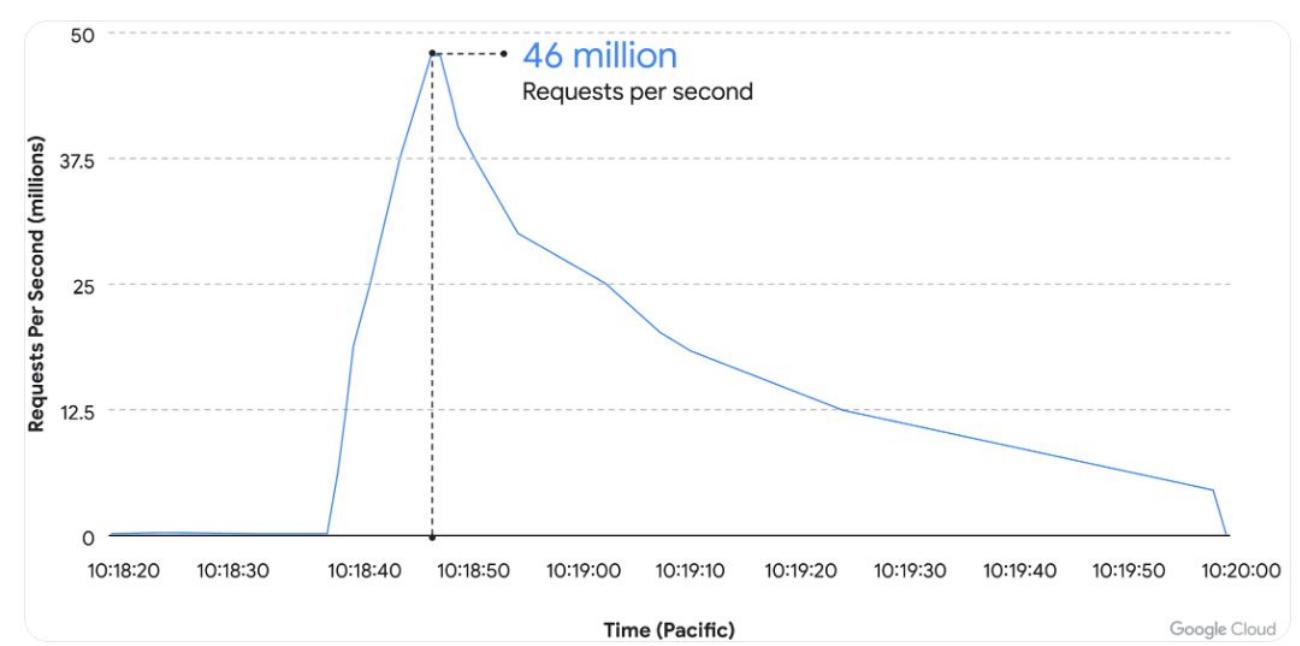


Basics of Distributed Denial of Service

A distributed denial-of-service (DDoS) attack is a malicious attempt to disrupt the normal traffic of a targeted server by overwhelming the target with a flood of Internet traffic from multiple systems.



In blog post of Google Cloud, it says that they managed to block a DDoS attack which peaked at 46 million requests per second



Tech Industry > Cyber Security

Massive DDoS attack delivered 37.4TB in 45 seconds, equivalent to 10,000 HD movies, to one victim IP address — Cloudflare blocks largest cyber assault ever recorded

News By Jowi Morales published June 21, 2025

This is the largest DDoS attack ever on record, so far.

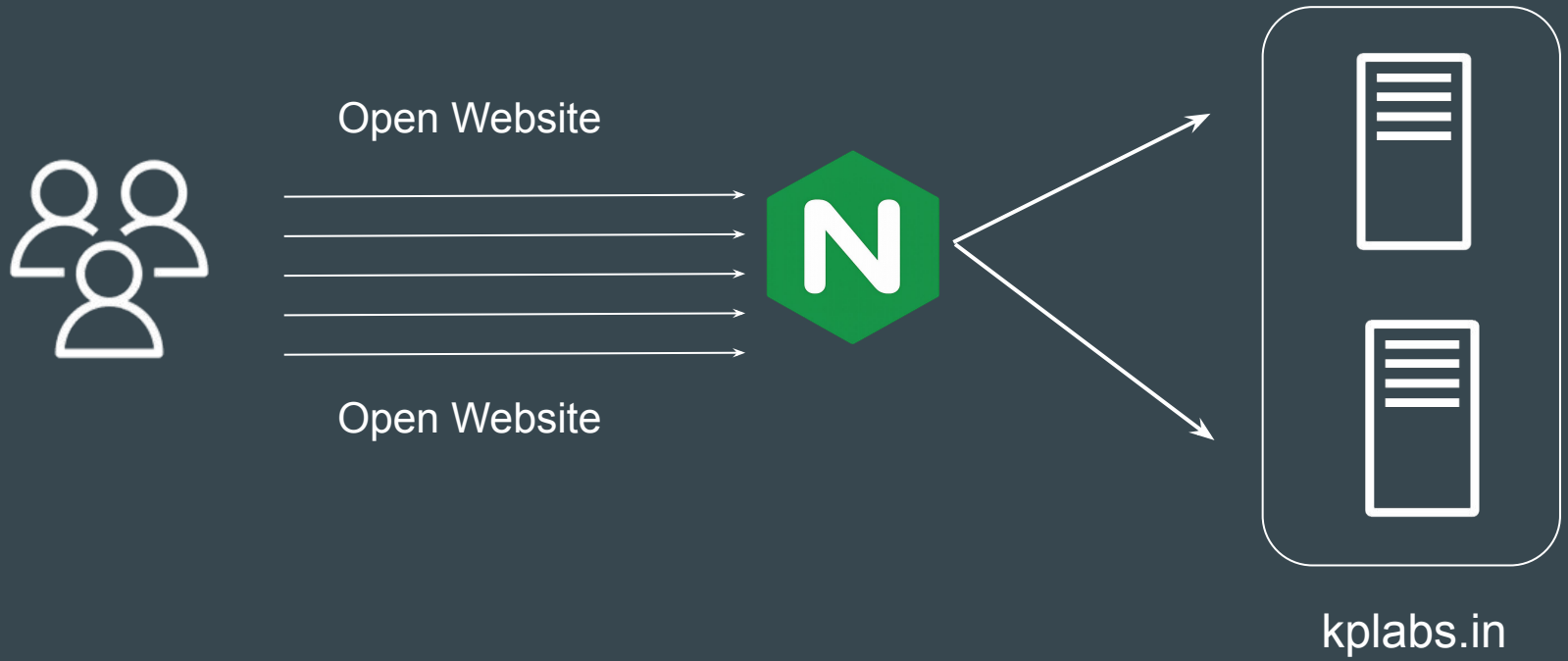


(Image credit: Shutterstock)

Whitelists and Blacklists

Setting the Base

If no allow or deny directives are not present, Nginx allows access to everyone by default.



Allow and Deny Directives

allow	Permits access for a specific IP, CIDR block, or all
deny	Blocks access for a specific IP, CIDR block, or all.

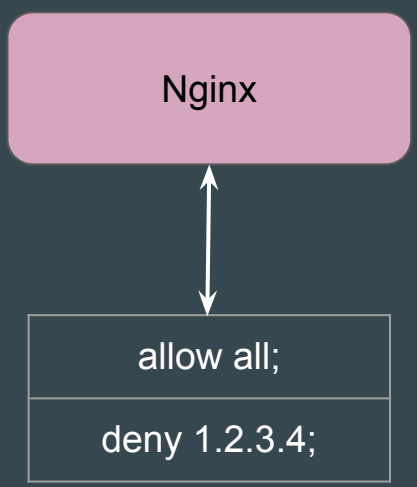


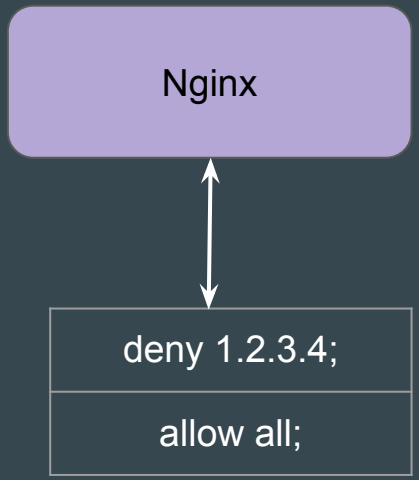
```
location /admin {
    allow 192.168.1.50;     # Your specific IP
    allow 10.0.0.0/24;      # Your local office network
    deny all;               # Block everyone else
}
```

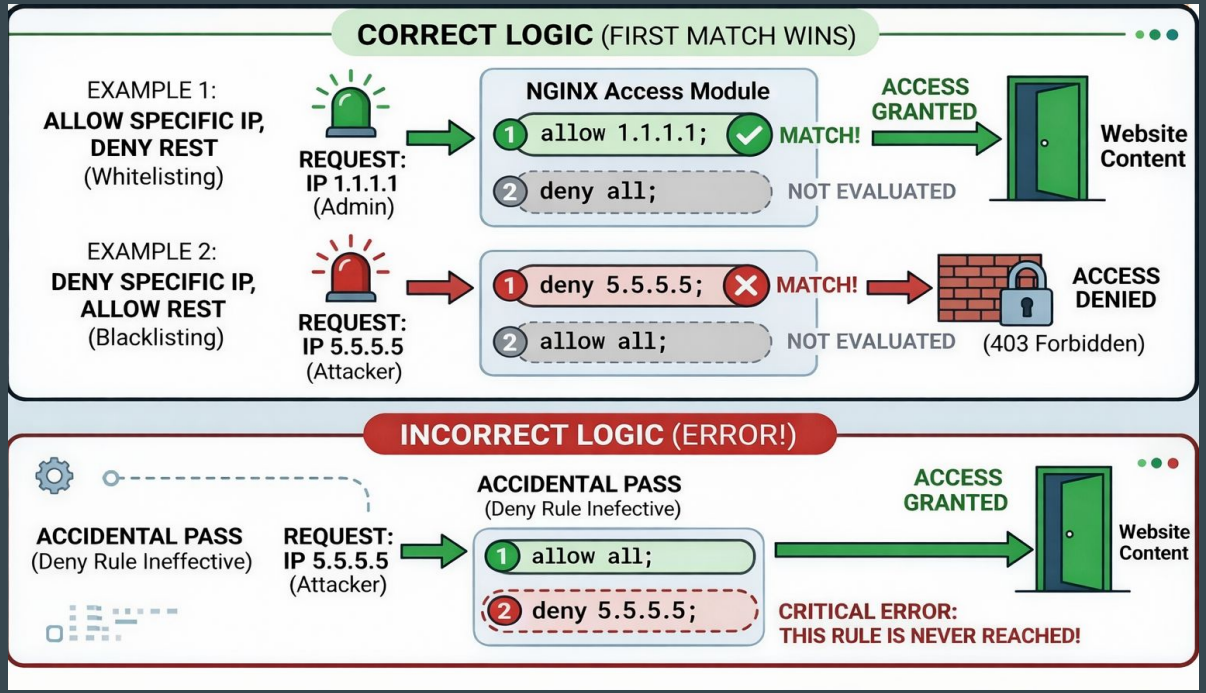
Important Note - Ordering in Whitelists/Blacklists

Ordering Matters

Nginx doesn't look at the whole list of rules to find the "best" match; it stops the moment it finds a rule that applies to the visitor.



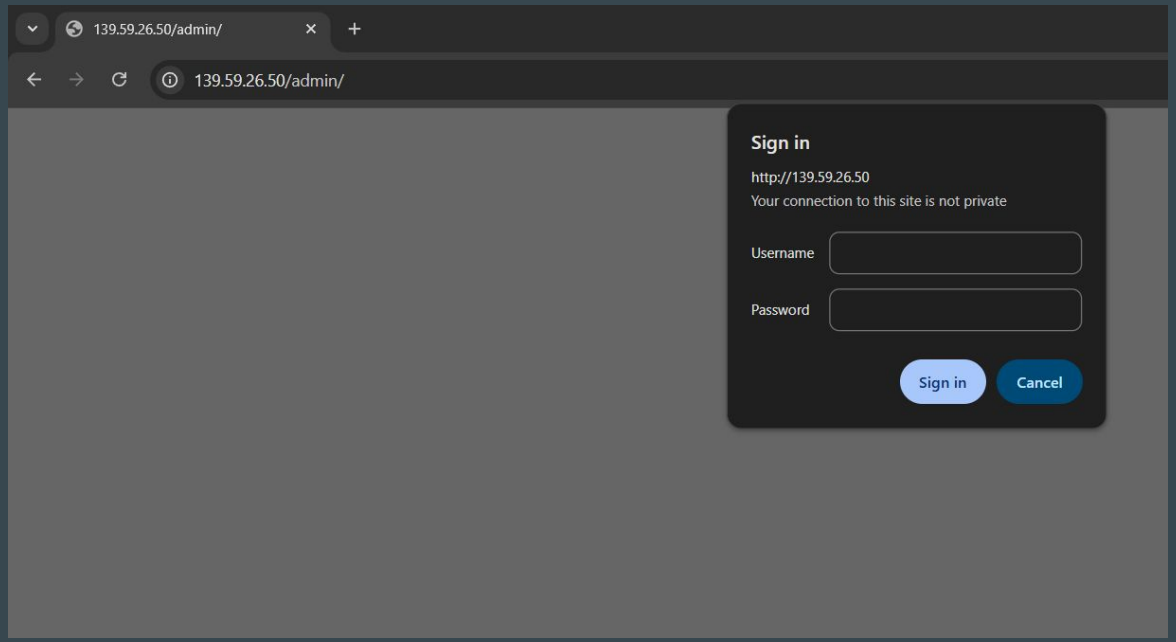



CORRECT LOGIC (FIRST MATCH WINS)
EXAMPLE 1:
ALLOW SPECIFIC IP, DENY REST
(Whitelisting)
REQUEST: IP 1.1.1.1 (Admin)
NGINX Access Module
1 allow 1.1.1.1;
MATCH!
2 deny all;
NOT EVALUATED
ACCESS GRANTED
Website Content
EXAMPLE 2:
DENY SPECIFIC IP, ALLOW REST
(Blacklisting)
REQUEST: IP 5.5.5.5 (Attacker)
1 deny 5.5.5.5;
MATCH!
2 allow all;
NOT EVALUATED
ACCESS DENIED
(403 Forbidden)
INCORRECT LOGIC (ERROR!)
ACCIDENTAL PASS
(Deny Rule Ineffective)
REQUEST: IP 5.5.5.5 (Attacker)
ACCIDENTAL PASS
(Deny Rule Inefective)
1 allow all;
2 deny 5.5.5.5;
ACCESS GRANTED
Website Content
CRITICAL ERROR:
THIS RULE IS NEVER REACHED!

Restricting Access with HTTP Basic Authentication

Setting the Base

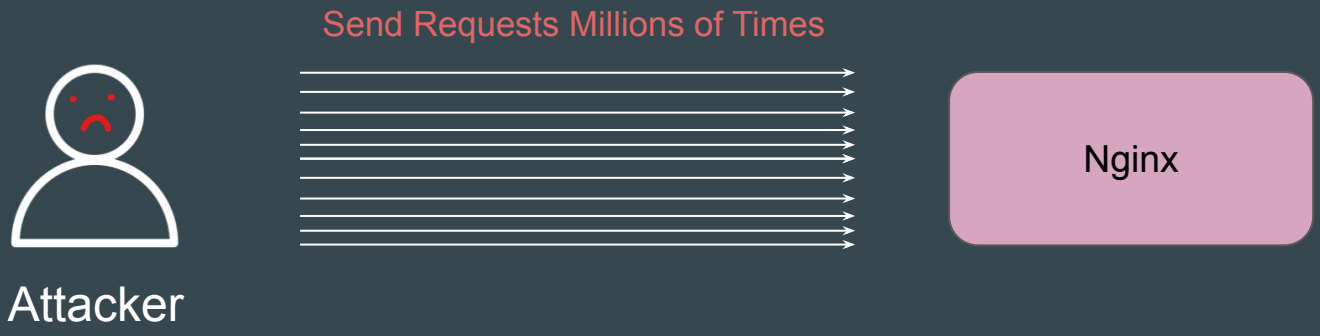
You can restrict access to your website or some parts of it by implementing a username/password authentication.



Rate Limiting with Nginx

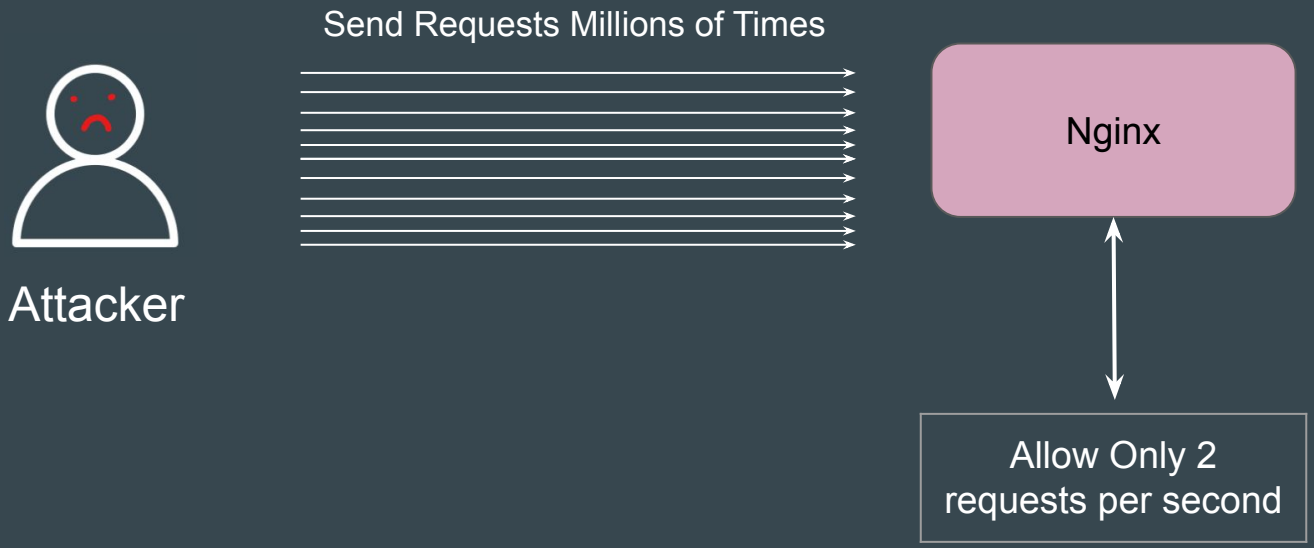
Understanding the Challenge

An attacker can send thousands of requests to your Nginx server in a short period of time to overwhelm the server, potentially leading to Denial-of-Service (DoS) conditions and resource exhaustion.



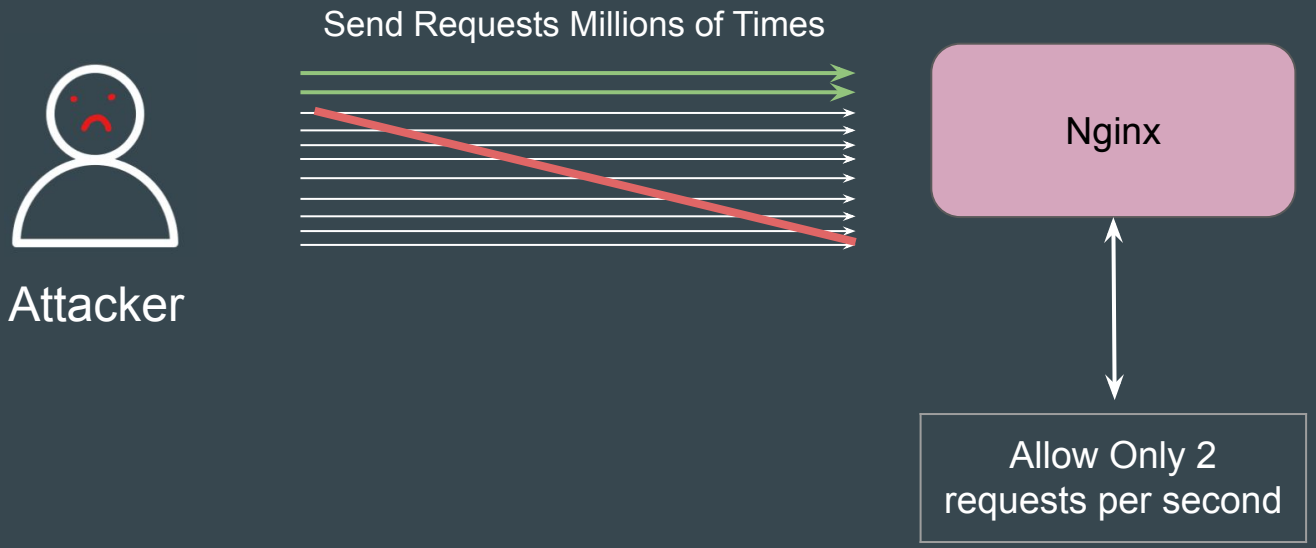
Setting the Base

Rate limiting can be used for security purposes, for example to slow down brute-force password-guessing attacks as well as protect against DDoS attacks.



Setting the Base

Rate limiting can be used for security purposes, for example to slow down brute-force password-guessing attacks as well as protect against DDoS attacks.



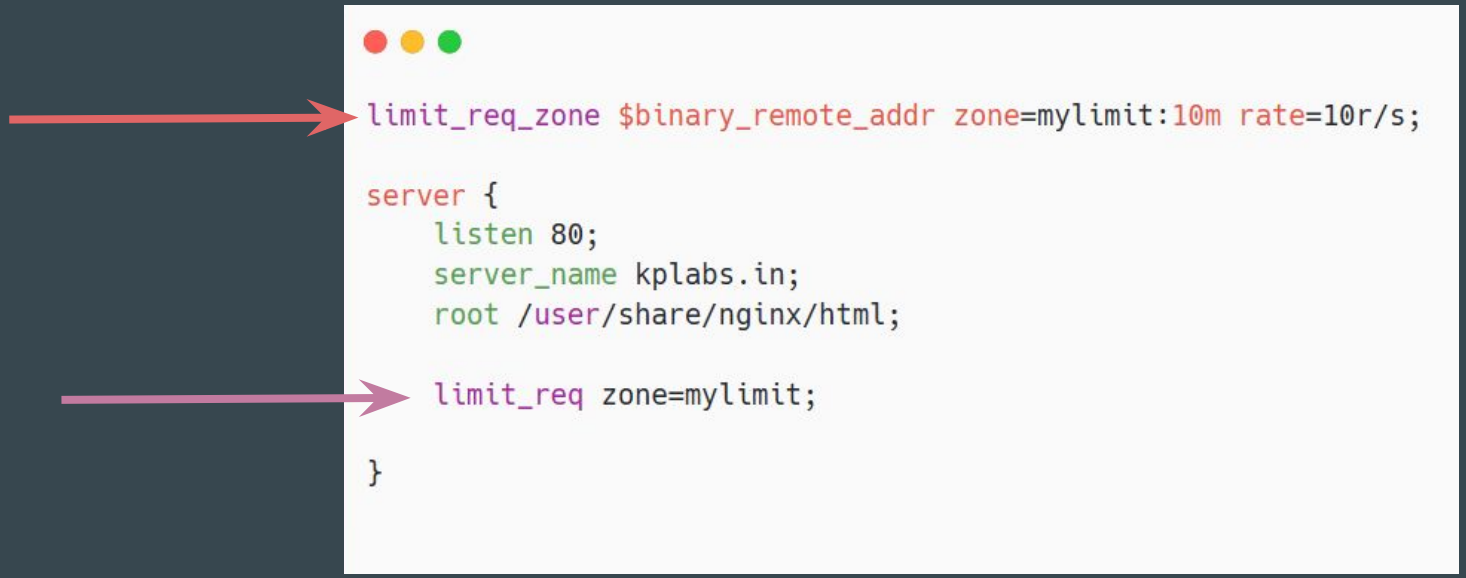
Configuration

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;
```



limit_req_zone	It tells Nginx: "I want to define a shared memory zone to track request rates."
$binary_remote_addr	This tells Nginx to track users by their IP address.
zone=mylimit:10m	The Storage Area. It creates a shared memory zone named mylimit. The 10m (10 Megabytes) is the size; 1MB can hold about 16,000 IP addresses, so 10MB can track roughly 160,000 unique IPs.
rate=10r/s	This sets the maximum rate to 10 requests per second.

Sample Configuration



Point to Note

What "10 requests per second" actually means in practice

1. It does not mean "allow the first 10 requests, then block".
2. It means "allow requests spaced out at 100 ms intervals on average".
3. If you want to allow a sudden burst (which is what most people expect), you must explicitly configure it.

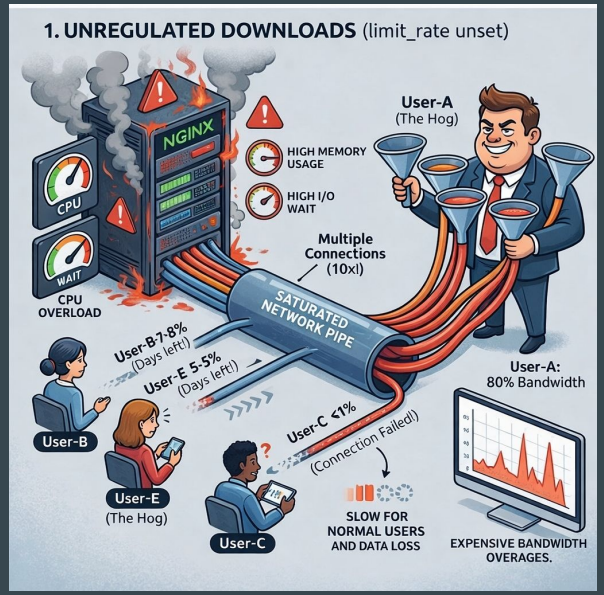

server {
listen 80;
server_name kplabs.in;
root /user/share/nginx/html;
limit_req zone=mylimit burst=15 nodelay;
}

Download Speed Throttling in Nginx

Unregulated Setup

User-A has opened multiple simultaneous connections (10x), saturating the server and taking 80% of the available bandwidth.

Other users are left with tiny, lagging streams.



Setting the Base

An unmanaged server is often a chaotic one. Without a limit_rate defined, Nginx will attempt to serve every file as fast as the network interface allows.

The limit_rate directive defines the maximum transmission speed for a connection.

```
location /downloads/ {
    limit_conn addr 1;    # Only 1 connection per IP
    limit_rate 500k;      # Limits speed to 500 KB/s per connection
}
```

Point to Note

Nginx allows you to set the rate based on variables.

For example, you could give "Premium" users faster speeds than "Free" users by checking a cookie or a header.

Modular Architecture

Basics of Modular Architecture

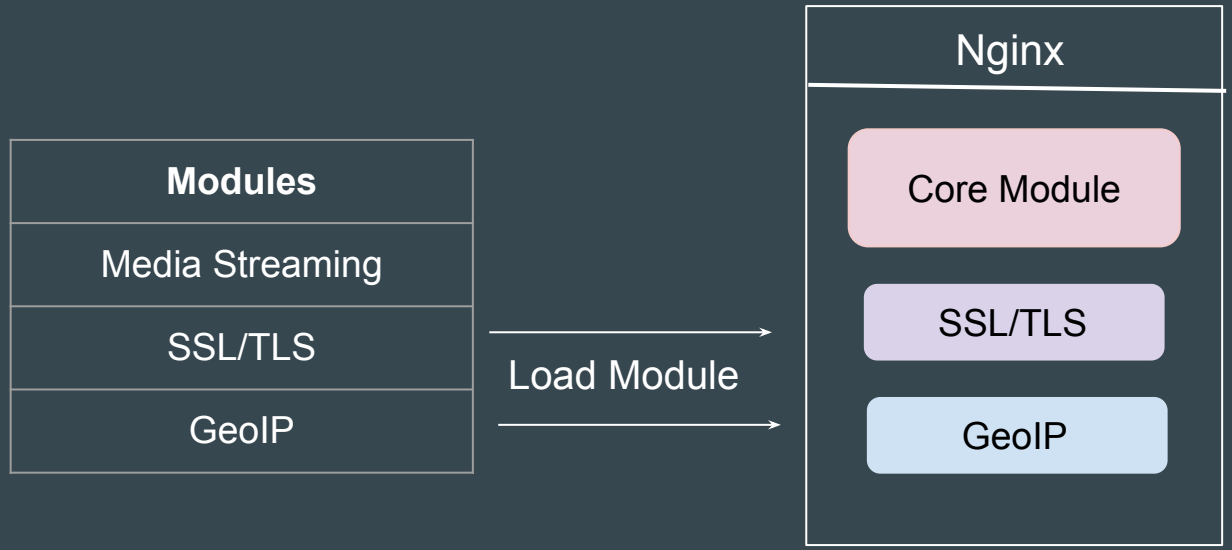
Modular Architecture refers to the design of any system composed of separate components that can be connected together.



Setting the Base

NGINX is actually a collection of modules.

Even very basic functions like HTTP, or serving static files inside HTTP – these basic things are modules.



Point to Note

You can extend NGINX by adding your own modules.

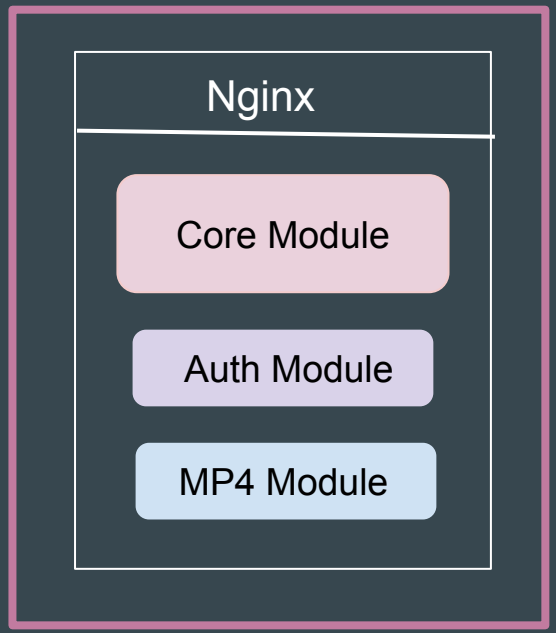
The modular architecture allows you to easily modify NGINX.

It’s what makes it so good, and so successful.

Static Modules

Static modules are compiled directly into the Nginx binary at build time.

To add or remove a module, you must recompile the entire Nginx source code.



Complete Nginx Package

Dynamic Modules

A dynamic module is compiled as a separate .so file (shared object) and must be loaded in your nginx.conf using the load_module directive.

Modules
Media Streaming
SSL/TLS
GeoIP



```
load_module modules/ngx_rtmp_module.so;          # Media Streaming (RTMP/HLS/DASH)
load_module modules/ngx_http_ssl_module.so;      # SSL/TLS termination
load_module modules/ngx_http_geoip_module.so;    # GeoIP country/city lookup

pid        /var/run/nginx.pid;
user       nginx;
worker_processes  auto;

events {
    worker_connections  1024;
}
```

Revision - Static vs Dynamic Modules

Feature	Static Modules	Dynamic Modules
Compilation	Compiled into the main nginx binary.	Compiled as separate Shared Object (.so) files.
Loading Method	Hardcoded; active the moment Nginx starts.	Loaded via the load_module directive in nginx.conf.
Flexibility	Low. Requires full recompilation to add/remove.	High. Can be enabled/disabled without recompilation.
Performance	Marginally faster (direct internal calls).	Negligible overhead (uses a shared object interface).
Use Case	Core features (Gzip, SSL, Proxy, Headers).	Optional/Heavy features (Image Filter, GeoIP, Lua, Brotli).

Web Application Firewalls

Simple Analogy

Imagine a security guard letting everyone into a concert as long as the ticket is valid.

He does not check what they're bringing inside their bags.

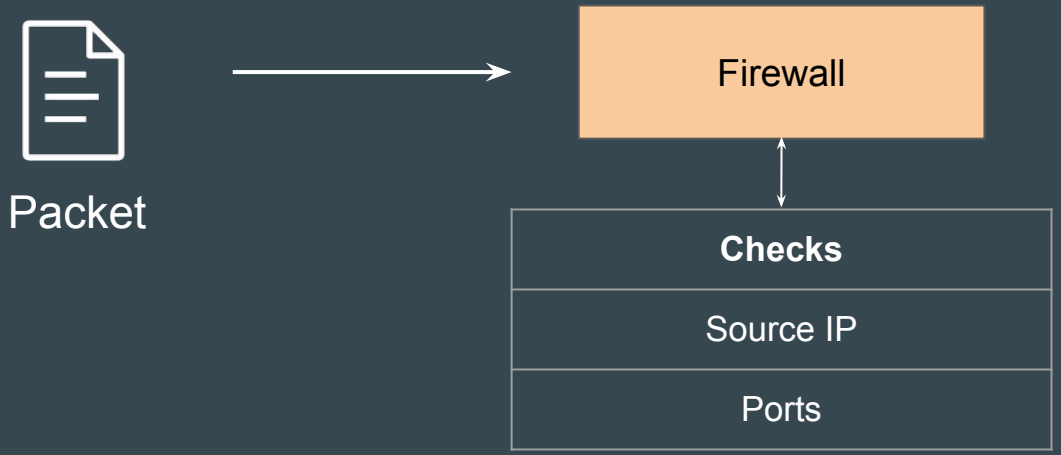
If someone brings in something dangerous hidden in their bag, the guard won’t notice.



Basics of Network-Level Firewalls

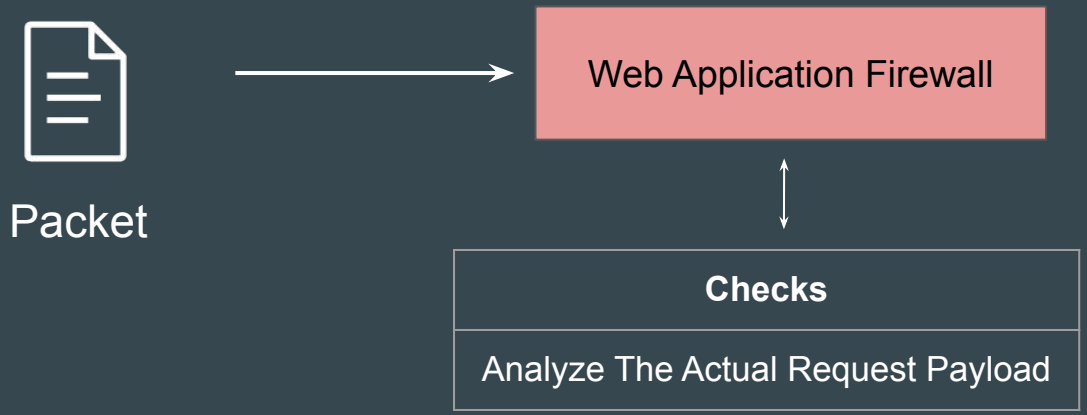
Network-level firewalls primarily checks the basic details like the (source IP) and where they want to go (destination IP and port).

These firewalls don't see inside the actual request.



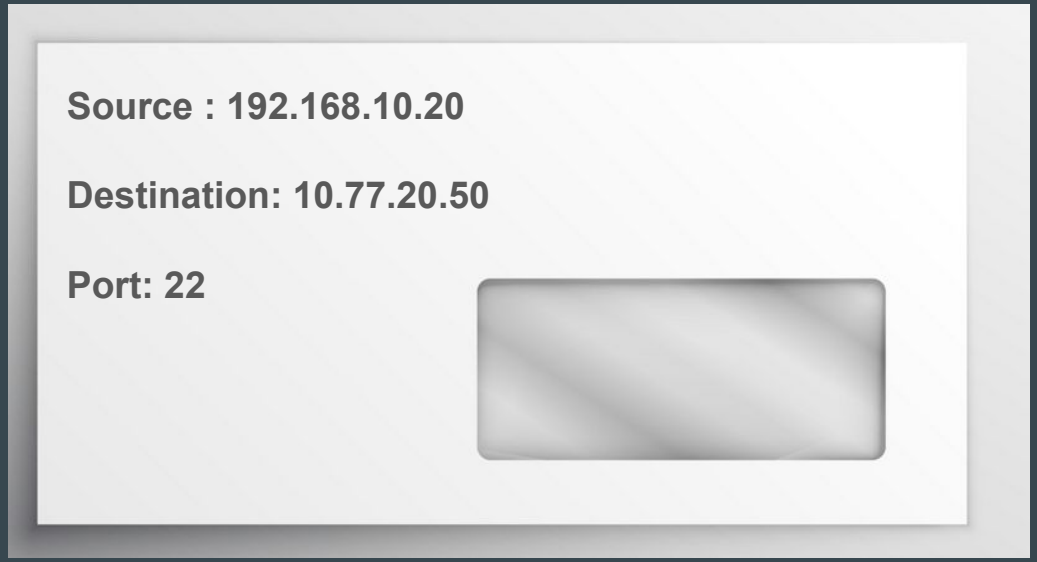
Web Application Firewall

A Web Application Firewall looks inside the web requests to identify the malicious code in the web request.



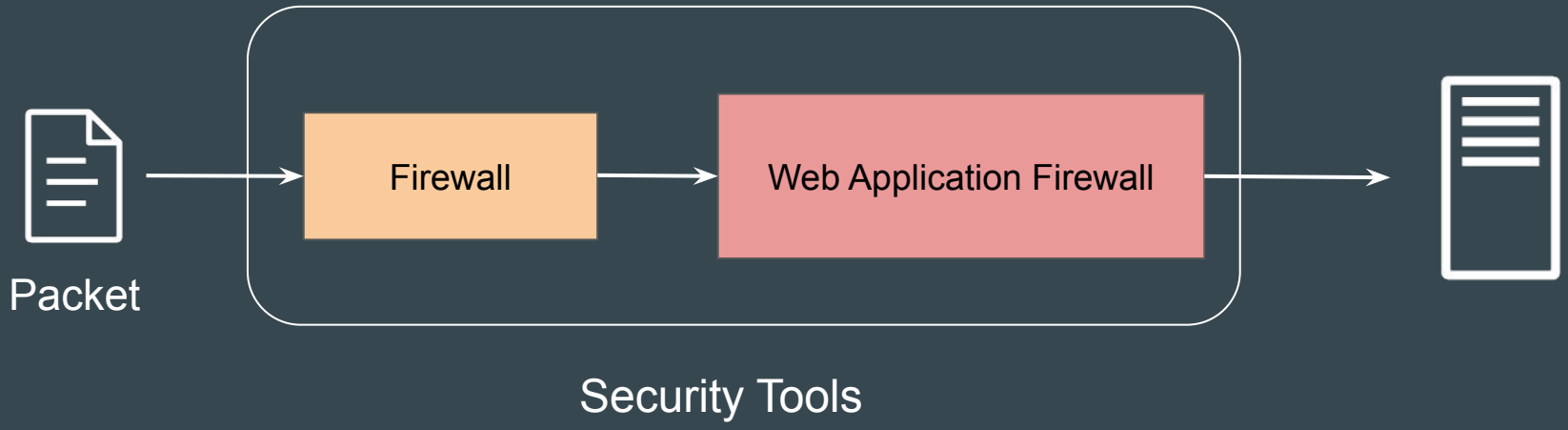
Example Analogy - Envelope

Network level Firewall reads the basic details from outside of envelope to finalize however WAF opens the envelope and, analyzes the inner contents of the envelope



Workflow Architecture

In a typical architecture, you will use Firewall and WAF to analyze traffic before sending it to the backend application.



WAF Provider Offerings

Modsecurity is one of the very popular open source web application firewall.

There are various other popular offerings from CloudFlare, Akamai, AWS and other providers.





AWS WAF

Asymmetric Key Encryption

Right Architecture is the Key

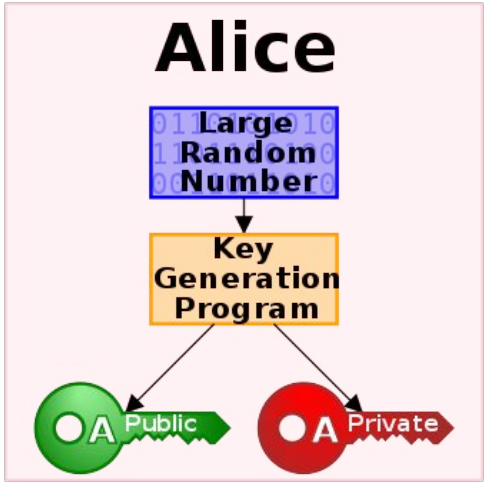
Overview of Asymmetric Key Encryption

Asymmetric cryptography, uses public and private keys to encrypt and decrypt data.

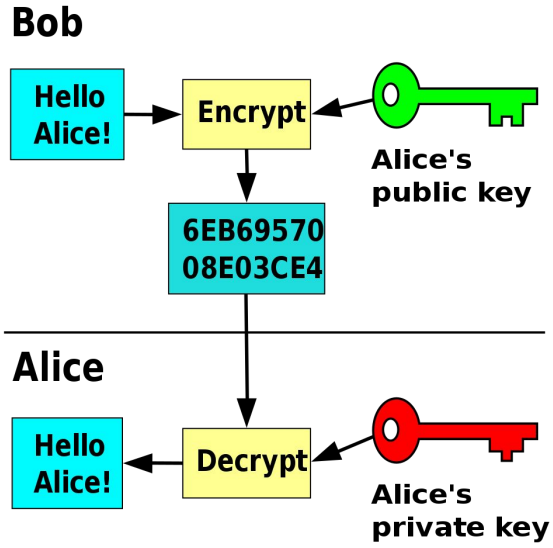
One key in the pair can be shared with everyone; it is called the public key. The other key in the pair is kept secret; it is called the private key.

Either of the keys can be used to encrypt a message; the opposite key from the one used to encrypt the message is used for decryption.

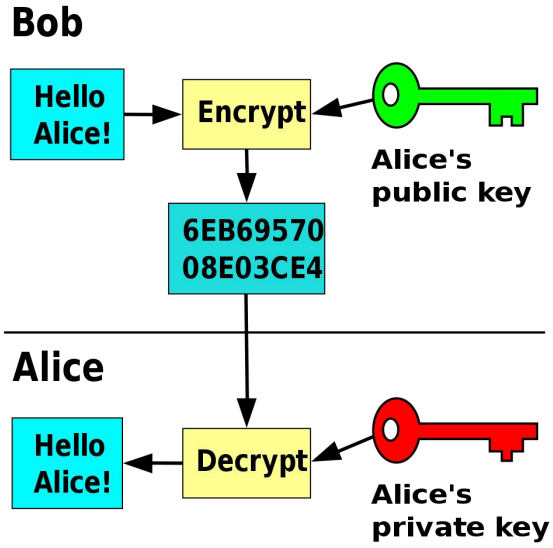
Step 1: Generation of Keys



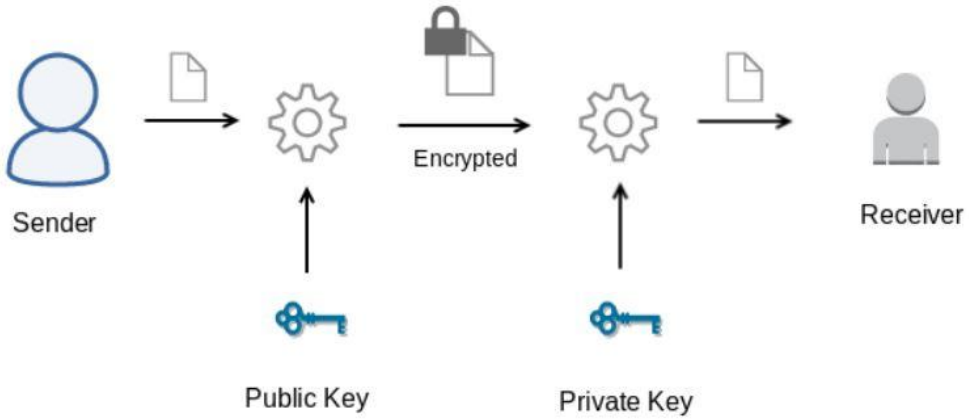
Step 2: Encryption and Decryption



Step 2: Encryption and Decryption

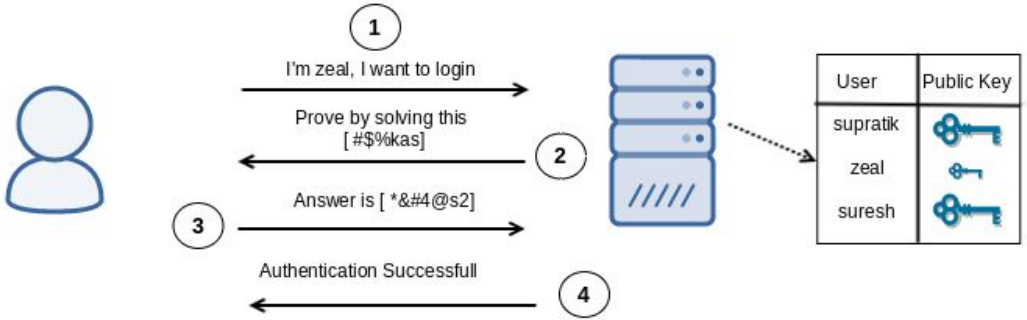


Step 2: Encryption and Decryption



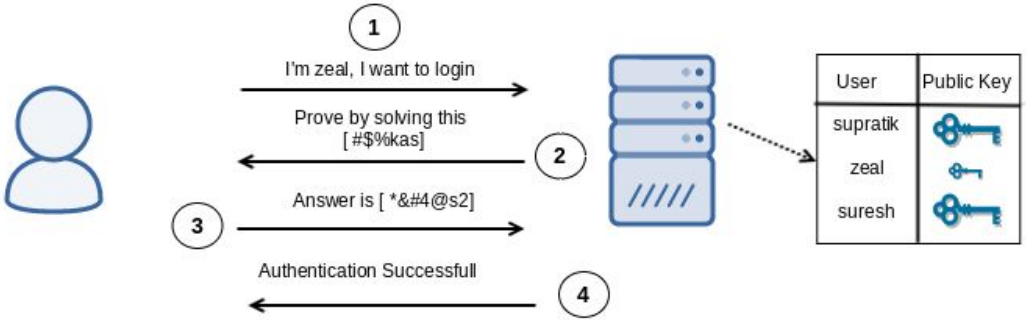
Use-Case of Asymmetric Key Encryption - Step 1

User zeal wants to log in to the server. Since the server uses a public key authentication, instead of taking the password from the user, the server will verify if the User claiming to be zeal actually holds the right private key.



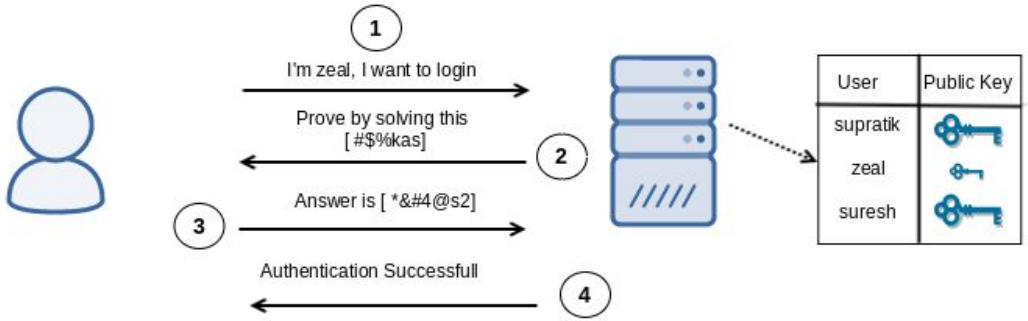
Use-Case of Asymmetric Key Encryption - Step 2

The server creates a simple challenge, 2+3=? and encrypts this challenge with the Public Key of the User and sends it back to the User. The challenge is sent in an encrypted format.



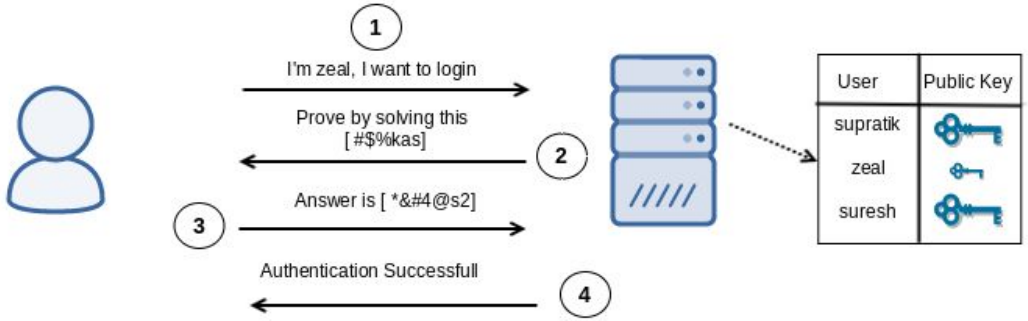
Use-Case of Asymmetric Key Encryption - Step 3

Since the user zeal holds the associated private key, he will be able to decrypt the message and compute the answer, which would be 5. Then, he will encrypt the message with the private key and send it back to the server.



Use-Case of Asymmetric Key Encryption - Step 4

The server decrypts the message with the user's Public Key and checks if the answer is correct. If yes, then the server will send an Authentication Successful message and the user will be able to log in.



Protocols

Because of the advantage that it offers, Asymmetric key encryption is used by variety of protocols.

Some of these include:

- PGP
- SSH
- Bitcoin
- TLS
- S/MIME

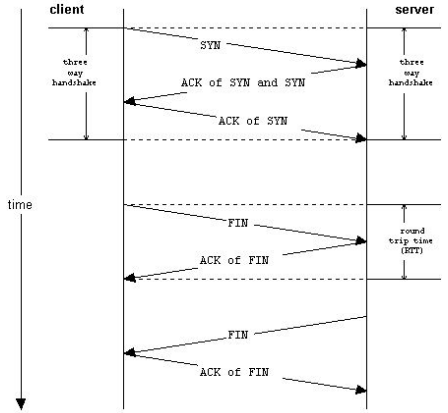
Plain Text vs Encrypted Protocols

Communication

Various Protocols

There are various protocols actively used :

- File Transfer Protocol (FTP)
- Domain Name System Protocol (DNS)
- Transmission Control Protocol (TCP)
- Secure File Transfer Protocol (SFTP)
- Hyper Text Transfer Protocol (HTTP)
- Internet Protocol (IP)



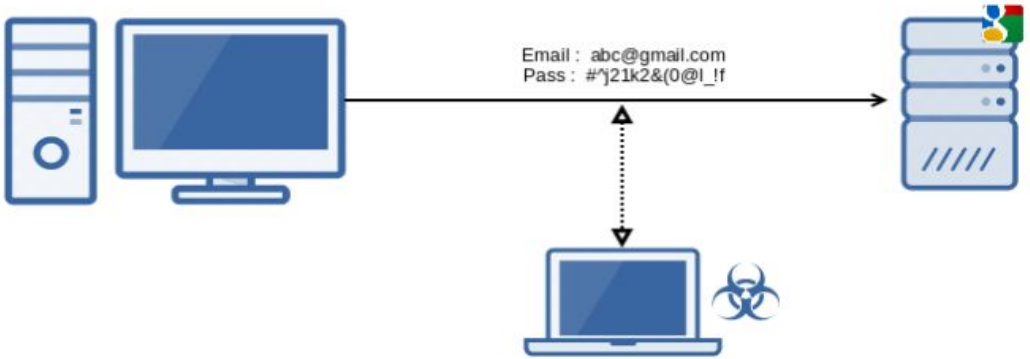
Time line of TCP client-server connection establishment and termination.


SANJIBVIDEO

2 computers want to talk to each other



After Encryption



HTTPS

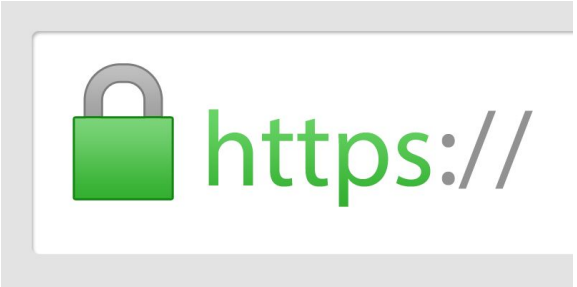
Secure Communication

Overview of HTTPS

HTTPS is an extension of HTTP.

In HTTPS, the communication is encrypted using Transport Layer Security (TLS)

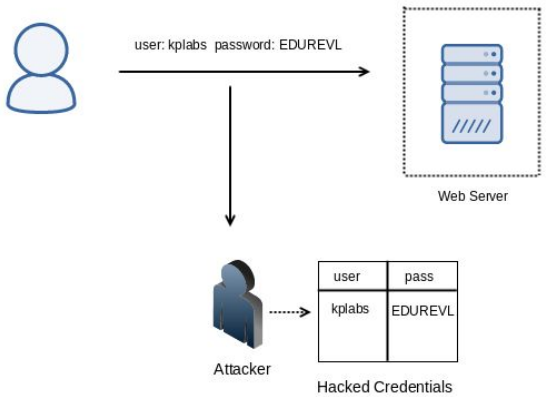
The protocol is therefore also often referred to as HTTP over TLS or HTTP over SSL.



Scenario 1: MITM Attacks

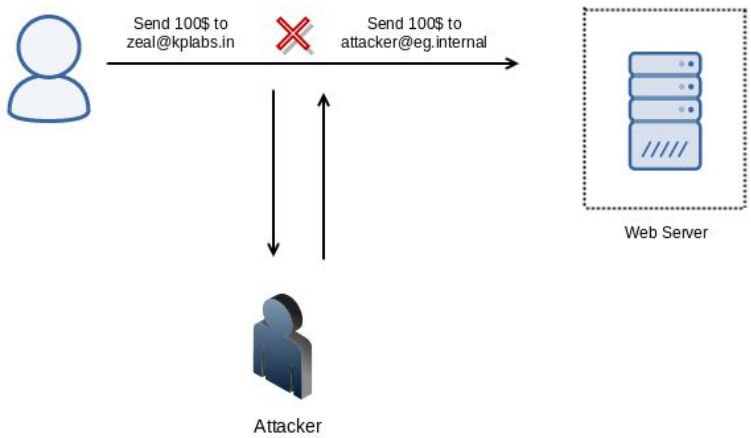
- User is sending their username and password in plaintext to a Web Server for authentication over a network.

- There is an Attacker sitting between them doing a MITM attack and storing all the credentials he finds over the network to a file:



Scenario 2: MITM & Integrity Attacks

- Attacker changing the payment details while packets are in transit.



Introduction to SSL/TLS

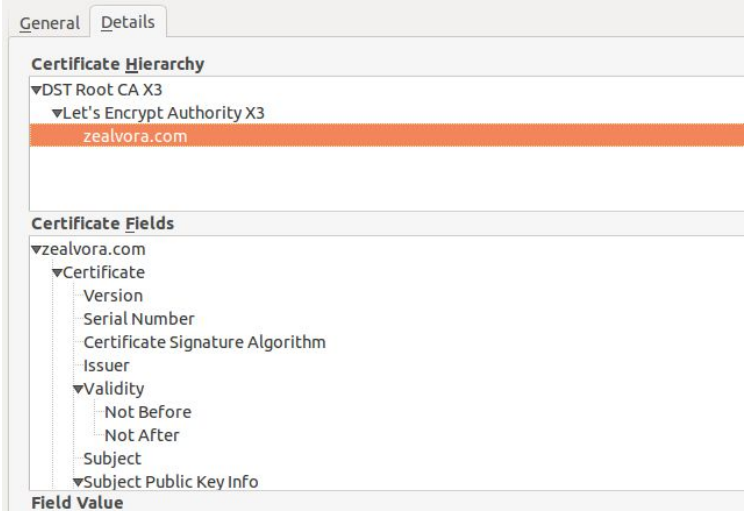
To avoid the previous two scenarios (and many more), various cryptographic standards were clubbed together to establish a secure communication over an untrusted network and they were known as SSL/TLS.

Protocol	Year
SSL 2.0	1995
SSL 3.0	1996
TLS 1.0	1999
TLS 1.1	2006
TLS 1.2	2008
TLS 1.3	2018

Understanding it in easy way

Every website has a certificate (like a passport which is issued by a trusted entity).

Certificate has lot of details like domain name it is valid for, the public key, validity and others.



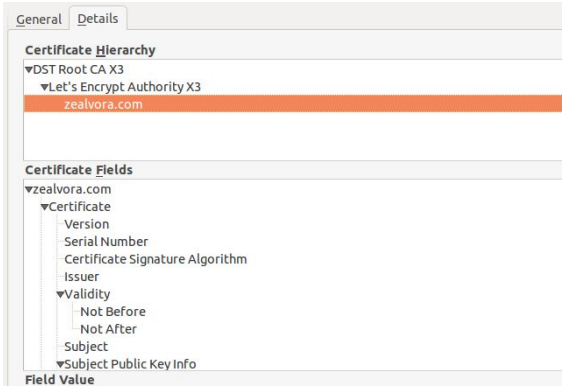
Understanding it in easy way

Browser (clients) verifies if it trusts the certificate issuer.

It will verify all the details of the certificate.

It will take the public key and initiate a negotiation.

Asymmetric key encryption is used to generate a new temporary symmetric key which will be used for secure communication.



Web Server Configuration

```
server {
       listen         80;
       server_name    zealvora.com;
       return         301 https://$server_name$request_uri;
}

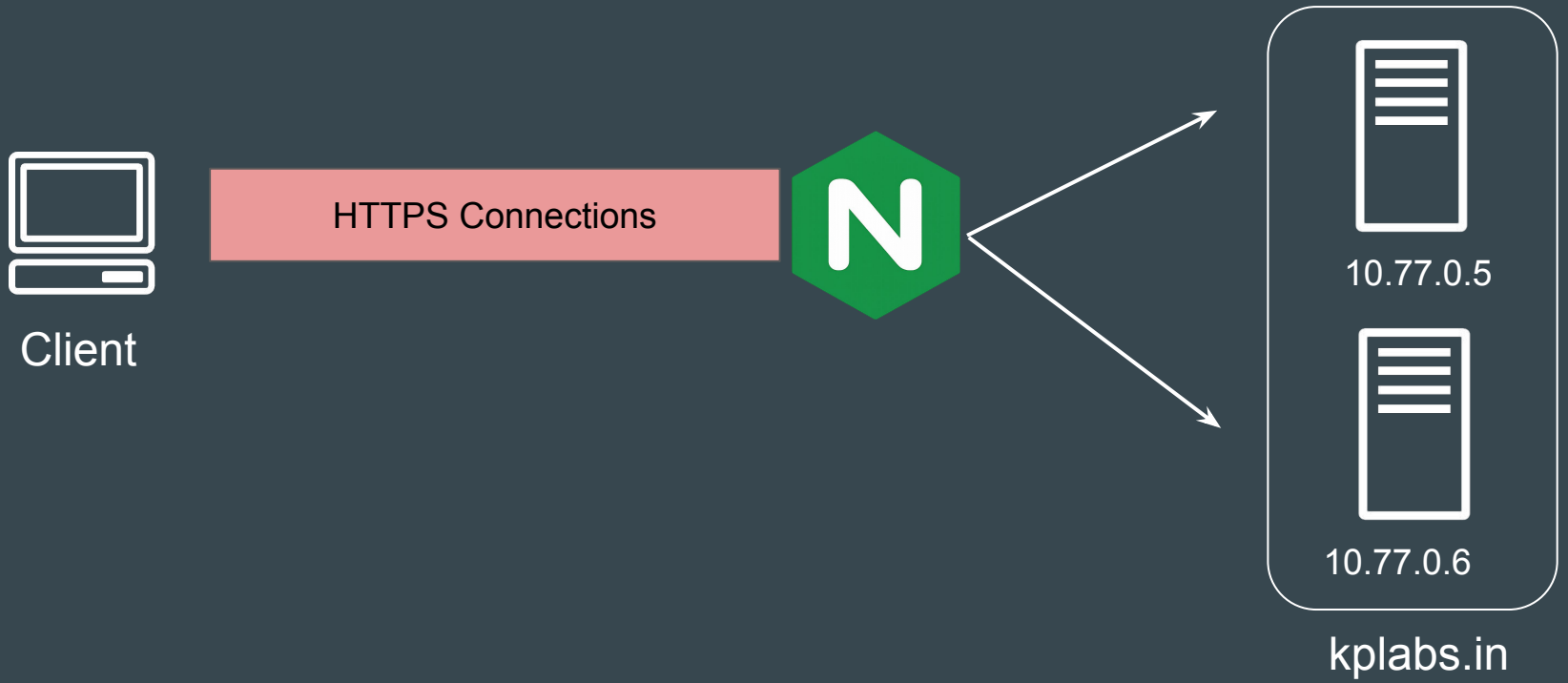

server {
  server_name zealvora.com;
  listen 443 default ssl;
  server_name zealvora.com;
  ssl_certificate /etc/letsencrypt/archive/zealvora.com/fullchain1.pem;
  ssl_certificate_key /etc/letsencrypt/archive/zealvora.com/privkey1.pem;

  location / {
    root /websites/zealvora/;
    include location-php;
    index index.php;
 }
 location ~ /.well-known {
        allow all;
 }
}
```

Configuring HTTPS servers

Basics Architecture

In this architecture, the encrypted connection terminates at Nginx level.



Configuration Directives

To configure an HTTPS server, the ssl parameter must be enabled on listening sockets in the server block, and the locations of the server certificate and private key files should be specified.

```
server {
    listen              80;
    listen              443 ssl;
    server_name         www.example.com;
    ssl_certificate     www.example.com.crt;
    ssl_certificate_key www.example.com.key;
    ...
}
```